



Hochschule Karlsruhe
University of Applied Sciences

Fakultät für Informatik und Wirtschaftsinformatik
Wirtschaftsinformatik

BACHELORTHESIS

Entwicklung einer digitalen Übungsplattform für mathematische Inhalte im Bereich des logischen Schließens

von	Sinan Kammerer
Matrikelnr.	83431
Arbeitsplatz	Hochschule Karlsruhe
Erstbetreuer*in	Prof. Dr. Reimar Hofmann
Zweitbetreuer*in	Prof. Dr. Stefanie Fassbender
Abgabetermin	31.03.2026

Karlsruhe, 31.03.2026

Vorsitzender des Prüfungsausschusses

Eidesstattliche Erklärung

Ich versichere, dass ich die vorliegende Bachelorarbeit selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht. Ich erkläre hiermit, dass bei der Erstellung dieser Arbeit generative KI-Systeme (Large Language Models) ausschließlich als Hilfsmittel zur sprachlichen Optimierung, zur Umformulierung eigener Entwürfe sowie zur Unterstützung bei der Strukturierung von Textabschnitten eingesetzt wurden.

Die Arbeit hat in gleicher oder ähnlicher Fassung noch keiner anderen Prüfungsbehörde vorgelegen.

Karlsruhe, 31.03.2026

Sinan Kammerer

Zusammenfassung

Aussagenlogik und logisches Schließen zählen zu den methodischen Grundlagen informatiknaher Studiengänge, bereiten Studierenden jedoch häufig Schwierigkeiten. Bestehende digitale Übungssysteme für formale Logik adressieren überwiegend die schrittweise Beweisführung mittels Inferenzregeln; das Bestimmen von Variablenbelegungen aus gegebenen Prämissenmengen – also die Frage, welche Aussagenwerte sich aus einer Formelmenge zwingend, gar nicht oder nicht eindeutig ableiten lassen – wird bislang nicht als eigenständiger Aufgabentyp behandelt. Die vorliegende Arbeit konzipiert und implementiert eine webbasierte Übungsplattform, die diesen Aufgabentyp für zwei Formen des logischen Schließens abbildet: direktes Schließen und Schließen mithilfe einer Fallunterscheidung. Aufgaben werden regelbasiert über einen Generate-and-Filter-Ansatz mit aufgabentypspezifischen Validierungskriterien erzeugt. Die korrekte Lösung jeder Aufgabe wird algorithmisch durch einen Deletion-Algorithmus auf Basis boolescher Tabellen bestimmt. Die Plattform umfasst ein dreistufiges Antwortschema, eine Feedbackkomponente mit Gegenbeispielkonstruktion sowie einen gamifizierten Lernpfad mit serienbasiertem Levelaufstieg. Die Architektur trennt fachliche Kernlogik (Python, SymPy, NumPy), Vermittlungsschicht (Django) und Präsentationsschicht (React, TypeScript). Die Evaluation zeigt eine Fehlerrate von 0 % bei der Solver-Verifikation gegen SymPy über 500 Aufgaben. Die Akzeptanzraten der Aufgabengenerierung variieren zwischen 19,4 % und 0,01 % je nach Aufgabentyp und Schwierigkeitsstufe; eine Vorgenerierungsstrategie kompensiert die daraus resultierenden Generierungsdauern. Eine empirische Erprobung mit Studierenden steht aus und bildet den nächsten Untersuchungsschritt.

Inhaltsverzeichnis

Abbildungsverzeichnis	8
Tabellenverzeichnis	9
1 Einleitung	10
1.1 Motivation und fachliche Relevanz	10
1.2 Problemstellung	11
1.3 Zielsetzung und Beitrag	12
1.4 Abgrenzung der Arbeit	13
2 Theoretische Grundlagen	15
2.1 Fachliche Grundlagen der Aussagenlogik	15
2.1.1 Grundbegriffe der Aussagenlogik	15
2.1.2 Wahrheitswerte und Belegungen	17
2.1.3 Modelle und erfüllende Belegungen	18
2.1.4 Logische Folgerung aus einer Prämissenmenge	18
2.1.5 Konjunktive Normalform und Unit Propagation	19
2.2 Formen des logischen Schließens	20
2.2.1 Direktes logisches Schließen	20
2.2.2 Schließen mithilfe einer Fallunterscheidung . .	21
2.3 Grundlagen des solverbasierten Lösungsverfahrens .	21
2.3.1 Boolesche Tabellen als Darstellung zulässiger Belegungen	21
2.3.2 Grundoperationen auf booleschen Tabellen . .	22
2.3.3 Forward Pass	25
2.3.4 Backward Pass	26
3 Stand der Forschung und verwandte Ansätze	27

3.1	Digitale Übungs- und Tutorsysteme für formale Logik	27
3.2	Einsatz generativer Sprachmodelle in Lern- und Feedbackszenarien	28
3.3	Einordnung der vorliegenden Arbeit	29
4	Konzeption und Entwurf	31
4.1	Aufgabentypen und Schwierigkeitsmodell	31
4.1.1	Schwierigkeitsparameter	32
4.1.2	Regelbasiert statt adaptiv	33
4.1.3	Levelstruktur und Aufteilung der Schwierigkeitsstufen	33
4.2	Aufgabengenerierung	34
4.2.1	Generate-and-Filter-Ansatz	34
4.2.2	Validierungskriterien für Direct Inference . . .	34
4.2.3	Validierungskriterien für Case Split	36
4.2.4	Begründung des Generierungsansatzes	36
4.2.5	Vorgenerierung	37
4.3	Wahl und Anpassung des Lösungsverfahrens	37
4.3.1	Anforderungen an das Lösungsverfahren . . .	38
4.3.2	Abgrenzung gegenüber alternativen Verfahren	38
4.3.3	Begründung der Verfahrenswahl	39
4.3.4	Anpassung gegenüber dem Literaturansatz . .	40
4.4	Eingabe- und Bewertungskonzept	40
4.4.1	Gestaltung der Eingabe	41
4.4.2	Abgrenzung gegenüber alternativen Eingabeformen	41
4.4.3	Bewertungslogik	42
4.5	Feedbackkonzept	42

4.5.1 Fehlertypen	42
4.5.2 Feedbackverhalten pro Fehlertyp	43
4.5.3 Einordnung und Begrenzung	44
4.6 Lernpfad und Progression	44
4.6.1 Stufenmodell: Geführter Lernpfad und Freies Üben	44
4.6.2 Levelaufstieg und Schwellenwerte	44
4.6.3 Ergänzende didaktische Elemente	45
4.7 Systemarchitektur	46
4.7.1 Schichtentrennung	46
4.7.2 Datenfluss einer Aufgabenbearbeitung	47
4.7.3 Vorgenerierung und Ergebnis-Cache	50
5 Implementierung	51
5.1 Technologieauswahl	51
5.2 Umsetzung der Kernlogik	52
5.2.1 Formeldarstellung und Aufgabengenerator . .	52
5.2.2 Solver	53
5.2.3 Feedbackkomponente	55
5.3 Umsetzung der Vermittlungsschicht	55
5.3.1 Anwendungsstruktur	55
5.3.2 Datenmodell	56
5.3.3 API-Endpunkte	58
5.3.4 Vorgenerierungsdienst	59
5.4 Umsetzung der Präsentationsschicht	60
5.4.1 Seitenstruktur und Routing	60
5.4.2 Aufgabenbearbeitung	62
5.4.3 Lernpfad-Darstellung	63
5.4.4 Freies Üben	64

5.4.5 Begleitende Seiten	64
5.5 Betriebsmodi und Einrichtung	65
5.6 Tests	65
6 Evaluation	67
6.1 Aufgabengenerierung	67
6.2 Korrektheit des Solvers	69
6.3 Funktionale Überprüfung	70
7 Diskussion und Fazit	73
7.1 Reflexion der Entwurfsentscheidungen	73
7.2 Einschränkungen	74
7.3 Fazit	75
7.4 Ausblick	76

Abbildungsverzeichnis

1	Schichtenmodell	48
2	Datenfluss einer Aufgabenbearbeitung	49
3	Beziehungen zwischen den Datenbankmodellen	56
4	Startseite der Anwendung	61
5	Lösung einer Aufgabe	62
6	Lernpfad-Ansicht direkt nach der Registrierung eines neuen Nutzers. . . .	63
7	Freies Üben-Ansicht.	64

Tabellenverzeichnis

1	Schwierigkeitskonfiguration nach Aufgabentyp und Level	32
2	Eingesetzte Technologien nach Schicht	52
3	REST-Endpunkte der Vermittlungsschicht	58
4	Ergebnisse der Aufgabengenerierung nach Aufgabentyp und Level	67
5	Ergebnisse der Solver-Verifikation gegen SymPy	70
6	Abgleich der umgesetzten Funktionalität mit der Zielsetzung	71

1 Einleitung

1.1 Motivation und fachliche Relevanz

In Studiengängen der Mathematik und Informatik zählen Aussagenlogik und logisches Schließen zu zentralen methodischen Grundlagen, um Aussagen formal beweisen und prüfen zu können. Wie wichtig dieses Thema ist, spiegelt sich auch in offiziellen Richtlinien wider: So definieren die Curricula-Empfehlungen für Informatik-Studiengänge Logik und diskrete Strukturen als mathematische Kernkompetenz (The Joint Task Force on Computing Curricula 2023).

Trotz ihrer zentralen fachlichen Bedeutung bereitet formale bzw. boolesche Logik vielen Studierenden erhebliche Lernschwierigkeiten (Herman u. a. 2012). Eine wesentliche Ursache dafür liegt in der Diskrepanz zwischen dem alltäglichen Sprachgebrauch und streng formaler Logik. Daher entwickeln Lernende häufig hartnäckige Fehlvorstellungen, welche sich unter anderem beim logischen Schließen bemerkbar machen (Herman u. a. 2012). Empirische Untersuchungen zeigen beispielsweise, dass Studierende die Implikation systematisch wie eine Konjunktion interpretieren oder den alltäglichen exklusiven Gebrauch von „oder“ auf die formale Disjunktion übertragen (Herman u. a. 2012). Daher können gerade bei der Auswertung von Wahrheitswerten innerhalb komplexer Prämissensysteme solche Misskonzeptionen zu klassischen Fehlschlüssen führen, die den Lernprozess erschweren.

Aus lerntheoretischer Sicht reicht es deshalb nicht aus, sich bei der Vermittlung von Logik auf punktuelle Aufgabenbeispiele zu beschränken. Der Grund hierfür liegt in der Natur des Faches: Bei formal-logischen Inhalten steht weniger das Auswendiglernen von Definitionen im Vordergrund, sondern vielmehr der Aufbau prozeduraler Kompetenzen (Herman u. a. 2012). Lerntheoretische Befunde deuten allgemein darauf hin, dass verteiltes Üben und wiederholte Anwendung den Aufbau prozeduraler Kompetenzen fördern (Donoghue und Hattie 2021). Dies legt nahe, dass auch bei formal-logischen Inhalten abstrakte Schluss- und Prüfschritte in variierenden Aufgabenstellungen wiederholt angewandt werden sollten, um Verständnis und Routine schrittweise aufzubauen.

Vor diesem Hintergrund stellen digitale Übungsplattformen eine sinnvolle Ergänzung zur klassischen Präsenzlehre dar, die das kontinuierliche und verteilte Üben im Alltag der Studierenden unterstützen können. Ein wesentlicher Vorteil liegt in der automatisierten Auswertung. Da Eingaben in Echtzeit überprüft werden können, erhalten die Studierenden ohne zusätzlichen personellen Betreuungsaufwand sofortiges Feedback zu ihren Lösungsansätzen (Shute 2007). Wenn Präsenzveranstaltungen auf diese Weise mit digitalen Übungsphasen kombiniert werden, wirkt sich das in der Praxis positiv auf das studentische Engagement und den generellen Lernerfolg aus (De Bruijn-Smolders und Prinsen 2024).

Neben der reinen Verfügbarkeit von Übungsmaterialien ist die langfristige Aufrechterhaltung der Lernmotivation ein entscheidender Erfolgsfaktor. Gerade im unbegleiteten

Selbststudium und bei derart formalisierten Inhalten wie der Aussagenlogik erfordert es ein hohes Maß an Eigenmotivation, über das gesamte Semester hinweg kontinuierlich zu üben. Um dieser Herausforderung zu begegnen und das studentische Engagement gezielt zu stützen, greifen moderne digitale Lernumgebungen zunehmend auf Gamification-Ansätze zurück. Diese umfassen Elemente wie beispielsweise Fortschrittstracking oder ein Levelsystem, welche den abstrakten Übungsalltag auflockern können. Solche spielerischen Elemente können sich messbar positiv auf das Lernverhalten, die Motivation und den letztlichen Lernerfolg auswirken (Sailer und Homner 2020; Dicheva u. a. 2015).

1.2 Problemstellung

Aus den beschriebenen Herausforderungen ergibt sich der Bedarf nach einer digitalen Übungsumgebung, die ein systematisches und eigenständiges Bearbeiten logischer Schlüsselaufgaben ermöglicht und dabei über statische Einzelaufgaben hinausgeht.

Eine solche Übungsumgebung muss fachlich geeignete Aufgaben nicht nur anzeigen, sondern diese in strukturierter Form bereitstellen oder erzeugen können. Dabei gilt es, verschiedene Aufgabentypen abzubilden, da sich logisches Schließen in der Praxis nicht auf ein einziges Lösungsschema reduzieren lässt.

Neben der inhaltlichen Vielfalt der Aufgaben stellt jedoch auch die Art der Interaktion eine wesentliche Hürde dar. Die Schwierigkeit bei einer digitalen Umgebung liegt oft darin, die Eingabemöglichkeiten so zu gestalten, dass das System sie verarbeiten kann, ohne die Studierenden durch eine zu komplizierte Bedienung zu überfordern. Die Gestaltung einer intuitiven Schnittstelle ist jedoch nur die erste Voraussetzung für ein funktionierendes System, da eine benutzerfreundliche Eingabe allein noch keinen didaktischen Mehrwert garantiert.

Damit ein solches System die Lehre wirksam entlasten kann, müssen die Aufgaben automatisch und logisch korrekt ausgewertet werden. Dies setzt voraus, dass für jede Aufgabe die fachlich richtige Lösung beziehungsweise die Menge der tatsächlich ableitbaren Aussagen algorithmisch bestimmbar ist. Die Problemstellung umfasst daher nicht nur die Prüfung gegebener Antworten, sondern bereits die grundlegende Herausforderung, logische Schlüsselaufgaben formal so zu modellieren, dass ihre Lösung jederzeit zuverlässig berechnet werden kann.

Ein digitales Übungssystem sollte den Studierenden zudem eine unmittelbare Rückmeldung zum Bearbeitungsergebnis ermöglichen. Darüber hinaus stellt sich die Frage, wie die Aufgaben so angeordnet werden können, dass sie einen nachvollziehbaren Kompetenzaufbau unterstützen und nicht lediglich als unverbundene Sammlung einzelner Aufgaben erscheinen. Ergänzend können strukturierende und in begrenztem Umfang auch spielerische Elemente den Übungsprozess als zusätzlicher didaktischer Ansatz unterstützen.

1.3 Zielsetzung und Beitrag

Aus der in Abschnitt 1.2 beschriebenen Problemstellung ergibt sich das übergeordnete Ziel, eine digitale Übungsumgebung zu konzipieren und prototypisch umzusetzen, die speziell auf das logische Schließen in der Aussagenlogik zugeschnitten ist. Die vorliegende Arbeit ist im Kontext der Lehre von Prof. Dr. Reimar Hofmann entstanden. Die geplante Übungsplattform ist als ergänzendes digitales Angebot zur Vorlesung Mathematik 1 konzipiert und orientiert sich an den dort verwendeten Lehrmaterialien, insbesondere hinsichtlich der Auswahl und Struktur der unterstützten Aufgabentypen zum logischen Schließen (Hofmann 2026). Sie soll selbstständiges und wiederholbares Üben gezielt unterstützen. Im Zentrum steht somit die Entwicklung eines lokal ausführbaren webbasierten Übungssystems, das fachlich geeignete Aufgaben bereitstellt, deren Bearbeitung auswertet und den Übungsprozess strukturiert begleitet.

Ein wesentliches Teilziel besteht darin, Aufgaben zum logischen Schließen automatisch zu erzeugen. Dabei wird berücksichtigt, dass sich logisches Schließen nicht auf ein einziges Bearbeitungsschema reduzieren lässt. Aus diesem Grund sollen innerhalb der Plattform verschiedene Aufgabentypen unterstützt werden. Konkret werden sowohl Aufgaben zum direkten Ziehen logischer Folgerungen als auch Aufgaben berücksichtigt, bei denen eine Fallunterscheidung notwendig ist. Darüber hinaus verfolgt die Arbeit das Ziel, unterschiedliche Schwierigkeitsgrade abzubilden, um einen schrittweisen Kompetenzaufbau zu ermöglichen. Die generierten Aufgaben sollen dabei nicht beliebig sein, sondern inhaltlich sinnvolle Prämissenmengen enthalten, die einen didaktischen Mehrwert für die Studierenden bieten.

Damit die erzeugten Aufgaben nicht nur präsentiert, sondern auch automatisch ausgewertet werden können, soll die fachlich korrekte Lösung jeder Aufgabe algorithmisch bestimmt werden. Für jede Variable einer gegebenen Prämissenmenge soll ermittelt werden können, ob sie aus den Prämissen zwingend wahr, zwingend falsch oder nicht eindeutig festgelegt ist. Dies setzt ein geeignetes Lösungsverfahren voraus, dessen Auswahl und Umsetzung Teil der vorliegenden Arbeit ist.

Darüber hinaus soll eine Eingabestruktur entwickelt werden, die für Studierende handhabbar und für das System eindeutig auswertbar ist, sowie eine darauf aufbauende automatische Bewertungslogik, die Nutzerantworten mit der formal bestimmten Lösung abgleicht.

Neben der automatischen Auswertung zielt die Arbeit auch auf eine unmittelbare Rückmeldung zum Bearbeitungsergebnis ab. Dadurch soll die Plattform die Bearbeitung von Aufgaben nicht nur bewerten, sondern Studierende in elementarer Form bei der Einordnung ihrer Antworten unterstützen. Die Rückmeldung ist dabei ergebnisbezogen ausgerichtet und ergänzt die automatische Bewertung um erste inhaltliche Hinweise bei fehlerhaften Antworten.

Ein weiterer Schwerpunkt der Arbeit liegt in der didaktischen Strukturierung des Übungs-

prozesses. Das System soll nicht lediglich eine lose Sammlung einzelner Aufgaben bereitstellen, sondern in eine Lernumgebung eingebettet sein, die einen nachvollziehbaren Kompetenzaufbau unterstützt. Dazu gehört insbesondere, dass Lernende schrittweise von einfacheren zu komplexeren Aufgabentypen und Schwierigkeitsstufen geführt werden. Darüber hinaus sollen Inhalte zu den Grundlagen der Aussagenlogik und weiterführende Referenzen die Aufgabenbearbeitung fachlich einrahmen.

Schließlich verfolgt die Arbeit das Ziel, den individuellen Lernstand sichtbar zu machen und damit selbstständiges Üben zusätzlich zu unterstützen. Dazu gehören insbesondere eine Fortschrittsdarstellung, ein Levelsystem sowie nutzerbezogene Informationen zum bisherigen Bearbeitungsverlauf. Ergänzend können in begrenztem Umfang auch spielerische Elemente den Übungsprozess unterstützen.

1.4 Abgrenzung der Arbeit

Die vorliegende Arbeit konzentriert sich auf die Konzeption und prototypische Umsetzung einer digitalen Übungsumgebung für das logische Schließen in der Aussagenlogik. Andere formale Logiksysteme, insbesondere weiterführende Bereiche wie die Prädikatenlogik oder allgemeinere formale Beweissysteme, sind nicht Gegenstand der Betrachtung. Auch innerhalb der Aussagenlogik wird nicht der gesamte denkbare Aufgabenraum abgedeckt. Der Fokus liegt vielmehr auf ausgewählten, für den Lernkontext zentralen Formen des logischen Schließens, die im Rahmen der Plattform durch Aufgaben zu direkten logischen Folgerungen und Fallunterscheidungen abgebildet werden.

In methodischer Hinsicht ist der Anspruch bewusst begrenzt. Die Generierung von Aufgaben, die Modellierung von Schwierigkeitsgraden sowie die Progression innerhalb der Anwendung erfolgen regelbasiert über vordefinierte formale Kriterien. Ein adaptiver Ansatz, der das Systemverhalten auf Basis individuellen Nutzerverhaltens dynamisch optimiert, wurde im Rahmen dieser Arbeit nicht verfolgt.

Das verwendete Lösungsverfahren baut konzeptionell auf einem bestehenden Literaturansatz auf und orientiert sich in seiner technischen Auslegung an kleinen Variablenmengen im Lehrkontext, nicht an einer möglichst allgemeinen oder hochskalierenden Problemlösung. Auch die Rückmeldefunktion ist in ihrem Umfang bewusst eingegrenzt: Sie ist auf eine unmittelbare und ergebnisbezogene Unterstützung ausgerichtet. Eine umfassende Diagnose individueller Denkwege oder ein vollständig ausgebautes intelligentes Tutorensystem ist nicht Gegenstand der Arbeit.

Die entwickelte Plattform ist als lokal ausführbarer webbasierter Prototyp zu verstehen und nicht als produktionsreifes Hochschulsystem. Weder Fragen des großflächigen Betriebs noch eine institutionelle Integration in bestehende Hochschulstrukturen stehen im Mittelpunkt. Ebenso ist keine umfassende empirische Evaluation der Lernwirksamkeit, Motivation oder Usability mit größeren Studierendengruppen vorgesehen. Der Fokus

liegt vielmehr auf der Konzeption und prototypischen Realisierung der fachlichen, didaktischen und technischen Kernkomponenten.

2 Theoretische Grundlagen

Hier werden fachliche und methodische Grundlagen beschrieben, auf denen die nachfolgenden Kapitel aufbauen.

2.1 Fachliche Grundlagen der Aussagenlogik

Die folgenden Abschnitte führen die grundlegenden Begriffe der Aussagenlogik ein, die für das Verständnis der in dieser Arbeit behandelten Aufgabentypen und des verwendeten Lösungsverfahrens vorausgesetzt werden.

2.1.1 Grundbegriffe der Aussagenlogik

Aussage

Der Ausgangspunkt der Aussagenlogik sind *Aussagen* im Sinn von deklarativen Sätzen, also Sätzen, denen prinzipiell ein Wahrheitswert zukommen kann. Typische Beispiele sind Behauptungen wie „Es regnet“ oder „ $3 + 5 = 8$ “. Fragen, Befehle oder Ausrufe zählen nicht dazu (Huth und Ryan 2004).

Aussagenvariable

Da die Aussagenlogik nicht am inhaltlichen Detail, sondern an der logischen Struktur interessiert ist, werden atomare Aussagen durch Aussagenvariablen repräsentiert. Eine Aussagenvariable steht somit stellvertretend für eine elementare Aussage. In der Literatur werden Aussagenvariablen häufig durch Symbole wie p , q oder r notiert. In der vorliegenden Arbeit werden, angelehnt an die verwendeten Lehrmaterialien, Großbuchstaben in alphabetischer Reihenfolge wie A , B , C oder D verwendet (Hofmann 2026). Auf diese Weise lassen sich auch komplexere Aussagen in symbolischer Form darstellen und weiterverarbeiten (Huth und Ryan 2004).

Atomare Aussage / atomare Formel

Eine atomare Aussage ist eine inhaltlich nicht weiter zerlegte elementare Aussage. Auf der formalen Ebene wird sie durch eine Aussagenvariable wie A , B oder C dargestellt. Solche Aussagenvariablen bilden die elementaren Bausteine der Sprache der Aussagenlogik.

Beispielsweise kann A für die Aussage „Die Zahl 5 ist gerade“ stehen. Der Aussage wird im Rahmen einer Bewertung ein Wahrheitswert zugewiesen. Eine atomare Formel besitzt damit noch keine innere logische Struktur, sondern erhält ihre Bedeutung erst durch die Zuordnung zu einer konkreten Aussage und deren Wahrheitswert (Huth und Ryan 2004).

Formel

Eine Formel ist ein syntaktisch korrekt gebildeter Ausdruck der Sprache der Aussagenlogik. Formeln entstehen entweder direkt als atomare Formeln oder rekursiv durch die Anwendung von Junktoren auf bereits vorhandene Formeln. Auf diese Weise können aus einfachen Aussagen schrittweise komplexere Ausdrücke aufgebaut werden.

Beispiele für Formeln sind $\neg A$, $A \vee C$ oder $(A \wedge B) \rightarrow ((\neg C) \vee B)$. Der rekursive Aufbau ist dabei wesentlich, da jede komplexe Formel letztlich auf atomare Aussagen zurückgeführt werden kann (Huth und Ryan 2004).

Junktoren / Operatoren

Junktoren sind die logischen Verknüpfungen, mit denen aus einfachen Aussagen komplexe Aussagen gebildet werden (Huth und Ryan 2004).

- Negation $\neg$
- Konjunktion $\wedge$
- Disjunktion $\vee$
- Implikation / Konditional $\rightarrow$

Neben den Basis-Junktoren können weitere Verknüpfungen in Form abgeleiteter Operatoren bestimmt werden. Dazu zählen insbesondere die Äquivalenz $\leftrightarrow$ sowie die exklusive Disjunktion $\oplus$.

Die Äquivalenz $A \leftrightarrow B$ drückt aus, dass A und B denselben Wahrheitswert besitzen. Die exklusive Disjunktion $A \oplus B$ ist genau dann wahr, wenn genau eine der beiden Aussagen wahr ist.

Syntax

Unter Syntax versteht man die formalen Bildungsvorschriften der Sprache der Aussagenlogik. Sie legen fest, welche Zeichen verwendet werden dürfen und nach welchen Regeln aus Aussagenvariablen und Junktoren zulässige Formeln entstehen. Eine Formel ist nur dann syntaktisch korrekt, wenn sie nach diesen Regeln aufgebaut ist.

Da lineare Schreibweisen mehrdeutig sein können, werden Formeln in der Regel durch Klammern eindeutig strukturiert. Ergänzend gelten Konventionen zur Bindungsstärke von Operatoren, etwa dass $\neg$ stärker bindet als $\wedge$, $\wedge$ stärker als $\vee$, $\vee$ stärker als $\oplus$, $\oplus$ stärker als $\rightarrow$ und $\rightarrow$ stärker als $\leftrightarrow$ (Huth und Ryan 2004).

Semantik

Unter Semantik versteht man die Bedeutungszuweisung zu den syntaktischen Ausdrücken. In der Aussagenlogik ist diese Semantik wahrheitsfunktional: Sobald die Wahrheitswerte der verwendeten Aussagenvariablen festgelegt sind, ist auch der Wahrheitswert der gesamten Formel bestimmt (Huth und Ryan 2004).

2.1.2 Wahrheitswerte und Belegungen

Wahrheitswerte

Die Aussagenlogik arbeitet mit genau zwei Wahrheitswerten: *Wahr* und *Falsch*. In technischen und algorithmischen Kontexten werden diese häufig auch durch die Werte 1 und 0 dargestellt (Huth und Ryan 2004).

Wahrheitstafel

Eine Wahrheitstafel stellt dar, welchen Wahrheitswert eine zusammengesetzte Formel für alle möglichen Belegungen ihrer atomaren Aussagen annimmt. Sie dient damit zur expliziten Beschreibung der Semantik logischer Operatoren. Für jede mögliche Kombination der Wahrheitswerte der beteiligten Aussagenvariablen wird angegeben, welchen Wahrheitswert der jeweilige Ausdruck besitzt (Huth und Ryan 2004).

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \oplus B$	$A \rightarrow B$	$A \leftrightarrow B$
0	0	1	0	0	0	1	1
0	1	1	0	1	1	1	0
1	0	0	0	1	1	0	0
1	1	0	1	1	0	1	1

Belegung

Eine Belegung ist eine Zuordnung, die jeder in einer Formel vorkommenden Aussagenvariablen einen Wahrheitswert zuweist. Für die Formel $A \vee \neg B$ wäre beispielsweise $A = 0$, $B = 1$ eine mögliche Belegung (Huth und Ryan 2004).

Auswertung einer Formel unter einer Belegung

Durch eine Belegung wird der Wahrheitswert einer Formel eindeutig bestimmt. Die Auswertung erfolgt kompositionell anhand der Wahrheitswerte der atomaren Aussagen und der Bedeutung der verwendeten Junktoren. Für $A \vee \neg B$ ergibt sich unter der Belegung $A = 0$, $B = 1$ der Wahrheitswert 0 (Huth und Ryan 2004).

Anzahl möglicher Belegungen

Enthält eine Formel n verschiedene Aussagenvariablen, so existieren 2^n mögliche Belegungen. Diese Anzahl entspricht zugleich der Zahl der Zeilen einer vollständigen Wahrheitstafel (Huth und Ryan 2004).

Für das in dieser Arbeit eingesetzte Lösungsverfahren ist dieser exponentielle Zusammenhang unmittelbar relevant, da die verwendeten booleschen Tabellen grundsätzlich alle möglichen Belegungen repräsentieren (vgl. Abschnitt 2.3).

2.1.3 Modelle und erfüllende Belegungen

Erfüllende Belegung

Eine Belegung heißt erfüllend, wenn die betrachtete Formel unter dieser Belegung den Wahrheitswert 1 annimmt. Für die Formel $A \vee \neg B$ ist beispielsweise die Belegung $A = 1$, $B = 1$ erfüllend, da sich daraus der Wahrheitswert 1 ergibt (Huth und Ryan 2004).

Erfüllende Belegungen einer Menge an Formeln

Der Begriff der erfüllenden Belegung lässt sich ebenso auf Mengen von Formeln übertragen. Eine Belegung ist genau dann erfüllend für eine Menge an Formeln, wenn jede einzelne Formel unter dieser Belegung den Wahrheitswert 1 annimmt. Für die vorliegende Arbeit ist dies besonders relevant, da die erzeugten Aufgaben auf Formelmengen beruhen und deren zulässige beziehungsweise erfüllende Belegungen die Grundlage der späteren automatischen Auswertung bilden.

Modell

Im Folgenden wird eine erfüllende Belegung auch als Modell einer Formelmenge bezeichnet. Eine Formelmenge heißt demnach erfüllbar, wenn sie mindestens ein Modell besitzt. Ein Modell beschreibt somit eine konkrete Zuordnung von Wahrheitswerten zu den in der betrachteten Formelmenge vorkommenden Aussagenvariablen, unter der jede Formel der Formelmenge erfüllt ist.

Eine Formelmenge kann mehrere Modelle besitzen. Die Menge aller Modelle beschreibt genau diejenigen Belegungen, unter denen jede Formel der betrachteten Formelmenge den Wahrheitswert 1 annimmt.

Erfüllbarkeit und SAT

Eine Formel beziehungsweise Formelmenge heißt erfüllbar, wenn mindestens eine erfüllende Belegung beziehungsweise mindestens ein Modell existiert. Das zugehörige Entscheidungsproblem, also die Frage, ob für eine gegebene Formel oder Formelmenge überhaupt eine erfüllende Belegung existiert, wird als *Satisfiability Problem* (SAT) bezeichnet (Huth und Ryan 2004).

Im Rahmen des in Abschnitt 2.3 beschriebenen Lösungsverfahrens wird die Erfüllbarkeit der gegebenen Prämissenmenge als Nebenprodukt des Forward Passes entschieden.

2.1.4 Logische Folgerung aus einer Prämissenmenge

Prämissen

Prämissen sind Aussagen beziehungsweise Formeln, die in einem logischen Schluss als gegeben angenommen werden. Von ihnen ausgehend wird untersucht, welche weiteren Aussagen sich logisch ableiten lassen (Huth und Ryan 2004).

Logische Folgerung

Eine Formel ψ folgt logisch aus einer Prämissenmenge Γ , wenn in jeder Belegung, in der alle Formeln aus Γ den Wahrheitswert 1 annehmen, auch ψ den Wahrheitswert 1 annimmt. Die Formel ψ wird dabei als *Konklusion* bezeichnet. Logische Folgerung beschreibt damit eine semantische Beziehung zwischen einer Menge von Formeln und einer weiteren Formel. In der Logik wird dies durch die Schreibweise

$$\Gamma \models \psi$$

ausgedrückt (Huth und Ryan 2004).

Beispiel

Für die Prämissenmenge

$$\Gamma = \{\neg B, A \vee B\}$$

und die Konklusion

$$\psi = A$$

gilt

$$\{\neg B, A \vee B\} \models A.$$

Damit alle Formeln aus Γ den Wahrheitswert 1 annehmen, muss zunächst $B = 0$ gelten, da andernfalls $\neg B$ falsch wäre. Unter dieser Voraussetzung kann $A \vee B$ nur dann den Wahrheitswert 1 besitzen, wenn $A = 1$ ist. Somit ist in jeder Belegung, die alle Prämissen erfüllt, auch die Konklusion A wahr.

Gegenmodell

Eine Formel ψ ist keine logische Folge einer Prämissenmenge Γ , wenn es mindestens eine Belegung gibt, in der alle Formeln aus Γ den Wahrheitswert 1 besitzen, ψ jedoch den Wahrheitswert 0 annimmt. Eine solche Belegung wird als Gegenmodell bezeichnet (Huth und Ryan 2004).

2.1.5 Konjunktive Normalform und Unit Propagation

Literal

Ein *Literal* ist eine Aussagenvariable oder ihre Negation. Beispielsweise A und $\neg A$ (Huth und Ryan 2004).

Konjunktive Normalform (KNF, engl. CNF)

Eine Formel liegt in *konjunktiver Normalform* vor, wenn sie als Konjunktion von Klauseln dargestellt ist, wobei eine Klausel eine Disjunktion von Literalen ist. Formeln der

Aussagenlogik lassen sich in eine äquivalente konjunktive Normalform überführen (Huth und Ryan 2004).

Unit Propagation

Unit Propagation ist eine Inferenzregel auf solchen Klauselmengen (Nieuwenhuis, Oliveras und Tinelli 2006). Enthält eine Klausel unter einer partiellen Belegung nur noch ein unbestimmtes Literal, während alle übrigen Literale falsch sind, so muss dieses Literal so belegt werden, dass die Klausel erfüllt ist (Nieuwenhuis, Oliveras und Tinelli 2006). Eine solche Klausel wird als *Einheitsklausel* bezeichnet (Nieuwenhuis, Oliveras und Tinelli 2006). Durch wiederholte Anwendung der Regel lassen sich weitere Variablenbelegungen direkt aus den Prämissen ableiten (Nieuwenhuis, Oliveras und Tinelli 2006).

Beispiel

$$(A) \wedge (\neg A \vee B)$$

Aus der Einheitsklausel (A) folgt die Belegung $A = 1$. Dadurch vereinfacht sich die zweite Klausel zu (B) , sodass auch $B = 1$ folgt.

2.2 Formen des logischen Schließens

Unter Voraussetzung der zugrunde liegenden Lehrmaterialien von Hofmann (2026) sollen in der vorliegenden Arbeit zwei Formen des logischen Schließens unterschieden werden: direktes Schließen sowie Schließen mithilfe einer Fallunterscheidung. Diese beiden Formen bilden zugleich die fachliche Grundlage der in der Übungsplattform vorgesehenen Aufgabentypen.

2.2.1 Direktes logisches Schließen

Direktes Schließen liegt vor, wenn sich aus einer gegebenen Prämissenmenge weitere Aussagenwerte unmittelbar ableiten lassen, ohne zusätzliche Annahmen zu treffen oder eine Fallunterscheidung durchführen zu müssen. Die Schlussfolgerung ergibt sich dabei direkt aus den vorhandenen Prämissen.

Beispiel

$$\begin{aligned} (P1) \quad & A \rightarrow B \\ (P2) \quad & A \end{aligned}$$

Aus Prämisse $(P2)$ folgt unmittelbar, dass $A = 1$ gilt. Damit $(P1)$ ebenfalls erfüllt ist, muss B den Wahrheitswert 1 besitzen. Es lässt sich somit direkt schließen, dass B wahr ist.

2.2.2 Schließen mithilfe einer Fallunterscheidung

Eine Fallunterscheidung wird benötigt, wenn sich eine Aussage nicht unmittelbar aus den gegebenen Prämissen ableiten lässt, sondern zunächst verschiedene mögliche Fälle betrachtet werden müssen. Ergibt sich in allen Fällen dieselbe Schlussfolgerung oder führt einer der Fälle zu einem Widerspruch, so kann daraus insgesamt eine logische Folgerung abgeleitet werden.

Beispiel

$$\begin{aligned}(P1) \quad & A \vee B \\(P2) \quad & A \rightarrow B \\(P3) \quad & B \rightarrow \neg A\end{aligned}$$

Zu Beginn lässt sich aus den drei Prämissen kein Wahrheitswert unmittelbar bestimmen. Daher wird zunächst der Fall $A = 1$ betrachtet.

Aus (P2) folgt dann unmittelbar, dass auch $B = 1$ gelten muss. Setzt man dies in (P3) ein, so folgt aus $B = 1$, dass $\neg A$ gelten muss, also $A = 0$.

Dies widerspricht der Annahme $A = 1$. Der Fall $A = 1$ ist also ausgeschlossen.

Damit bleibt nur noch der Fall $A = 0$. Aus (P1) folgt dann unmittelbar, dass $B = 1$ gelten muss, damit die Disjunktion erfüllt ist.

Damit sind die Wahrheitswerte eindeutig bestimmt:

$$A = 0, \quad B = 1.$$

2.3 Grundlagen des solverbasierten Lösungsverfahrens

Der folgende Abschnitt beschreibt die Grundlagen des Deletion-Algorithmus auf Basis boolescher Arrays nach Díaz-Macías und Moral (2023), der im Rahmen der vorliegenden Arbeit als Lösungsverfahren aufgegriffen wird. Soweit nicht anders gekennzeichnet, orientiert sich die Darstellung im Abschnitt 2.3 an dieser Quelle.

2.3.1 Boolesche Tabellen als Darstellung zulässiger Belegungen

Grundlage des Algorithmus ist die Darstellung von Formeln beziehungsweise Formelmengen über boolesche Tabellen.

Eine boolesche Tabelle ist an eine feste Menge von Aussagenvariablen gebunden und beschreibt für jede mögliche Belegung dieser Variablen, ob sie die betrachtete Formelmengen erfüllt. Der Tabellenwert 1 bedeutet dabei, dass die entsprechende Belegung zulässig beziehungsweise erfüllend ist, während 0 eine nicht zulässige Belegung kennzeichnet (Díaz-Macías und Moral 2023).

Auf diese Weise wird eine Formelmengen nicht mehr nur als syntaktischer Ausdruck betrachtet, sondern zugleich als Menge von Belegungen, unter denen sie erfüllt ist. Diese

Darstellungsform bildet die Grundlage für die im weiteren Verlauf beschriebenen Tabellenoperationen.

Beispiel

Für die Formel

$$A \rightarrow B$$

ergibt sich die folgende Darstellung ihrer zulässigen Belegungen:

A	B	zulässig
0	0	1
0	1	1
1	0	0
1	1	1

Die Tabelle zeigt, dass nur die Belegung $A = 1, B = 0$ nicht mit der Formel vereinbar ist. Alle übrigen Belegungen erfüllen die Formel und werden daher mit dem Tabellenwert 1 gekennzeichnet.

2.3.2 Grundoperationen auf booleschen Tabellen

Um boolesche Tabellen im Lösungsverfahren weiterverarbeiten zu können, werden auf ihnen grundlegende Operationen ausgeführt. Diese dienen nicht nur der formalen Manipulation von Tabellen, sondern erfüllen eine inhaltliche Funktion im Lösungsprozess: Informationen aus verschiedenen Formeln beziehungsweise Formelmengen werden zunächst zusammengeführt und anschließend auf kleinere Mengen von Aussagenvariablen projiziert.

Eine zentrale Operation ist die *Kombination* zweier boolescher Tabellen. Beschreibt eine Tabelle die zulässigen Belegungen einer ersten Formelmenge und eine weitere Tabelle die zulässigen Belegungen einer zweiten Formelmenge, so enthält jede Tabelle zunächst nur einen Teil der insgesamt relevanten Information. Für das Lösungsverfahren ist jedoch entscheidend, welche Belegungen mit beiden Formelmengen zugleich vereinbar sind. Genau das wird durch die Kombination bestimmt.

In der kombinierten Tabelle bleiben nur diejenigen Belegungen erhalten, die in beiden Ausgangstabellen zulässig sind. Belegungen, die bereits mit einer der beiden Tabellen unvereinbar sind, werden ausgeschlossen. Die Kombination bewirkt somit, dass die in verschiedenen Formeln enthaltene Information zu einer gemeinsamen Beschreibung der noch möglichen Belegungen zusammengeführt wird. In der Darstellung mit 0- und 1-Werten entspricht dies einer Verknüpfung durch logisches UND (Díaz-Macías und Moral 2023).

Falls beide Tabellen nicht über dieselbe Menge von Aussagenvariablen definiert sind, wird eine *Erweiterung* auf einen gemeinsamen Variablenraum durchgeführt, da sie sonst nicht unmittelbar miteinander kombiniert werden können. Inhaltlich kommt dabei keine neue Information hinzu. Vielmehr wird dieselbe Information auf zusätzliche Aussagenvariablen ausgedehnt, die für die betreffende Tabelle selbst keine Rolle spielen. Eine Tabelle über der Variablen A wird beispielsweise bei einer Erweiterung auf die Variablenmenge $\{A, B\}$ so gelesen, dass ihr Tabellenwert nur von A abhängt und für beide möglichen Wahrheitswerte von B unverändert bleibt (Díaz-Macías und Moral 2023).

Beispiel

Gegeben sind die Formeln:

$$F_1 = A \rightarrow B \quad \text{und} \quad F_2 = A$$

Für F_1 und F_2 ergeben sich die folgenden booleschen Tabellen:

A	B	F_1		A	F_2		A	B	F_2
0	0	1		0	0	$\xrightarrow{\text{Erweiterung}}$	0	0	0
0	1	1		0	0		0	1	0
1	0	0		1	1		1	0	1
1	1	1					1	1	1

Werden beide Tabellen kombiniert, so bleiben nur noch diejenigen Belegungen erhalten, die sowohl F_1 als auch F_2 erfüllen:

A	B	$F_1 \wedge F_2$
0	0	0
0	1	0
1	0	0
1	1	1

Die Kombination zeigt somit, dass nur noch die Belegung $A = 1, B = 1$ mit beiden Formeln zugleich vereinbar ist.

Eine weitere grundlegende Operation ist die *Marginalisierung*. Sie dient dazu, eine Aussagenvariable aus einer Tabelle zu entfernen, ohne die für die verbleibenden Variablen relevante Information zu verlieren. Dabei wird die zu eliminierende Variable nicht einfach gestrichen. Stattdessen wird für jede Belegung der verbleibenden Aussagenvariablen geprüft, ob sie sich durch mindestens einen Wahrheitswert der entfernten Variable zu einer vollständigen zulässigen Belegung ergänzen lässt.

Ist eine solche Ergänzung möglich, so bleibt die Belegung der verbleibenden Aussagenvariablen erhalten. Ist keine solche Ergänzung möglich, so wird sie ausgeschlossen. Die

Marginalisierung kann daher als eine Projektion der in der Tabelle enthaltenen Information auf eine kleinere Variablenmenge verstanden werden: Erhalten bleibt genau die Aussage darüber, welche Belegungen der übrigen Variablen weiterhin möglich sind. In der Darstellung mit 0- und 1-Werten entspricht dies einer Reduktion über logisches ODER (Díaz-Macías und Moral 2023).

Fortführung des Beispiels

Ausgehend von der kombinierten Tabelle kann nun die Aussagenvariable A marginalisiert werden. Ziel ist es, nur noch zu betrachten, welche Belegungen für B weiterhin möglich sind.

A	B	$F_1 \wedge F_2$		B	$(F_1 \wedge F_2)[A = 0] \vee (F_1 \wedge F_2)[A = 1]$
0	0	0	$\xrightarrow{\text{Marginalisierung von } A}$	0	0
0	1	0			
1	0	0		1	1
1	1	1			

Die marginalisierte Tabelle entsteht, indem für jeden möglichen Wahrheitswert von B geprüft wird, ob es mindestens einen passenden Wahrheitswert von A gibt, sodass die ursprüngliche kombinierte Tabelle den Wert 1 annimmt. Für $B = 0$ ist dies nicht der Fall, da beide zugehörigen Zeilen den Wert 0 besitzen. Für $B = 1$ existiert dagegen mit $A = 1$ eine zulässige vollständige Belegung. Nach der Marginalisierung bleibt somit genau die Information erhalten, dass B den Wahrheitswert 1 annehmen muss.

Neben Kombination und Marginalisierung wird im Lösungsverfahren eine weitere Operation benötigt: die *Einschränkung* (Conditioning) einer booleschen Tabelle auf einen festen Wahrheitswert. Dabei wird eine Aussagenvariable auf einen vorgegebenen Wert fixiert und aus der Tabelle entfernt. Erhalten bleiben nur diejenigen Belegungen der übrigen Variablen, die zusammen mit dem fixierten Wert in der ursprünglichen Tabelle als zulässig gekennzeichnet waren (Díaz-Macías und Moral 2023).

Fortführung des Beispiels

Ausgehend von der kombinierten Tabelle wird nun die Variable A auf den Wert 1 eingeschränkt.

A	B	$F_1 \wedge F_2$		B	$(F_1 \wedge F_2)[A = 1]$
0	0	0	$\xrightarrow{\text{Conditioning } A=1}$	0	0
0	1	0			
1	0	0		1	1
1	1	1			

Aus der Tabelle werden nur diejenigen Zeilen übernommen, in denen A den Wert 1 besitzt. Die Spalte A entfällt, da die Variable nun festgelegt ist. Die resultierende Tabelle zeigt, dass unter der Einschränkung $A = 1$ nur noch die Belegung $B = 1$ zulässig ist. Diese Operation wird im Backward Pass eingesetzt, um bereits bestimmte Variablenwerte in die jeweilige Bucket-Tabelle einzurechnen (vgl. Abschnitt 2.3.4).

Für das Lösungsverfahren sind diese Operationen grundlegend. Durch die Kombination wird die Information aus mehreren Formeln beziehungsweise Formelmengen gebündelt. Durch die Marginalisierung wird diese Information anschließend auf die verbleibenden Aussagenvariablen verdichtet. Die Einschränkung ermöglicht es, bereits gewonnene Erkenntnisse über einzelne Variablen in die verbleibende Darstellung einzurechnen. Auf diese Weise kann schrittweise bestimmt werden, welche Belegungen weiterhin zulässig sind und welche Informationen über die noch betrachteten Aussagenvariablen aus der gesamten Formelmenge folgen.

2.3.3 Forward Pass

Das Lösungsverfahren gliedert sich in zwei aufeinander aufbauende Phasen: einen Forward Pass, der die in den Tabellen enthaltene Information schrittweise verdichtet, und einen Backward Pass, der daraus die Belegungen der einzelnen Variablen bestimmt.

Der Forward Pass bildet den ersten Hauptschritt des solverbasierten Lösungsverfahrens. Ausgangspunkt ist eine Menge boolescher Tabellen, die die in den gegebenen Formeln beziehungsweise Formelmengen enthaltene Information repräsentieren. Im einfachsten Fall wird dabei zunächst zu jeder Formel eine eigene Tabelle erzeugt. Das Ziel des Forward Passes besteht darin, diese verteilte Information schrittweise zu verdichten, indem Aussagenvariablen nacheinander aus der Darstellung entfernt werden (Díaz-Macías und Moral 2023).

Hierzu wird eine feste Löschreihenfolge für die betrachteten Aussagenvariablen festgelegt. Für jede Variable dieser Reihenfolge werden zunächst alle Tabellen ausgewählt, in denen die betreffende Variable vorkommt. Diese Tabellen bilden gemeinsam einen sogenannten Bucket (Díaz-Macías und Moral 2023).

Innerhalb eines solchen Buckets werden die enthaltenen Tabellen zunächst kombiniert. Auf diese Weise wird die gesamte zur aktuellen Aussagenvariable gehörige Teilinformation in einer gemeinsamen Tabelle zusammengeführt. Anschließend wird die betrachtete Aussagenvariable aus dieser kombinierten Tabelle marginalisiert. Das Ergebnis ist eine neue, reduzierte Tabelle, die die Konsequenz des aktuellen Schritts nur noch in Bezug auf die verbleibenden Aussagenvariablen enthält. Diese reduzierte Tabelle wird anschließend in die verbleibende Tabellenmenge zurückgeführt und im weiteren Eliminationsprozess berücksichtigt.

Der Forward Pass endet, sobald alle Aussagenvariablen der festgelegten Löschreihenfolge verarbeitet wurden. Zu diesem Zeitpunkt liegt nicht nur die für den nachfolgenden Backward Pass benötigte Zwischenrepräsentation vor, sondern es kann auch entschieden

werden, ob die zugrunde liegende Formelmenge überhaupt erfüllbar ist. Die Aufgabe ist genau dann lösbar, wenn die resultierende Repräsentation noch mindestens eine zulässige Belegung zulässt. Damit liefert der Forward Pass zugleich die Entscheidung des Satisfiability Problems (SAT) für die betrachtete Formelmenge (Díaz-Macías und Moral 2023).

2.3.4 Backward Pass

Der Backward Pass bildet den zweiten Hauptschritt des solverbasierten Lösungsverfahrens. Während im Forward Pass die in der Formelmenge enthaltene Information schrittweise verdichtet und in Form von Buckets gespeichert wird, dient der Backward Pass dazu, aus diesen Zwischenergebnissen Belegungen der Aussagenvariablen zu bestimmen. Voraussetzung dafür ist, dass die Aufgabe erfüllbar ist (Díaz-Macías und Moral 2023). Die Bestimmung der Variablenbelegungen erfolgt in umgekehrter Löschreihenfolge. Für jede Aussagenvariable wird zunächst der zugehörige Bucket betrachtet, also die im Forward Pass gespeicherte Menge derjenigen Tabellen, die für den Eliminationsschritt dieser Variable verwendet wurden. Diese Tabellen werden erneut zu einer gemeinsamen Tabelle kombiniert. Anschließend wird diese Tabelle auf bereits bekannte Aussagenwerte eingeschränkt, die in den bisherigen Schritten des Backward Passes bereits bestimmt wurden. Auf Basis dessen wird geprüft, ob die aktuell betrachtete Aussagenvariable den Wahrheitswert 1, 0 oder beide Werte annehmen kann, ohne dass ein Widerspruch entsteht. Ist nur einer der beiden Werte mit den bisher bestimmten Aussagenwerten vereinbar, so ist die Variable eindeutig festgelegt. Sind dagegen beide Werte weiterhin möglich, so lässt sich aus der gegebenen Formelmenge kein konkreter Schluss auf den Wahrheitswert dieser Variable ziehen. Diese Unterscheidung bildet die Grundlage für das in der vorliegenden Arbeit verwendete Antwortschema mit den Ausprägungen „wahr“, „falsch“ und „kein konkreter Schluss möglich“ (vgl. Abschnitt 1.3).

3 Stand der Forschung und verwandte Ansätze

Nachdem die fachlichen und algorithmischen Grundlagen der Arbeit dargelegt wurden, ordnet das folgende Kapitel die vorliegende Arbeit in den bestehenden Forschungs- und Entwicklungskontext ein. Dazu werden zunächst bestehende digitale Übungssysteme für formale Logik vorgestellt, anschließend der Einsatz generativer Sprachmodelle in Lernszenarien beleuchtet und die vorliegende Arbeit gegenüber den betrachteten Ansätzen eingeordnet.

3.1 Digitale Übungs- und Tutorensysteme für formale Logik

Zur Einordnung der vorliegenden Arbeit wurden bestehende digitale Übungs- und Tutorensysteme für formale Logik recherchiert. Die Recherche umfasste einschlägige Konferenzreihen (AIED, ITiCSE, AAAI) sowie Datenbanksuchen in ACM Digital Library und Google Scholar mit Suchbegriffen wie 'logic tutor', 'propositional logic exercises' und 'automated logic assessment'. Die im Folgenden vorgestellten Systeme wurden ausgewählt, weil sie explizit aussagenlogische Aufgaben adressieren und hinsichtlich ihres Aufgabentyps oder ihrer didaktischen Ausrichtung einen Bezug zur vorliegenden Arbeit aufweisen. Systeme, die ausschließlich Prädikatenlogik oder Modallogik behandeln, wurden nicht berücksichtigt. Die Darstellung erhebt keinen Anspruch auf Vollständigkeit, deckt jedoch die für den vorliegenden Vergleich wesentlichen Ansätze ab. Neuere Publikationen zu Logic-ITA liegen nach Kenntnis des Autors nicht vor, sodass unklar bleibt, ob das System heute noch aktiv eingesetzt oder weiterentwickelt wird.

Das System *Logic-ITA* wurde bereits 2005 an der University of Sydney vorgestellt und zählt zu den frühen webbasierten intelligenten Lehrassistenten für die Aussagenlogik. Studierende konstruieren darin formale Beweise schrittweise, indem sie zu jedem Schritt eine Formel eingeben, die verwendete Inferenz- oder Äquivalenzregel auswählen und die referenzierten Vorgängerzeilen angeben. Das System prüft jeden Schritt und gibt kontextualisiertes Feedback. Darüber hinaus bietet es der Lehrperson Werkzeuge zur Konfiguration von Aufgaben und zur Analyse des Lernfortschritts der gesamten Klasse. In einer Evaluation mit über 600 Studierenden konnte eine signifikante Verbesserung der Prüfungsleistungen nachgewiesen werden (Yacef 2005).

Ittis ist ein an der TU Dortmund entwickeltes, ebenfalls webbasiertes System, das Aufgaben in Aussagenlogik, Modallogik und Prädikatenlogik abdeckt. Es unterstützt insbesondere das Modellieren von Szenarien durch logische Formeln sowie das Prüfen von Erfüllbarkeit und Normalformen. Das System ist modular aufgebaut und erlaubt die Kombination mehrerer Aufgabentypen zu zusammenhängenden Übungseinheiten. Für jede Aufgabenart können eigene Feedbackmechanismen definiert werden, die den Studierenden unmittelbare Rückmeldungen zu ihren Eingaben liefern (Geck u. a. 2018).

Logic-Muse verfolgt einen weitergehenden Ansatz und modelliert den individuellen Lernstand der Studierenden mithilfe eines Bayesianischen Netzwerks. Das System wurde für

das logische Schließen in der Aussagenlogik entwickelt und stellt Übungsaktivitäten in verschiedenen Kontexten bereit. Auf Basis eines von Experten validierten Fehlerkatalogs wählt das System die nächste Aufgabe gezielt so aus, dass sie die noch nicht beherrschten Kompetenzen adressiert. In einer Studie mit knapp 300 Studierenden wurde das Lernstandsmodell sowohl mit klassischem Bayesian Knowledge Tracing als auch mit einem neuronalen Ansatz evaluiert (Nkambou u. a. 2023).

LogicLearner ist eine an der Columbia University entstandene Open-Source-Webanwendung, die auf das schrittweise Bearbeiten aussagenlogischer Beweise ausgerichtet ist und in einem kürzlich erschienenen Preprint beschrieben wird. Das System validiert jeden einzelnen Beweisschritt, identifiziert Fehler und kann auf Basis eines integrierten Beweisalgorithmus Hinweise erzeugen, die den Studierenden schrittweise zur Lösung führen. Ergänzend verfügt das System über einen Aufgabengenerator, der automatisch Beweisaufgaben mit konfigurierbarer Schrittzahl erzeugen kann (Inamdar u. a. 2025).

Die vorgestellten Systeme adressieren überwiegend die schrittweise Beweisführung mittels Inferenzregeln oder das Modellieren logischer Szenarien. Das Bestimmen von Variablenbelegungen aus einer gegebenen Prämissenmenge, also das semantische Schließen über die Menge der erfüllenden Modelle, wird in den betrachteten Arbeiten nicht als eigenständiger Aufgabentyp behandelt. Ebenso setzen die genannten Systeme in der Regel auf vordefinierte oder manuell erstellte Aufgabenbanken, während eine automatische, regelbasierte Aufgabengenerierung mit integrierter algorithmischer Lösungsbestimmung in dieser Form nicht beschrieben wird.

3.2 Einsatz generativer Sprachmodelle in Lern- und Feedbackszenarien

Neben den in Kapitel 3.1 vorgestellten regelbasierten und wissensmodellierten Systemen werden zunehmend auch große Sprachmodelle (Large Language Models, LLMs) in digitalen Lernumgebungen eingesetzt. Ihr Potenzial liegt insbesondere in der Fähigkeit, natürlichsprachliche Rückmeldungen zu erzeugen, Fehler in studentischen Eingaben zu diagnostizieren und als dialogbasierte Tutoren zu agieren. Dieser Abschnitt beleuchtet den aktuellen Einsatz von LLMs in Lern- und Feedbackszenarien und ordnet deren Stärken und Grenzen im Hinblick auf den vorliegenden Anwendungskontext ein.

Aktuelle Studien zeigen, dass LLMs durchaus in der Lage sind, studentische Fehler zu erkennen und darauf bezogene Hinweise zu formulieren. In einer Untersuchung zum Einsatz von GPT-4 innerhalb eines intelligenten Tutorensystems für College Algebra konnte das Modell Fehler mit einer Genauigkeit von etwa 80 % diagnostizieren. Gleichzeitig erwies sich jedoch ein erheblicher Anteil der erzeugten Hinweise als zu allgemein, inhaltlich falsch oder die Lösung vorwegnehmend (Reddig, Arora und MacLellan 2025). Dieses Ergebnis verdeutlicht ein grundlegendes Spannungsfeld: LLMs erzeugen sprachlich flüssige und oft plausibel wirkende Rückmeldungen, deren fachliche Korrektheit jedoch nicht

garantiert ist.

Ein zentrales Problem in diesem Zusammenhang ist das sogenannte Halluzinieren, also die Erzeugung von Ausgaben, die syntaktisch korrekt, aber inhaltlich falsch oder unbegründet sind. Dieses Phänomen tritt domänenübergreifend auf (Ji u. a. 2023). Für Aufgaben, die exaktes logisches oder mathematisches Schließen erfordern, ist es besonders problematisch, da bereits ein einzelner fehlerhafter Zwischenschritt eine gesamte Ableitung ungültig machen kann. Dass Sprachmodelle gerade bei formallogischen Aufgaben an systematische Grenzen stoßen, zeigen auch die Befunde von Mirzadeh u. a. (2025).

Für den Einsatz in Lernsystemen ist das besonders kritisch, da fehlerhafte Rückmeldungen nicht nur den Lernprozess behindern, sondern Fehlvorstellungen bei Studierenden verstärken können. So stellen die Autoren von LogicLearner fest, dass ChatGPT bei aussagenlogischen Beweisaufgaben zwar gelegentlich korrekte Antworten liefert, die Studierenden jedoch nicht schrittweise zur Lösung hinführt, den bisherigen Bearbeitungsfortschritt nicht berücksichtigt und keinen strukturierten Wiederholungs- oder Übungsmodus bietet (Inamdar u. a. 2025). Damit fehlen wesentliche didaktische Funktionen, die für ein systematisches Übungssystem erforderlich wären.

Gleichzeitig ist festzuhalten, dass LLMs und regelbasierte Systeme einander nicht ausschließen. Vielmehr lassen sich LLMs als ergänzende Komponente in bestehende Systeme integrieren, etwa für die Erzeugung natürlichsprachlicher Erklärungen auf Basis einer algorithmisch gesicherten Lösung. Die in der Literatur beschriebenen Ansätze deuten darauf hin, dass intelligente Tutorensysteme als fachliche Verankerung dienen können, um die Schwächen von LLMs gezielt zu kompensieren (Reddig, Arora und MacLellan 2025). Eine solche hybride Architektur setzt jedoch voraus, dass die fachliche Korrektheit des Systems nicht vom Sprachmodell abhängt, sondern unabhängig davon gewährleistet ist.

Für die vorliegende Arbeit ergibt sich daraus eine klare Konsequenz: Da die Aufgaben der Übungsplattform auf formal-logischen Prämissenmengen beruhen, deren Lösung eindeutig bestimmbar ist, wird die fachliche Auswertung vollständig durch einen algorithmischen Solver übernommen (vgl. Abschnitt 4.3). Die Korrektheit der Lösung ist damit nicht probabilistisch, sondern formal gesichert. Dieser Ansatz schließt eine spätere Ergänzung durch LLM-basierte Komponenten, etwa im Bereich der natürlichsprachlichen Feedbackerzeugung, nicht aus, setzt aber die algorithmische Lösungsbestimmung als unverzichtbare Grundlage voraus.

3.3 Einordnung der vorliegenden Arbeit

Die in Kapitel 3.1 vorgestellten Systeme unterstützen Studierende beim Erlernen formaler Logik auf unterschiedliche Weise, adressieren dabei jedoch überwiegend die schrittweise Beweisführung mittels Inferenzregeln oder das Modellieren logischer Szenarien. Die

vorliegende Arbeit verfolgt einen anderen Schwerpunkt: Im Zentrum steht das Bestimmen von Variablenbelegungen aus gegebenen Prämissenmengen, also die Frage, welche Aussagenwerte sich aus einer Formelmenge zwingend, gar nicht oder nicht eindeutig ableiten lassen. Dieser Aufgabentyp orientiert sich unmittelbar an den Lehrmaterialien der zugrunde liegenden Vorlesung von Hofmann (2026) und wurde in den betrachteten Arbeiten nicht als eigenständiger Übungsgegenstand behandelt.

Gegenüber den in Kapitel 3.2 betrachteten LLM-basierten Ansätzen setzt die vorliegende Arbeit bewusst auf eine algorithmische Lösungsbestimmung, die fachliche Korrektheit formal garantiert. Aufgabengenerierung, Lösung und Bewertung erfolgen regelbasiert und sind nicht von der Ausgabequalität eines Sprachmodells abhängig. Der Lernstand wird ebenfalls regelbasiert über vordefinierte Schwellenwerte und Levelstrukturen erfasst, ohne ein individuelles Lernstandsmodell im Sinne der in Kapitel 3.1 beschriebenen Systeme einzusetzen.

Der Beitrag der Arbeit liegt damit weniger in der Entwicklung einzelner neuartiger Verfahren als vielmehr in der integrierten Zusammenführung von automatischer Aufgabengenerierung, algorithmischer Lösungsbestimmung, standardisierter Eingabe und Bewertung sowie einer strukturierten Lernumgebung zu einem geschlossenen Übungssystem für spezifische Aufgabentypen der Aussagenlogik.

4 Konzeption und Entwurf

Aufbauend auf den in Kapitel 2 beschriebenen fachlichen Grundlagen und der in Abschnitt 1.3 formulierten Zielsetzung werden im Folgenden die konzeptionellen Entscheidungen erläutert, die bei der Gestaltung der Übungsplattform getroffen wurden. Die Konzeption orientiert sich dabei an drei übergeordneten Gestaltungsprinzipien:

Erstens soll die fachliche Korrektheit der Plattform nicht von Heuristiken oder probabilistischen Verfahren abhängen, sondern algorithmisch abgesichert sein. Aufgabengenerierung, Lösungsbestimmung und Bewertung erfolgen daher ausnahmslos regelbasiert und formal fundiert.

Zweitens soll die Plattform einen strukturierten Kompetenzaufbau unterstützen, der Studierende schrittweise von einfacheren zu komplexeren Aufgabenformen führt. Dazu gehören eine gezielte Abstufung der Schwierigkeitsgrade, eine gesteuerte Freischaltlogik zwischen Aufgabentypen sowie eine fachliche Einrahmung durch begleitende Inhalte zu den Grundlagen der Aussagenlogik und weiterführende Referenzen.

Drittens sollen motivationsfördernde Elemente den Übungsprozess unterstützen und das selbstständige Lernen im Alltag erleichtern. Hierzu zählen insbesondere eine sichtbare Fortschrittsdarstellung, ein Levelsystem mit serienbasiertem Aufstieg sowie personalisierbare Profildarstellungen.

Aus diesen Prinzipien ergibt sich ein Nutzungsablauf, der die verschiedenen Komponenten der Plattform zu einem zusammenhängenden Lernprozess verbindet: Nach der Registrierung durchlaufen Studierende zunächst einen geführten Lernpfad, in dem Aufgaben mit steigendem Schwierigkeitsgrad bearbeitet werden. Jede Aufgabe wird nach der Bearbeitung automatisch ausgewertet; bei fehlerhaften Antworten erhalten die Studierenden eine unmittelbare, ergebnisbezogene Rückmeldung. Der Lernfortschritt wird dabei persistent gespeichert und über die Benutzeroberfläche sichtbar gemacht. Erst nach Abschluss aller Stufen des geführten Lernpfads wird ein freier Übungsmodus freigeschaltet, in dem Aufgabentypen und Schwierigkeitsgrade frei wählbar sind.

Die nachfolgenden Abschnitte beschreiben die einzelnen konzeptionellen Entscheidungen im Detail: die Abbildung der fachlichen Aufgabentypen und deren Schwierigkeitsmodell (Abschnitt 4.1), die Strategie der Aufgabengenerierung (Abschnitt 4.2), die Wahl und Anpassung des Lösungsverfahrens (Abschnitt 4.3), das Eingabe- und Bewertungskonzept (Abschnitt 4.4), das Feedbackkonzept (Abschnitt 4.5), die Gestaltung des Lernpfads und der Progression (Abschnitt 4.6) sowie die Systemarchitektur (Abschnitt 4.7).

4.1 Aufgabentypen und Schwierigkeitsmodell

Die in Abschnitt 2.2 beschriebenen zwei Formen des logischen Schließens bilden die fachliche Grundlage für die Aufgabentypen der Plattform. Das direkte Schließen, bei dem sich Variablenbelegungen unmittelbar aus den Prämissen ableiten lassen, wird als Aufgabentyp `DIRECT_INFERENCE` umgesetzt. Das Schließen mithilfe einer Fallunterscheidung wird als Aufgabentyp `CASE_SPLIT` abgebildet.

4.1.1 Schwierigkeitsparameter

Der Schwierigkeitsgrad wird für beide Aufgabentypen über eine regelbasierte Konfiguration gesteuert. Pro Aufgabentyp und Level legt diese Konfiguration fünf Parameter fest: die Anzahl der Aussagenvariablen, die Anzahl der Prämissen, die maximale Verschachtelungstiefe der Formeln, die Menge der zulässigen logischen Operatoren sowie deren relative Gewichtung bei der Formelgenerierung. Die konkreten Belegungen dieser Parameter für alle Level sind in Tabelle 1 zusammengefasst.

Tabelle 1: Schwierigkeitskonfiguration nach Aufgabentyp und Level

Aufgabentyp	Level	Variablen	Prämissen	Max. Tiefe	Erlaubte Operatoren
DIRECT_INFERENCE	1	2	2	1	$\neg, \vee, \rightarrow, \oplus, \leftrightarrow$
DIRECT_INFERENCE	2	3	2	2	$\neg, \wedge, \vee, \rightarrow, \oplus, \leftrightarrow$
DIRECT_INFERENCE	3	4	2	2	$\neg, \wedge, \vee, \rightarrow, \oplus, \leftrightarrow$
DIRECT_INFERENCE	4	4	2	3	$\neg, \wedge, \vee, \rightarrow, \oplus, \leftrightarrow$
CASE_SPLIT	1	3	4	1	$\neg, \vee, \rightarrow, \oplus, \leftrightarrow$
CASE_SPLIT	2	4	5	2	$\neg, \wedge, \vee, \rightarrow, \oplus, \leftrightarrow$
CASE_SPLIT	3	5	6	2	$\neg, \wedge, \vee, \rightarrow, \oplus, \leftrightarrow$

Zwei Aspekte der Konfiguration verdienen eine nähere Erläuterung. Erstens wird die Konjunktion ($\wedge$) in den jeweils ersten Leveln beider Aufgabentypen bewusst ausgeschlossen. Der Grund dafür liegt in der Natur des Operators: Tritt eine Konjunktion $A \wedge B$ als eigenständige Prämisse auf, so legt sie beide Teilaussagen unmittelbar fest und verkürzt dadurch den Ableitungsprozess so stark, dass kaum noch ein Schlussvorgang stattfindet. Ohne Konjunktion müssen die Studierenden tatsächlich über die Wahrheitsbedingungen der verbleibenden Operatoren nachdenken, was dem didaktischen Zweck der Einstiegslevel besser entspricht. Mit wachsender Anzahl der Variablen und Prämissen sowie zunehmender Formeltiefe steigt die Wahrscheinlichkeit, dass sich die Schlussfolgerungen nicht mehr unmittelbar aus den Prämissen ergeben. In diesem Kontext wird die Konjunktion dann als sinnvolle Ergänzung aufgenommen, da sie die Komplexität der Ableitungsprozesse erhöht und damit besser zum Lernziel passt.

Zweitens unterscheidet sich die Prämissenanzahl zwischen den beiden Aufgabentypen deutlich. DIRECT_INFERENCE-Aufgaben verwenden durchgängig zwei Prämissen. Dies orientiert sich an den zugrunde liegenden Lehrmaterialien von Hofmann (2026), in denen Aufgaben zum direkten Schließen ebenfalls auf Basis weniger Prämissen gestellt werden. Bei CASE_SPLIT-Aufgaben steigt die Prämissenanzahl dagegen von vier auf sechs an. Eine Fallunterscheidung setzt voraus, dass zwischen den Variablen genügend Abhängigkeiten bestehen, um verschiedene Fälle mit unterschiedlichen Konsequenzen zu erzeugen. Die steigende Prämissenanzahl über die Level hinweg bietet sich daher als natürlicher Hebel zur Schwierigkeitssteigerung an, da mit jeder zusätzlichen Prämisse die Zahl der zu berücksichtigenden Zusammenhänge wächst.

Ergänzend zu den in der Tabelle aufgeführten Parametern wird jedem Operator ein

Gewicht zugeordnet, das seine Wahrscheinlichkeit bei der zufälligen Formelerzeugung steuert. So erhalten die Operatoren $\oplus$ und $\leftrightarrow$ in allen Leveln ein reduziertes Gewicht, da diese in den zugrunde liegenden Lehrmaterialien seltener vorkommen.

4.1.2 Regelbasiert statt adaptiv

Das Schwierigkeitsmodell ist bewusst regelbasiert gestaltet. Wie in der Abgrenzung der Arbeit (vgl. Abschnitt 1.4) beschrieben, wurde auf einen adaptiven Ansatz verzichtet, bei dem das System sein Verhalten auf Basis individuellen Nutzerverhaltens dynamisch anpasst. Ein solches Modell würde die Komplexität der Anwendung deutlich steigern und setzt empirische Daten über Lernverläufe voraus, die im Rahmen eines Prototyps nicht verfügbar sind. Die regelbasierte Konfiguration hat demgegenüber den Vorteil, dass die Schwierigkeitsparameter transparent definiert sind und sich bei Bedarf ohne algorithmische Anpassungen ändern lassen.

4.1.3 Levelstruktur und Aufteilung der Schwierigkeitsstufen

Für den Aufgabentyp DIRECT INFERENCE sind vier Level vorgesehen. Die Levelstruktur wurde unter Berücksichtigung der zugrunde liegenden Lehrmaterialien entworfen (Hofmann 2026), folgt deren Aufgabenabfolge jedoch nicht schematisch. Ausschlaggebend war vielmehr das Ziel, den Schwierigkeitsgrad fein abgestuft zu variieren, ohne den grundlegenden Lösungsansatz zu verändern. Die Studierenden wenden in jedem Level dasselbe Prinzip des direkten Schließens an, jedoch auf Formeln mit zunehmender Variablenanzahl (von 2 auf 4), steigender Verschachtelungstiefe (von 1 auf 3) und einer konstanten Anzahl von 2 Prämissen an. Durch die schrittweise Erweiterung der Operatormenge und der Formeltiefe entsteht eine wahrnehmbare, aber nicht sprunghafte Steigerung des Schwierigkeitsgrads zwischen den Leveln.

Für den Aufgabentyp CASE SPLIT sind drei Level vorgesehen. Die geringere Levelanzahl erklärt sich durch den qualitativ anderen Charakter dieses Aufgabentyps: Im Vergleich zu DIRECT INFERENCE steigt bei CASE SPLIT die Komplexität pro Level stärker an. Jedes Level erweitert sowohl die Variablenanzahl als auch die Prämissenanzahl gleichzeitig (von 3 Variablen mit 4 Prämissen über 4 Variablen mit 5 Prämissen bis zu 5 Variablen mit 6 Prämissen). Da bei diesem Aufgabentyp stets eine vollständige Fallunterscheidung durchgeführt werden muss, führt jede zusätzliche Variable zu einem größeren Lösungsaufwand. Drei Level decken den didaktisch sinnvollen Bereich ab, ohne Zwischenstufen einzuführen, die sich nur marginal voneinander unterscheiden würden. Zusätzlich ist die Aufgabengenerierung für CASE SPLIT rechenintensiver, da die Validierungskriterien strenger sind und deutlich mehr Kandidaten verworfen werden müssen, was die praktische Machbarkeit feingranularer Zwischenstufen einschränkt.

4.2 Aufgabengenerierung

Die Aufgabengenerierung verfolgt das Ziel, für jeden Aufgabentyp und jedes Level automatisch Prämissenmengen zu erzeugen, die sowohl fachlich korrekt als auch didaktisch geeignet sind. Dabei reicht es nicht aus, beliebige aussagenlogische Formeln zufällig zusammensetzen, da die überwiegende Mehrheit solcher Zufallskombinationen keine sinnvollen Übungsaufgaben ergibt. Stattdessen muss sichergestellt werden, dass die erzeugten Prämissenmengen bestimmte strukturelle und inhaltliche Eigenschaften besitzen, die sie für den jeweiligen Aufgabentyp geeignet machen.

4.2.1 Generate-and-Filter-Ansatz

Die Aufgabengenerierung folgt einem Generate-and-Filter-Ansatz. In einem ersten Schritt werden auf Basis der in Abschnitt 4.1 beschriebenen Schwierigkeitsparameter zufällige Prämissenmengen erzeugt. Dazu wird für jede Prämisse rekursiv eine Formel aufgebaut, indem aus der für das jeweilige Level zulässigen Operatormenge ein Operator gewählt und auf zufällig ausgewählte Variablen oder bereits erzeugte Teilformeln angewendet wird. Die maximale Verschachtelungstiefe und die Operatorgewichte steuern dabei die Struktur der entstehenden Formeln.

In einem zweiten Schritt wird die erzeugte Prämissenmenge durch aufgabentypspezifische Validierungskriterien geprüft. Nur wenn alle Kriterien erfüllt sind, wird die Aufgabe akzeptiert. Andernfalls wird die Prämissenmenge verworfen und eine neue erzeugt. Dieser Vorgang wird so lange wiederholt, bis eine gültige Aufgabe gefunden wurde.

Die Validierungskriterien sind für die beiden Aufgabentypen unterschiedlich gestaltet, da sich die fachlichen Anforderungen an eine geeignete Aufgabe grundlegend unterscheiden. Die folgenden Abschnitte beschreiben die jeweiligen Kriterien im Detail.

4.2.2 Validierungskriterien für Direct Inference

Eine Prämissenmenge wird als geeignete DIRECT INFERENCE-Aufgabe akzeptiert, wenn sie die folgenden Bedingungen erfüllt:

Keine konstanten Prämissen. Jede Prämisse muss mindestens eine Aussagenvariable enthalten. Konstante Ausdrücke, die unabhängig von jeder Belegung wahr oder falsch sind, tragen nicht zum Schlussprozess bei und werden daher ausgeschlossen.

Vollständige Variablennutzung. Jede der für das jeweilige Level vorgesehenen Variablen muss in mindestens einer Prämisse vorkommen. Ungenutzte Variablen wären für die Studierenden nicht ableitbar und würden die Aufgabe verfälschen.

Nicht-triviale Verknüpfung. Mindestens eine Prämisse muss zwei oder mehr Variablen enthalten. Dieses Kriterium ist insbesondere für Level 1 relevant, da dort nur zwei Prämissen erzeugt werden. Bestünden beide Prämissen jeweils nur aus einer einzelnen Variable, wäre kein logischer Schlussvorgang erforderlich.

Erfüllbarkeit ohne Trivialität. Die Prämissenmenge muss erfüllbar sein, das heißt, es muss mindestens ein Modell existieren (vgl. Abschnitt 2.1.3). Gleichzeitig darf die Prämissenmenge nicht trivial erfüllbar sein, also nicht alle 2^n möglichen Belegungen als Modelle zulassen. Im ersten Fall wäre die Aufgabe widersprüchlich und damit unlösbar, im zweiten Fall enthielte die Prämissenmenge keine verwertbare Information.

Begrenzte Einzelprämissen-Stärke. Keine einzelne Prämisse darf für sich allein bereits drei oder mehr Variablen festlegen. Dieses Kriterium verhindert, dass eine einzige Prämisse den Großteil der Aufgabe löst und die übrigen Prämissen überflüssig werden.

Levelspezifische Trivialitätssteuerung. Ab Level 3 darf keine Prämisse aus einer einzelnen Variable oder deren Negation bestehen, da solche Prämissen den Wahrheitswert unmittelbar festlegen und damit die Aufgabe vereinfachen. In Level 2 hingegen wird genau eine solche triviale Prämisse gefordert, um den Studierenden einen eindeutigen Einstiegspunkt in die Ableitung zu bieten, von dem aus die weiteren Schlüsse gezogen werden können.

Levelübergreifende Trivialitätssteuerung. Prämissen, in denen Konjunktionen einzelner Literale vorkommen (z. B. $A \wedge B \wedge (C \vee D)$), sind nicht zulässig, da sie den Wahrheitswert dieser Variablen unmittelbar festlegen und somit den Schwierigkeitsgrad der Aufgabe reduzieren würden. Solche Konjunktionen sind daher nur innerhalb verschachtelter Ausdrücke erlaubt, wo sie nicht direkt als Ableitungsgrundlage dienen, sondern im Kontext komplexerer logischer Strukturen auftreten (z. B. $(A \wedge B) \vee C \vee D$).

Ableitbarkeit durch direktes Schließen. Aus den gegebenen Prämissen muss sich durch wiederholte Anwendung der Unit Propagation (vgl. Abschnitt 2.1.5) der Wahrheitswert von mindestens $n - 1$ der n Variablen konkret bestimmen lassen. Die Unit Propagation dient hier als hinreichendes Kriterium: Gelingt die Ableitung auf diesem Weg, ist sichergestellt, dass die Aufgabe tatsächlich durch schrittweises direktes Schließen lösbar ist. Dieses Kriterium stellt sicher, dass die Aufgabe tatsächlich zum Aufgabentyp des direkten Schließens passt und dass bei maximal einer Variablen kein konkreter Schluss möglich ist. Das ist so festgelegt, um den Studierenden die Möglichkeit zu geben, die Schlussfolgerungen Schritt für Schritt zu verfolgen, ohne von einer größeren Zahl unbestimmter Variablen überfordert zu werden.

Keine redundanten Prämissen. Das Entfernen einer beliebigen Prämisse muss die Menge der Modelle verändern, also zu einer echt größeren Lösungsmenge führen. Wäre eine Prämisse redundant, trüge sie keine Information zur Aufgabe bei und würde die Studierenden lediglich durch einen scheinbar relevanten, tatsächlich aber wirkungslosen Ausdruck ablenken.

Zusammenhang der Prämissen. Die Prämissen müssen über gemeinsame Variablen einen zusammenhängenden Graphen bilden. Dazu wird jede Prämisse als Knoten betrachtet, und zwischen zwei Prämissen wird eine Kante erzeugt, wenn sie mindestens eine gemeinsame Variable enthalten. Die Prämissenmenge ist nur dann zulässig, wenn dieser Graph zusammenhängend ist. Unverbundene Prämissengruppen würden zu unabhängigen Teilaufgaben führen, die keinen gemeinsamen Schlussvorgang erfordern.

4.2.3 Validierungskriterien für Case Split

Für den Aufgabentyp CASE SPLIT gelten strengere und qualitativ andere Kriterien, da hier nicht das direkte Schließen, sondern die systematische Fallunterscheidung im Vordergrund steht:

Grundlegende Qualitätskriterien. Die Kriterien zu konstanten Prämissen, vollständiger Variablennutzung, Redundanzfreiheit und Zusammenhang gelten analog zum Aufgabentyp DIRECT INFERENCE.

Anzahl an Modellen. Die Prämissenmenge darf maximal zwei erfüllende Modelle besitzen. Diese Bedingung stellt sicher, dass für maximal eine Variable kein konkreter Schluss möglich ist, während alle anderen Variablen durch die Prämissen festgelegt sind. Das gilt ebenso analog zum Aufgabentyp DIRECT INFERENCE.

Keine direkte Ableitbarkeit zu Beginn. Durch die initiale Anwendung der Unit Propagation darf sich aus den Prämissen kein einziger Variablenwert direkt ableiten lassen. Dieses Kriterium grenzt den Aufgabentyp scharf von DIRECT INFERENCE ab: Die Studierenden müssen erkennen, dass ein direkter Schluss nicht möglich ist und stattdessen eine Fallunterscheidung erforderlich wird.

Sinnvolle Fallunterscheidungsvariablen. Es müssen mindestens zwei Variablen existieren, bei denen die Annahme eines Wahrheitswerts zu einer konsistenten Lösung führt, während die Annahme des entgegengesetzten Wahrheitswerts einen Widerspruch erzeugt. Dieses Kriterium gewährleistet, dass die Aufgabe eine echte Fallunterscheidung erfordert und mehrere Variablen als sinnvoller Ausgangspunkt für eine solche Fallunterscheidung dienen können. Die Forderung nach mindestens zwei solchen Variablen stellt sicher, dass die Fallunterscheidung kein isoliertes Phänomen einer einzelnen Variable ist, sondern ein strukturelles Merkmal der gesamten Prämissenmenge darstellt.

4.2.4 Begründung des Generierungsansatzes

Die Entscheidung für einen Generate-and-Filter-Ansatz wurde gegenüber zwei Alternativen abgewogen: einem konstruktiven Verfahren, das Aufgaben gezielt aus vordefinierten Mustern zusammensetzt, sowie einem vorlagenbasierten Ansatz mit manuell erstellten Aufgabensammlungen.

Ein konstruktives Verfahren hätte den Vorteil, dass jede erzeugte Aufgabe per Konstruktion die geforderten Eigenschaften besitzt und keine nachträgliche Filterung notwendig wäre. Ein solches Verfahren für die hier benötigten Aufgabentypen zu entwerfen wäre jedoch erheblich aufwendiger, da die Validierungskriterien — insbesondere die Forderung nach Lösbarkeit und höchstens zwei Modellen bei beiden Aufgabentypen oder die Bedingung, dass mindestens $n - 1$ Variablen bei DIRECT INFERENCE durch Unit Propagation ableitbar sein müssen — schwer konstruktiv zu garantieren sind. Darüber hinaus würde ein konstruktives Verfahren tendenziell Aufgaben mit erkennbaren strukturellen Mustern erzeugen, was die Vielfalt der Aufgaben einschränken könnte.

Ein vorlagenbasierter Ansatz mit manuell erstellten Aufgaben wäre zwar einfach umzusetzen, widerspricht jedoch dem Ziel der Plattform, eine unbegrenzte Menge an Übungsaufgaben bereitzustellen. Zudem müssten bei einer Erweiterung der Level oder Schwierigkeitsparameter neue Aufgaben manuell nachgepflegt werden.

Der Generate-and-Filter-Ansatz verbindet die Vorteile beider Alternativen: Die zufällige Formelerzeugung sorgt für eine hohe Aufgabenvielfalt, während die Validierungskriterien die didaktische Qualität sicherstellen. Der Nachteil des Ansatzes besteht darin, dass je nach Aufgabentyp und Level ein erheblicher Anteil der zufällig erzeugten Kandidaten verworfen wird, was die Generierungsdauer erhöhen kann. Diesem Problem wird durch eine Vorgenerierungsstrategie begegnet.

4.2.5 Vorgenerierung

Da die Aufgabengenerierung insbesondere für den Aufgabentyp CASE SPLIT rechenintensiv ist und teilweise mehrere Minuten dauert — die strengeren Validierungskriterien führen dazu, dass deutlich mehr Kandidaten erzeugt und verworfen werden — werden Aufgaben nicht bei jeder Nutzeranfrage in Echtzeit generiert, sondern vorab erzeugt und gespeichert. Für jede Kombination aus Aufgabentyp und Level wird ein Vorrat an validierten Aufgaben in einer Datenbank gespeichert. Fordert ein Nutzer eine Aufgabe an, wird eine noch nicht bearbeitete Aufgabe aus der Datenbank ausgewählt und ausgeliefert. Die konkrete Dimensionierung, der Mechanismus zur Nachgenerierung bei sinkendem Bestand sowie die technische Umsetzung der Vorgenerierung werden in Kapitel 5 beschrieben.

4.3 Wahl und Anpassung des Lösungsverfahrens

Damit die in Abschnitt 4.2 beschriebenen Aufgaben nicht nur generiert und angezeigt, sondern auch automatisch ausgewertet werden können, muss für jede Prämissenmenge die fachlich korrekte Lösung algorithmisch bestimmt werden. Die zentrale Anforderung an das Lösungsverfahren ergibt sich unmittelbar aus der Aufgabenstellung: Für jede Aussagenvariable einer gegebenen Prämissenmenge soll ermittelt werden, ob sie aus den Prämissen zwingend wahr, zwingend falsch oder nicht eindeutig festgelegt ist (vgl. Abschnitt 1.3). In diesem Abschnitt wird begründet, warum der in Abschnitt 2.3 beschriebene Deletion-Algorithmus auf Basis boolescher Tabellen als Lösungsverfahren gewählt

wurde, und in welcher Form er für den vorliegenden Anwendungsfall angepasst wird.

4.3.1 Anforderungen an das Lösungsverfahren

Aus der Zielsetzung der Plattform ergeben sich drei wesentliche Anforderungen an das Lösungsverfahren. Erstens muss es die Erfüllbarkeit einer gegebenen Prämissenmenge entscheiden können, da widersprüchliche Prämissenmengen von der Plattform nicht als Aufgaben verwendet werden sollen. Zweitens muss es für jede einzelne Variable der Prämissenmenge bestimmen können, ob deren Wahrheitswert durch die Prämissen eindeutig festgelegt ist oder ob mehrere Belegungen mit den Prämissen vereinbar sind. Drittens muss das Verfahren für die in der Plattform vorgesehenen Variablenmengen von zwei bis fünf Variablen effizient genug sein, um die Lösung innerhalb einer für den Übungsbetrieb akzeptablen Zeit zu berechnen.

Über diese drei Grundanforderungen hinaus ergibt sich aus der didaktischen Zielsetzung eine weitere Anforderung: Das Verfahren soll nicht nur eine korrekte Lösung liefern, sondern auch Zwischenergebnisse bereitstellen, die für eine spätere Feedbackgenerierung genutzt werden können. Insbesondere soll es möglich sein, auf Basis der im Lösungsprozess erzeugten Datenstrukturen Gegenbeispiele zu konstruieren, die den Studierenden bei fehlerhaften Antworten veranschaulichen, warum eine bestimmte Schlussfolgerung nicht zwingend ist (vgl. Abschnitt 4.5).

4.3.2 Abgrenzung gegenüber alternativen Verfahren

Im Rahmen der Konzeption wurden neben dem Deletion-Algorithmus zwei weitere Ansätze zur Lösungsbestimmung betrachtet.

Ein naheliegender Ansatz besteht in der vollständigen Enumeration aller Belegungen über eine Wahrheitstafel. Bei n Variablen werden alle 2^n möglichen Belegungen erzeugt und jeweils geprüft, ob sie die gesamte Prämissenmenge erfüllen. Aus der resultierenden Menge der Modelle lässt sich anschließend für jede Variable bestimmen, ob sie in allen Modellen denselben Wert annimmt oder nicht. Für die in der Plattform vorgesehenen Variablenmengen von zwei bis fünf Variablen wäre dieser Ansatz effizient genug. Er wird tatsächlich in der Aufgabengenerierung eingesetzt, um die Modellmenge einer Kandidatenprämissenmenge zu bestimmen und die Validierungskriterien zu prüfen (vgl. Abschnitt 4.2). Für das eigentliche Lösungsverfahren wurde er jedoch nicht gewählt, da er keine strukturierten Zwischenergebnisse erzeugt, die für die Feedbackgenerierung nutzbar wären. Die vollständige Aufzählung liefert lediglich die Menge der Modelle als Endergebnis, ohne Informationen darüber zu bewahren, welche Prämissen welche Variablen einschränken und auf welchem Weg eine bestimmte Belegung ausgeschlossen wurde.

Als zweite Alternative wurden Backtracking-basierte Suchverfahren betrachtet, wie sie in der Literatur unter anderem als Branch-and-Bound-Ansätze für aussagenlogische Erfüllbarkeitsprobleme beschrieben werden (Gu u. a. 1997). Solche Verfahren explorieren

den Suchraum der Variablenbelegungen systematisch, indem sie schrittweise Variablen festlegen und Teilbäume abschneiden, sobald ein Widerspruch erkannt wird. Für reine Erfüllbarkeitsprobleme sind solche Verfahren in der Praxis oft sehr effizient. Für den vorliegenden Anwendungsfall sind sie jedoch nicht unmittelbar geeignet, da sie primär darauf ausgelegt sind, eine einzelne erfüllende Belegung zu finden oder die Unerfüllbarkeit nachzuweisen (Gu u. a. 1997). Die Bestimmung der Marginalinformation pro Variable — also die Frage, ob eine Variable über alle Modelle hinweg einen festen Wert annimmt — erfordert bei diesen Verfahren zusätzlichen Aufwand, etwa die vollständige Enumeration aller Lösungen oder wiederholte Aufrufe mit zusätzlichen Annahmen.

4.3.3 Begründung der Verfahrenswahl

Das in Abschnitt 2.3 beschriebene Lösungsverfahren basiert auf dem von Díaz-Macías und Moral (2023) vorgestellten Deletion-Algorithmus auf Basis boolescher Tabellen. Dieser Algorithmus wurde als Grundlage für das Lösungsverfahren der Plattform gewählt, da er die genannten Anforderungen in besonderer Weise erfüllt.

Der Algorithmus liefert als Ergebnis nicht nur eine Entscheidung über die Erfüllbarkeit der Prämissenmenge, sondern stellt durch die schrittweise Variablenelimination im Forward Pass und die anschließende Rekonstruktion im Backward Pass (vgl. Abschnitt 2.3.3 und 2.3.4) genau die benötigte Marginalinformation bereit: Für jede Variable kann im Backward Pass geprüft werden, ob sie unter den verbleibenden Konsistenzbedingungen nur den Wert wahr, nur den Wert falsch oder beide Werte annehmen kann. Diese dreistufige Unterscheidung entspricht unmittelbar dem in Abschnitt 1.3 beschriebenen Antwortschema der Plattform.

Darüber hinaus bewahrt der Algorithmus in jedem Eliminationsschritt die verwendeten Tabellen als sogenannte Buckets (Díaz-Macías und Moral 2023). Diese gespeicherten Zwischenergebnisse ermöglichen es, den Lösungsprozess nach Abschluss schrittweise zu rekonstruieren. Im Kontext der vorliegenden Plattform wird diese Eigenschaft in der prototypischen Feedbackkomponente genutzt: Bei einem bestimmten Fehlertyp — wenn ein Studierender eine Variable fälschlich als eindeutig festgelegt einstuft — traversiert die Feedbackkomponente die gespeicherten Buckets in umgekehrter Eliminationsreihenfolge, um eine vollständige, konsistente Belegung zu konstruieren, die als Gegenbeispiel dient und zeigt, dass die Variable auch den entgegengesetzten Wahrheitswert annehmen kann (vgl. Abschnitt 4.5). Die übrigen Feedbackfälle — etwa die Rückmeldung bei einer falsch bestimmten Belegung oder einer verpassten Schlussfolgerung — basieren hingegen auf einem einfachen Vergleich der Nutzereingabe mit dem Solver-Ergebnis und könnten auch ohne die Bucket-Zwischenergebnisse realisiert werden. Die Wahl des Deletion-Algorithmus wurde daher nicht allein durch das aktuelle Feedbacksystem motiviert, sondern auch durch die Überlegung, dass die gespeicherten Zwischenergebnisse bei einer späteren Erweiterung der Feedbackkomponente — etwa für die Konstruktion schrittweiser Erklärungen oder die Identifikation fehlerverursachender Prämissen — eine natürliche Grundlage bieten.

Die dem Verfahren zugrunde liegende Darstellungsform der booleschen Tabellen ist für

den vorliegenden Anwendungskontext ebenfalls geeignet. Die Tabellen repräsentieren die zulässigen Belegungen einer Formel oder Formelmenge explizit als mehrdimensionale Wahrheitswerttabellen (vgl. Abschnitt 2.3). Diese Darstellung ist für kleine Variablenmengen nicht nur effizient berechenbar, sondern auch konzeptionell nachvollziehbar, da sie direkt an die aus der Lehre bekannten Wahrheitstafeln anknüpft.

4.3.4 Anpassung gegenüber dem Literaturansatz

Der in Díaz-Macías und Moral (2023) beschriebene Algorithmus umfasst neben der Grundvariante der Marginalisierung mehrere Optimierungsstrategien, darunter eine paarweise Marginalisierung, die Ausnutzung funktionaler Abhängigkeiten zwischen Variablen sowie Strategien zur optimalen Wahl der Marginalisierungsmethode pro Eliminationsschritt. Im Rahmen der vorliegenden Arbeit wird ausschließlich die Grundvariante des Verfahrens umgesetzt. Die Entscheidung gegen die Implementierung der weitergehenden Optimierungen wurde bewusst getroffen und lässt sich wie folgt begründen.

Die in der Plattform erzeugten Aufgaben umfassen maximal fünf Aussagenvariablen. Bei fünf Variablen besitzt eine vollständige boolesche Tabelle $2^5 = 32$ Einträge. Selbst wenn im Verlauf des Forward Passes mehrere Tabellen kombiniert werden müssen, bleiben die resultierenden Datenstrukturen klein genug, um ohne wahrnehmbare Verzögerung verarbeitet zu werden. Die im Paper beschriebenen Optimierungen zielen darauf ab, die Tabellengrößen bei deutlich größeren Variablenmengen zu reduzieren, indem beispielsweise funktionale Abhängigkeiten ausgenutzt werden, um die Dimensionalität der Tabellen zu verringern. Für Variablenmengen der hier verwendeten Größenordnung bieten diese Optimierungen keinen praktischen Vorteil.

Darüber hinaus erhöht jede zusätzliche Optimierungsstrategie die Komplexität der Implementierung und erschwert die Nachvollziehbarkeit des Verfahrens. Da die vorliegende Arbeit einen Prototyp entwickelt, der die Machbarkeit des Gesamtkonzepts demonstrieren soll (vgl. Abschnitt 1.4), wurde der Implementierungsaufwand bewusst auf die für den Anwendungsfall ausreichende Grundvariante beschränkt. Eine spätere Erweiterung um Optimierungsstrategien wäre möglich, falls die Plattform auf größere Variablenmengen ausgedehnt werden sollte.

4.4 Eingabe- und Bewertungskonzept

In Abschnitt 1.3 wurde das dreistufige Antwortschema eingeführt, das pro Variable zwischen *wahr*, *falsch* und *kein konkreter Schluss möglich* unterscheidet. Dieses Schema bildet die Grundlage sowohl für die Eingabegestaltung als auch für die automatische Bewertung innerhalb der Plattform. Im Folgenden wird beschrieben, wie das Antwortschema in eine konkrete Eingabeform überführt wird, welche Alternativen betrachtet und verworfen wurden und nach welcher Logik die Bewertung erfolgt.

4.4.1 Gestaltung der Eingabe

Die Eingabe ist so gestaltet, dass für jede Variable der Prämissenmenge genau eine Antwort aus den drei Optionen *wahr*, *falsch* und *kein konkreter Schluss möglich* ausgewählt wird. Technisch wird dies über eine Gruppe von Radio-Buttons realisiert, die pro Variable angezeigt wird. Die Wahl dieser Eingabeform wurde aus mehreren Gründen getroffen.

Erstens erzwingt die Radio-Button-Darstellung, dass für jede Variable genau eine der drei Optionen gewählt wird. Es ist weder möglich, eine Variable ohne Antwort zu lassen, noch mehrere Optionen gleichzeitig auszuwählen. Dadurch entfällt die Notwendigkeit, unvollständige oder mehrdeutige Eingaben zu behandeln, und die Bewertungslogik kann sich auf den reinen Vergleich zwischen gewählter und korrekter Antwort beschränken.

Zweitens ist die Eingabe über Radio-Buttons für die Studierenden ohne Einarbeitungszeit verständlich. Da die Interaktion auf das Auswählen einer Option reduziert ist, kann sich der Nutzer vollständig auf die fachliche Beurteilung konzentrieren, anstatt auf die Bedienung der Eingabeschnittstelle.

4.4.2 Abgrenzung gegenüber alternativen Eingabeformen

Im Rahmen der Konzeption wurden mehrere alternative Eingabeformen betrachtet und verworfen.

Freitexteingabe. Eine Freitexteingabe, bei der Studierende ihre Schlussfolgerungen in natürlicher Sprache oder formaler Notation formulieren, hätte den Vorteil, offenere Antworten zu ermöglichen und möglicherweise Einblick in den Denkprozess der Studierenden zu geben. Für die vorliegende Plattform wurde dieser Ansatz jedoch verworfen, da die automatische Auswertung natürlichsprachlicher Antworten eine Sprachverarbeitung erfordern würde, die über den Umfang eines Prototyps hinausgeht. Auch formale Eingaben in logischer Notation würden zusätzliche Anforderungen an die Eingabevalidierung und die Fehlertoleranz stellen, etwa bei syntaktischen Fehlern oder äquivalenten Umformulierungen.

Wahrheitstafel-Eingabe. Ein alternativer Ansatz bestünde darin, die Studierenden eine Wahrheitstafel ausfüllen zu lassen, anstatt nur das Ergebnis der logischen Analyse pro Variable abzufragen. Dieser Ansatz hätte den Vorteil, den Lösungsweg selbst zum Gegenstand der Bewertung zu machen. Er wurde jedoch verworfen, da er die Aufgabenbearbeitung deutlich aufwändiger gestalten und die Eingabeschnittstelle erheblich komplexer machen würde. Zudem würde sich der Fokus der Übung verschieben: Statt das logische Schlussfolgerungsvermögen zu trainieren, würde primär die korrekte Erstellung von Wahrheitstafeln geübt. Das Ziel der Plattform ist hingegen, die Fähigkeit zu fördern, aus einer gegebenen Prämissenmenge korrekte Schlüsse zu ziehen (vgl. Abschnitt 1.3).

Drag-and-Drop-Zuordnung. Eine weitere Möglichkeit bestünde darin, die Variablen per Drag-and-Drop in Kategorien wie *wahr*, *falsch* oder *unbestimmt* einzuordnen. Funk-

tional wäre diese Eingabeform äquivalent zur Radio-Button-Darstellung, da in beiden Fällen pro Variable genau eine Kategorie zugewiesen wird. Zudem wäre der Implementierungsaufwand höher, ohne dass die didaktische Qualität der Eingabe verbessert würde.

4.4.3 Bewertungslogik

Die Bewertung einer Nutzereingabe erfolgt durch einen variablenweisen Vergleich mit der vom Solver bestimmten korrekten Lösung (vgl. Abschnitt 4.3). Für jede Variable wird geprüft, ob die vom Studierenden gewählte Antwort mit dem Solver-Ergebnis übereinstimmt. Dabei wird jede Variable unabhängig bewertet: Eine Variable gilt als korrekt beantwortet, wenn die gewählte Antwort exakt dem Solver-Ergebnis entspricht, und als fehlerhaft in allen anderen Fällen. Teilweise korrekte Antworten — etwa die Wahl von *wahr*, wenn der korrekte Wert *kein konkreter Schluss möglich* lautet — werden nicht als partiell richtig gewertet, sondern als Fehler behandelt.

Ein Gesamtversuch gilt genau dann als korrekt, wenn alle Variablen der Aufgabe richtig beantwortet wurden. Diese Gesamtbewertung fließt in die Fortschrittslogik ein (vgl. Abschnitt 4.6): Nur ein vollständig korrekter Versuch wird als Erfolg gezählt und erhöht die Erfolgsserie für den Levelaufstieg. Bei einem fehlerhaften Versuch wird die Erfolgsserie des aktuellen Levels zurückgesetzt, das erreichte Level bleibt jedoch erhalten.

Für jede einzelne Variable, die fehlerhaft beantwortet wurde, wird zusätzlich ein variablenspezifisches Feedback erzeugt (vgl. Abschnitt 4.5).

4.5 Feedbackkonzept

In Abschnitt 1.3 wurde als Teilziel der Arbeit formuliert, dass die Plattform bei fehlerhaften Antworten eine unmittelbare, ergebnisbezogene Rückmeldung bereitstellen soll. Im Folgenden wird beschrieben, wie dieses Ziel konzeptionell umgesetzt wird. Dabei wird zunächst die Klassifikation der möglichen Fehlertypen eingeführt, anschließend das jeweilige Feedbackverhalten pro Fehlertyp erläutert und abschließend die bewusste Begrenzung des Feedbacksystems begründet und gegenüber weitergehenden Ansätzen abgegrenzt.

4.5.1 Fehlertypen

Das im vorherigen Abschnitt beschriebene Bewertungskonzept bestimmt pro Variable, ob die Nutzereingabe mit der korrekten Lösung übereinstimmt. Bei einer Abweichung wird der spezifische Fehlertyp aus der Kombination von Nutzereingabe und korrekter Lösung abgeleitet. Es ergeben sich drei Fehlertypen, die jeweils unterschiedliche Rückmeldungen erfordern.

Unnötige Festlegung. Dieser Fehlertyp liegt vor, wenn der Studierende eine Variable als *wahr* oder *falsch* einstuft, obwohl die korrekte Lösung *kein konkreter Schluss möglich* lautet. Der Fehler besteht darin, dass der Studierende der Variable einen eindeutigen Wahrheitswert zuschreibt, den die Prämissenmenge nicht erzwingt. Es handelt sich um

eine Überinterpretation: Die vom Studierenden angenommene Belegung mag mit den Prämissen vereinbar sein, ist aber nicht die einzig mögliche.

Verpasste Schlussfolgerung. Dieser Fehlertyp tritt auf, wenn der Studierende *kein konkreter Schluss möglich* wählt, obwohl die Variable durch die Prämissen eindeutig auf *wahr* oder *falsch* festgelegt ist. Der Fehler besteht darin, dass der Studierende eine logische Konsequenz nicht erkennt, die sich aus den Prämissen zwingend ergibt.

Falsche Belegung. Dieser Fehlertyp liegt vor, wenn der Studierende *wahr* wählt, obwohl die korrekte Lösung *falsch* lautet, oder umgekehrt. Der Fehler besteht darin, dass der Studierende zwar erkennt, dass die Variable eindeutig festgelegt ist, aber den falschen Wahrheitswert bestimmt.

4.5.2 Feedbackverhalten pro Fehlertyp

Für jeden der drei Fehlertypen erzeugt die Feedbackkomponente eine spezifische Rückmeldung. Die Rückmeldungen unterscheiden sich dabei sowohl in ihrem Informationsgehalt als auch in der Art der zugrunde liegenden Berechnung.

Unnötige Festlegung. Bei diesem Fehlertyp geht die Rückmeldung über eine reine Fehlermeldung hinaus: Die Feedbackkomponente konstruiert ein Gegenbeispiel in Form einer vollständigen, konsistenten Belegung aller Variablen, die zeigt, dass die betreffende Variable auch den entgegengesetzten Wahrheitswert annehmen kann. Dieses Gegenbeispiel wird dem Studierenden als konkreter Beleg dafür präsentiert, dass seine Festlegung nicht zwingend aus den Prämissen folgt. Für die Konstruktion des Gegenbeispiels greift die Feedbackkomponente auf die im Lösungsprozess gespeicherten Bucket-Zwischenergebnisse des Solvers zurück (vgl. Abschnitt 4.3.3): Sie durchläuft die Buckets in umgekehrter Eliminationsreihenfolge und bestimmt schrittweise für jede Variable einen konsistenten Wahrheitswert, der mit den Prämissen vereinbar ist und zugleich der vom Studierenden gewählten Belegung widerspricht. Auf diese Weise entsteht eine vollständige Belegung, die als nachvollziehbares Gegenbeispiel dient.

Verpasste Schlussfolgerung. Bei diesem Fehlertyp teilt die Rückmeldung dem Studierenden mit, dass die betreffende Variable durch die Prämissen eindeutig festgelegt ist, und nennt den korrekten Wahrheitswert. Die Rückmeldung basiert auf einem einfachen Vergleich der Nutzereingabe mit dem Solver-Ergebnis und erfordert keinen Zugriff auf die Bucket-Zwischenergebnisse.

Falsche Belegung. Auch bei diesem Fehlertyp beschränkt sich die Rückmeldung auf den Hinweis, dass der gewählte Wahrheitswert nicht korrekt ist, und nennt den tatsächlich aus den Prämissen folgenden Wert. Wie bei der verpassten Schlussfolgerung basiert die Berechnung auf dem Vergleich mit dem Solver-Ergebnis.

4.5.3 Einordnung und Begrenzung

Das beschriebene Feedbackkonzept ist als unmittelbare, ergebnisbezogene Rückmeldung im Sinne von Shute (2007) einzuordnen: Die Studierenden erhalten zeitnah eine Aussage darüber, ob ihre Antwort korrekt ist, und im Fehlerfall einen Hinweis auf die korrekte Lösung oder ein konkretes Gegenbeispiel.

Die Rückmeldung bleibt dabei bewusst auf das Ergebnis beschränkt und geht nicht auf den Denkweg der Studierenden ein. Insbesondere liefert das System keine schrittweise Erklärung, warum eine Variable einen bestimmten Wahrheitswert annimmt, und es diagnostiziert keine zugrundeliegenden Fehlvorstellungen — etwa die in Herman u. a. (2012) beschriebene systematische Fehlinterpretation der Implikation als Konjunktion. Diese Begrenzung ist dem Prototypcharakter der Arbeit geschuldet (vgl. Abschnitt 1.4). Die im Solver gespeicherten Bucket-Zwischenergebnisse bieten jedoch eine Grundlage für weitergehende Feedbackformen, die in einer späteren Ausbaustufe genutzt werden könnten (vgl. Kapitel 7.4).

4.6 Lernpfad und Progression

Die in den vorangehenden Abschnitten beschriebenen Komponenten — Aufgabentypen, Generierung, Lösung, Eingabe und Feedback — werden innerhalb der Plattform nicht als isolierte Funktionen angeboten, sondern in eine strukturierte Lernumgebung eingebettet, die einen schrittweisen Kompetenzaufbau unterstützen soll (vgl. Abschnitt 1.3). Im Folgenden wird beschrieben, wie der Lernpfad aufgebaut ist, nach welcher Logik Inhalte freigeschaltet werden und welche Elemente den Übungsprozess zusätzlich begleiten.

4.6.1 Stufenmodell: Geführter Lernpfad und Freies Üben

Die Plattform unterscheidet zwei Nutzungsmodi. Im geführten Lernpfad bearbeiten Studierende Aufgaben in einer festgelegten Reihenfolge: Zunächst werden alle Level des Aufgabentyps DIRECT INFERENCE durchlaufen, anschließend die Level des Aufgabentyps CASE SPLIT. Diese Reihenfolge spiegelt die fachliche Abhängigkeit wider, da das Schließen mithilfe einer Fallunterscheidung das direkte Schließen als Teilkompetenz voraussetzt (vgl. Abschnitt 2.2). Der Aufgabentyp CASE SPLIT wird daher erst freigeschaltet, wenn alle Level des Typs DIRECT INFERENCE abgeschlossen sind.

Nach Abschluss des geführten Lernpfads wird der Modus *Freies Üben* freigeschaltet. In diesem Modus können Studierende beliebig zwischen Aufgabentypen und Leveln wählen und Aufgaben ohne Einschränkung wiederholen. Der Modus richtet sich an Studierende, die den strukturierten Kompetenzaufbau abgeschlossen haben und gezielt bestimmte Aufgabentypen oder Schwierigkeitsgrade vertiefen möchten.

4.6.2 Levelaufstieg und Schwellenwerte

Innerhalb des geführten Lernpfads erfolgt der Aufstieg zum nächsten Level über eine Erfolgsserie: Der Studierende muss eine definierte Anzahl aufeinanderfolgender Aufgaben

vollständig korrekt lösen. Bei einer fehlerhaften Antwort wird die aktuelle Erfolgsserie auf null zurückgesetzt, das bereits erreichte Level bleibt jedoch erhalten. Dadurch wird einerseits sichergestellt, dass ein Levelaufstieg tatsächlich eine konsistente Beherrschung des aktuellen Schwierigkeitsgrads widerspiegelt, andererseits wird vermieden, dass ein einzelner Fehler den gesamten bisherigen Fortschritt zunichtemacht.

Die erforderliche Länge der Erfolgsserie ist pro Level individuell festgelegt und nimmt mit steigendem Level ab. Für den Aufgabentyp DIRECT INFERENCE betragen die Schwellenwerte fünf, vier, drei und zwei korrekte Aufgaben in Folge für die Level eins bis vier. Für den Aufgabentyp CASE SPLIT betragen sie drei, zwei und eine korrekte Aufgabe in Folge für die Level eins bis drei. Die abnehmenden Schwellenwerte folgen der Überlegung, dass höhere Level inhaltlich anspruchsvollere Aufgaben enthalten (vgl. Abschnitt 4.1) und bereits das Erreichen des jeweiligen Levels eine gewisse Kompetenz voraussetzt. Die konkreten Zahlenwerte beruhen auf einer Einschätzung des Autors und sind nicht empirisch validiert. Die in Abschnitt 1.1 zitierten lerntheoretischen Befunde legen allgemein nahe, dass wiederholte Anwendung den Aufbau prozeduraler Kompetenzen fördert (Donoghue und Hattie 2021), liefern jedoch keine Aussage über die optimale Anzahl an Wiederholungen für die vorliegenden Aufgabentypen.

Eine gleichbleibend hohe Anzahl an Wiederholungen wäre in höheren Leveln unverhältnismäßig und könnte demotivierend wirken, da die Aufgaben dort mehr Zeit und kognitive Anstrengung erfordern. Die geringere Anzahl an erforderlichen richtig beantworteten Aufgaben für den Aufgabentyp CASE SPLIT im Vergleich zum Aufgabentyp DIRECT INFERENCE spiegelt die höhere fachliche Komplexität der Fallunterscheidungsaufgaben wider. Wie in Abschnitt 4.1 beschrieben, umfassen Aufgaben dieses Typs eine größere Anzahl an Variablen und Prämissen sowie die Notwendigkeit einer vollständigen Fallunterscheidung, was den Bearbeitungsaufwand pro Aufgabe deutlich erhöht.

4.6.3 Ergänzende didaktische Elemente

Neben dem Lernpfad und der Levelstruktur stellt die Plattform weitere Elemente bereit, die den Übungsprozess begleiten. Diese lassen sich in drei Bereiche unterteilen: motivationsfördernde Elemente, Lernstandsinformationen sowie fachliche Begleitmaterialien.

Motivationsfördernde Elemente

Im Bereich der Motivationsförderung setzt die Plattform gezielt auf Gamification-Mechanismen. Unter Gamification wird in dieser Arbeit der Einsatz spieltypischer Gestaltungselemente in einem nicht-spielerischen Kontext verstanden (Sailer und Homner 2020). Ein zentrales Element ist dabei die visuelle Gestaltung des Lernpfads selbst. Die einzelnen Level werden als Checkpoints dargestellt, die über Verbindungslinien miteinander verbunden sind. Dabei wird der aktuelle Checkpoint durch eine Animation hervorgehoben, bereits abgeschlossene Checkpoints werden als erfolgreich markiert und noch nicht freigeschaltete Checkpoints als gesperrt gekennzeichnet. Diese Darstellung gibt den Studierenden jederzeit eine räumliche Orientierung über ihren Fortschritt innerhalb des Lernpfads.

Darüber hinaus wird die aktuelle Erfolgsserie während der Aufgabenbearbeitung sichtbar gemacht, und die längste jemals erreichte Serie wird als persönlicher Highscore festgehalten. Bei dem Abschluss eines Aufgabentyps wird den Studierenden eine visuelle Erfolgsrückmeldung angezeigt, die den erreichten Fortschritt würdigt. Nach Abschluss des gesamten Lernpfads wird der Modus Freies Üben freigeschaltet, was als übergeordneter Anreiz für die vollständige Bearbeitung aller Level dient.

Ergänzend bietet die Plattform ein anpassbares Avatar-System, das eine personalisierte Darstellung ermöglicht. Dieses Element dient primär der Identifikation mit der Plattform und der Auflockerung des Übungsalltags, nicht der fachlichen Leistungsbewertung.

Lernstandsinformationen

Um selbstständiges Üben zusätzlich zu unterstützen, macht die Plattform den individuellen Lernstand sichtbar. Eine Statistikübersicht stellt die Gesamtzahl der gelösten Aufgaben, die Anzahl der korrekt gelösten Aufgaben, die aktuelle Erfolgsserie sowie den persönlichen Highscore dar. Ergänzend zeigt eine Fortschrittsdarstellung pro Aufgabentyp an, welche Level bereits abgeschlossen sind, welches Level aktuell bearbeitet wird und wie viele korrekte Antworten in Folge noch für den nächsten Levelaufstieg erforderlich sind. Diese Informationen ermöglichen es den Studierenden, ihren Lernverlauf quantitativ einzuordnen.

Fachliche Begleitmaterialien

Im Bereich der fachlichen Begleitung stellt die Plattform zwei ergänzende Inhaltsseiten bereit. Eine Seite zu den Grundlagen der Aussagenlogik fasst die für die Aufgabenbearbeitung relevanten Konzepte zusammen und bietet Studierenden die Möglichkeit, bei Unsicherheiten die fachlichen Grundlagen nachzuschlagen, ohne die Plattform verlassen zu müssen. Eine Seite mit weiterführenden Referenzen verweist auf die externen zugrundeliegende Lehrmaterialien, auf denen die Aufgabentypen basieren. Beide Seiten sind jederzeit zugänglich und unabhängig vom aktuellen Lernfortschritt nutzbar.

4.7 Systemarchitektur

Die in den vorangehenden Abschnitten beschriebenen Konzepte wurden bisher einzeln betrachtet. In diesem Abschnitt wird beschrieben, wie diese architektonisch zusammenwirken.

4.7.1 Schichtentrennung

Die Plattform ist in drei Schichten gegliedert: eine fachliche Kernlogik, eine Vermittlungsschicht und eine Präsentationsschicht.

Die Kernlogik umfasst den Aufgabengenerator (vgl. Abschnitt 4.2), den Deletion-Algorithmus auf Basis boolescher Tabellen (vgl. Abschnitt 4.3) und die Feedbackkomponente (vgl. Abschnitt 4.5). Diese drei Komponenten besitzen keine Abhängigkeit zu einem Webframework oder einer Datenbank. Die Trennung ist bewusst gewählt: Die fachliche Korrektheit der Aufgabengenerierung, der Lösungsbestimmung und der Feedbackerzeugung soll isoliert prüfbar sein, ohne dass dafür ein laufender Webserver oder eine bestehende Datenbankverbindung erforderlich ist. Tests auf dieser Ebene prüfen ausschließlich die algorithmische Korrektheit und können unabhängig vom Rest der Anwendung ausgeführt werden.

Die Vermittlungsschicht stellt die Schnittstelle zwischen Kernlogik und Präsentationsschicht her. Sie nimmt Anfragen der Präsentationsschicht entgegen, ruft die entsprechenden Komponenten der Kernlogik auf und verwaltet den persistenten Anwendungszustand: Aufgaben, Lösungsversuche, Nutzerfortschritt und Benutzerdaten. Die Bewertungslogik (vgl. Abschnitt 4.4) und die Progressionslogik (vgl. Abschnitt 4.6) sind auf dieser Ebene angesiedelt, da sie sowohl auf das Ergebnis des Solvers als auch auf den gespeicherten Nutzerzustand zugreifen müssen.

Die Präsentationsschicht ist für die Darstellung der Aufgaben, die Entgegennahme der Nutzereingaben und die Anzeige von Feedback und Fortschritt zuständig. Sie trifft keine fachlichen Entscheidungen, sondern gibt die Eingaben des Nutzers an die Vermittlungsschicht weiter und stellt deren Antworten dar.

Ergänzend zu den drei Verarbeitungsschichten greift die Vermittlungsschicht auf einen persistenten Speicher in Form einer Datenbank zu, in dem folgende Kategorien von Daten abgelegt werden: vorgenerierte Aufgaben; Zugangs- und Profildaten eines Nutzers; einzelne Lösungsversuche pro Nutzer, damit dieselbe Aufgabe einem Nutzer nicht erneut zugewiesen wird; der Fortschritt eines Nutzers über Sitzungen hinweg, einschließlich der aktuellen Erfolgsserie und des erreichten Levels; sowie gecachte Solver-Ergebnisse, um wiederholte Berechnungen bei mehrfach bearbeiteten Aufgaben zu vermeiden (vgl. Abschnitt 4.7.3). Der persistente Speicher ist keine eigenständige Verarbeitungsschicht, da er keine fachliche Logik ausführt, sondern ausschließlich als Ressource der Vermittlungsschicht dient. Die konkrete Wahl der Datenbank und das Datenmodell werden in Kapitel 5 beschrieben.

4.7.2 Datenfluss einer Aufgabenbearbeitung

Der Datenfluss einer einzelnen Aufgabenbearbeitung verbindet die in Kapitel 4 beschriebenen Konzepte zu einem zusammenhängenden Ablauf.

Die Präsentationsschicht fordert eine Aufgabe für einen bestimmten Aufgabentyp und ein bestimmtes Level an. Die Vermittlungsschicht wählt eine vorgenerierte, vom anfragenden Nutzer noch nicht bearbeitete Aufgabe aus dem gespeicherten Vorrat aus (vgl.

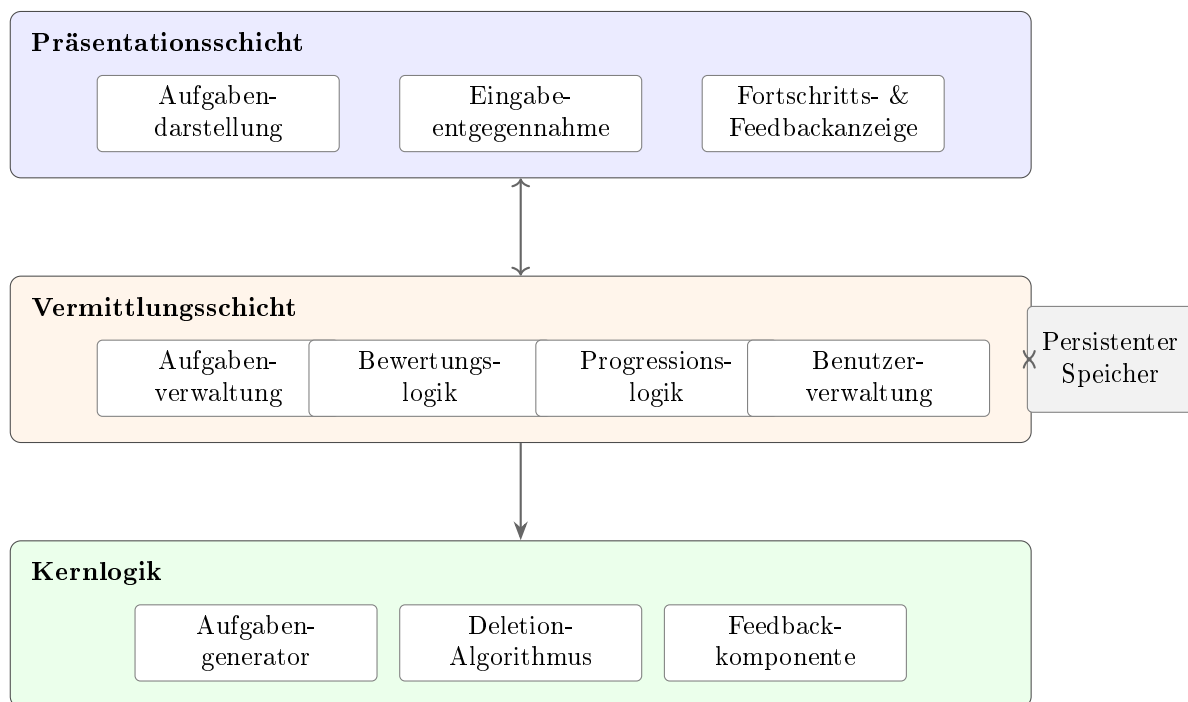
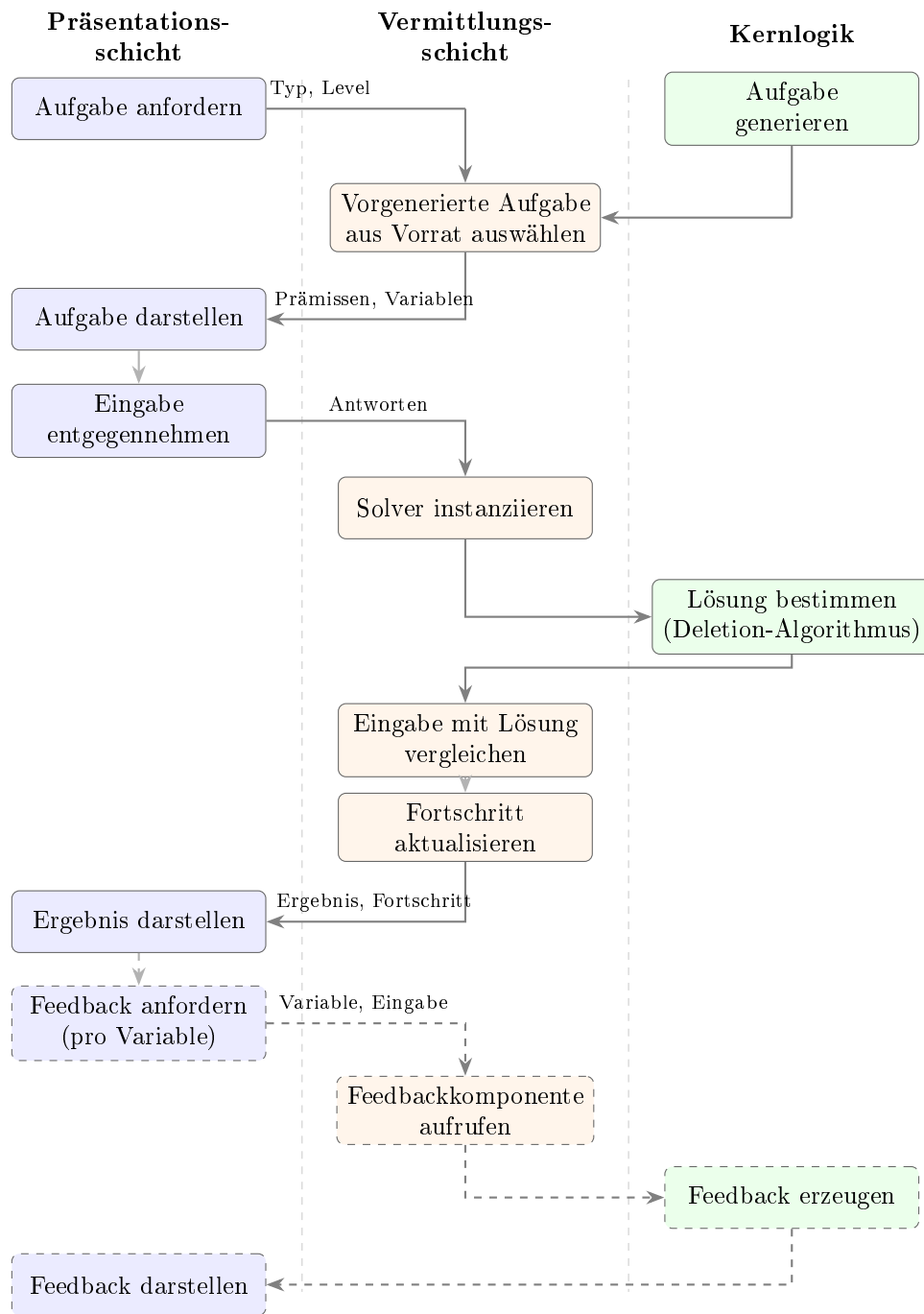


Abbildung 1: Schichtenmodell

Abschnitt 4.2.5) und liefert deren Prämissen und Variablen zurück. Die Präsentationsschicht stellt die Aufgabe dar und nimmt pro Variable eine Antwort gemäß dem in Abschnitt 4.4 beschriebenen Eingabekonzept entgegen.

Nach dem Absenden übermittelt die Präsentationsschicht die Antworten an die Vermittlungsschicht. Dort wird der Solver mit den Prämissen der Aufgabe instanziiert, die korrekte Lösung bestimmt und variablenweise mit den Nutzereingaben verglichen (vgl. Abschnitt 4.4). Anschließend aktualisiert die Vermittlungsschicht den Fortschritt des Nutzers nach der in Abschnitt 4.6.2 beschriebenen Logik und gibt das Gesamtergebnis an die Präsentationsschicht zurück. Für fehlerhaft beantwortete Variablen kann die Präsentationsschicht einzeln Feedback anfragen, wobei die Vermittlungsschicht die Feedbackkomponente der Kernlogik aufruft und den erzeugten Text weiterreicht (vgl. Abschnitt 4.5).



Gestrichelte Elemente: Feedbackabruf bei fehlerhafter Antwort

Abbildung 2: Datenfluss einer Aufgabenbearbeitung

4.7.3 Vorgenerierung und Ergebnis-Cache

Zwei konzeptionelle Entscheidungen betreffen die Laufzeit der Plattform.

Die Vorgenerierung wurde bereits in Abschnitt 4.2.5 eingeführt. Architektonisch bedeutet sie, dass pro Kombination aus Aufgabentyp und Level ein Vorrat an validierten Aufgaben im persistenten Speicher vorgehalten wird. Fordert ein Nutzer eine Aufgabe an, wird eine Aufgabe aus diesem Vorrat ausgewählt, anstatt den Generierungsprozess in Echtzeit durchzuführen. Die Antwortzeit wird dadurch unabhängig von der Generierungsdauer. Sollte der Vorrat für einen bestimmten Nutzer erschöpft sein oder keine initiale Vorgenerierung stattgefunden haben, wird eine Aufgabe unmittelbar generiert und gleichzeitig eine asynchrone Nachgenerierung weiterer Aufgaben angestoßen.

Da dieselbe Aufgabe von mehreren Nutzern bearbeitet werden kann, wird zusätzlich ein Ergebnis-Cache eingesetzt. Er speichert das Ergebnis eines Solver-Durchlaufs zu einer bestimmten Aufgabe und liefert es bei erneuten Anfragen ohne Neuberechnung aus. Bei den in der Plattform verwendeten Variablenmengen ist die Laufzeit des Solvers ohnehin gering; der Cache vermeidet jedoch wiederholte Berechnungen bei häufig bearbeiteten Aufgaben.

Die konkrete technische Umsetzung der beschriebenen Architektur — die verwendeten Frameworks, Datenmodelle und Schnittstellendefinitionen — wird im folgenden Kapitel beschrieben.

5 Implementierung

In Kapitel 4 wurden die konzeptionellen Entscheidungen beschrieben, die der Plattform zugrunde liegen. Das vorliegende Kapitel beschreibt, wie diese Konzepte technisch umgesetzt wurden. Die Darstellung folgt dabei der in Abschnitt 4.7 eingeführten Schichtenarchitektur: Nach der Begründung der eingesetzten Technologien (Abschnitt 5.1) werden zunächst die Kernlogik (Abschnitt 5.2), anschließend die Vermittlungsschicht (Abschnitt 5.3) und die Präsentationsschicht (Abschnitt 5.4) beschrieben. Abschnitt 5.5 erläutert die beiden Betriebsmodi der Anwendung, und Abschnitt 5.6 beschreibt das Testkonzept.

5.1 Technologieauswahl

Die Wahl der eingesetzten Technologien orientiert sich an der in Abschnitt 4.7.1 beschriebenen Schichtentrennung. Für jede Schicht wurden Technologien gewählt, die den jeweiligen Anforderungen entsprechen und im Rahmen eines Prototyps eine zügige Umsetzung ermöglichen.

Die Kernlogik ist in Python umgesetzt. Die Bibliothek SymPy dient zur symbolischen Darstellung und Manipulation der aussagenlogischen Formeln (SymPy Development Team 2025). Sie bietet eine Baumstruktur für logische Ausdrücke, die sich direkt für die rekursive Formelgenerierung (vgl. Abschnitt 4.2.1) und für die Auswertung von Belegungen eignet. Für die booleschen Tabellen des Deletion-Algorithmus (vgl. Abschnitt 2.3) wird NumPy (Harris u. a. 2020) eingesetzt.

Die Vermittlungsschicht verwendet das pythonbasierte Webframework Django zusammen mit dem Django REST Framework für die Bereitstellung der API-Endpunkte (Django Software Foundation 2025). Django wurde gewählt, weil es neben dem HTTP-Handling auch eine integrierte Benutzerverwaltung mit Authentifizierung, ein ORM für die Datenbankbindung und ein Migrationssystem für Schemaänderungen bereitstellt. Diese Funktionalitäten müssten bei leichtgewichtigeren Frameworks zusätzlich konfiguriert oder über Drittbibliotheken eingebunden werden. Als Datenbank wird SQLite verwendet. Für den vorliegenden Prototyp, der primär die Machbarkeit des Gesamtkonzepts demonstrieren soll und nicht unter realistischen Lastbedingungen mit vielen gleichzeitigen Nutzern betrieben wird, ist SQLite ausreichend. Für einen produktiven Einsatz im Lehrbetrieb wäre der Wechsel auf ein Datenbanksystem mit Unterstützung für parallele Schreibzugriffe (z. B. PostgreSQL) empfehlenswert. Die Datenbank erfordert keine separate Installation und wird als einzelne Datei im Projektverzeichnis gespeichert, was die Einrichtung und Weitergabe der Anwendung vereinfacht.

Die Präsentationsschicht ist als Single-Page-Application mit React und TypeScript umgesetzt. React wurde gewählt, weil die komponentenbasierte Architektur eine Wiederverwendung der Aufgabenbearbeitungskomponente für beide Nutzungsmodi — Lernpfad und Freies Üben — ermöglicht (vgl. Abschnitt 4.6) (Meta Platforms, Inc. 2025). TypeScript ergänzt React um statische Typisierung, wodurch Fehler in der Kommunikation

mit der API zur Entwicklungszeit erkannt werden können (Microsoft 2025). Für das Styling wird Tailwind CSS eingesetzt (Tailwind Labs Inc. 2026), und für Bedienelemente wie Radio-Buttons und Dialoge wird die Komponentenbibliothek Radix UI verwendet, die barrierefreie Standardkomponenten bereitstellt (Radix UI 2026).

Tabelle 2 fasst die eingesetzten Technologien zusammen.

Tabelle 2: Eingesetzte Technologien nach Schicht

Schicht	Technologie	Zweck
Kernlogik	Python 3.11	Programmiersprache
	SymPy	Symbolische Formeldarstellung
	NumPy	Boolesche Tabellen (Arrays)
Vermittlungsschicht	Django 5.2	Webframework, ORM, Authentifizierung
	Django REST Framework	REST-API
	SQLite	Datenbank
	Whitenoise	Statischer Dateiserver
Präsentationsschicht	Waitress	WSGI-Server (Produktionsserver)
	React 18	UI-Framework
	TypeScript	Statische Typisierung
	Tailwind CSS	Styling
	Radix UI	Barrierefreie Komponenten

5.2 Umsetzung der Kernlogik

Die Kernlogik umfasst drei Module, die den in Kapitel 4 beschriebenen Konzepten entsprechen: den Aufgabengenerator, den Solver und die Feedbackkomponente. Alle drei Module sind als reine Python-Klassen und -Funktionen umgesetzt und besitzen keine Abhängigkeit zu Django oder zur Datenbank.

5.2.1 Formeldarstellung und Aufgabengenerator

Die Darstellung der aussagenlogischen Formeln stützt sich auf die Klassen des SymPy-Moduls `sympy.logic.boolalg`. Jede Formel wird als Baum aus den Knotentypen `Symbol`, `Not`, `And`, `Or`, `Implies`, `Xor` und `Equivalent` aufgebaut. Die Aussagenvariablen einer Aufgabe werden über die SymPy-Funktion `symbols()` erzeugt und alphabetisch benannt (`A`, `B`, `C`, ...), wobei die Anzahl durch die Schwierigkeitskonfiguration des jeweiligen Levels festgelegt ist (vgl. Abschnitt 4.1).

Die rekursive Formelgenerierung erfolgt über die Funktion `random_formula()`. Sie erhält die verfügbaren Variablen, die maximale Verschachtelungstiefe, die zulässigen Operatoren und deren Gewichte als Parameter. Im Basisfall — wenn die maximale Tiefe erreicht

ist — gibt die Funktion eine zufällig gewählte Variable oder deren Negation zurück. Im rekursiven Fall wird ein Operator per gewichteter Zufallsauswahl bestimmt und auf rekursiv erzeugte Teilformeln angewendet.

Der in Abschnitt 4.2.1 beschriebene Generate-and-Filter-Ansatz ist in der Klasse `TaskGenerator` implementiert. Die Methode `generate_task()` erzeugt in einer Schleife zufällige Prämissenmengen und prüft jede gegen die aufgabentypspezifischen Validierungskriterien (vgl. Abschnitt 4.2.2f.). Die Prüfungen dieser Kriterien werden in die Funktionen `is_good_task_type_direct_inference` und `is_good_task_type_case_split` ausgelagert, die die jeweiligen Kriterien für die Aufgabentypen *Direct Inference* und *Case Split* implementieren.

Die Schleife durchläuft bis zu 50.000 Iterationen. Dieser Wert orientiert sich an der Kombination mit der niedrigsten Akzeptanzrate: Für Case Split Level 3 beträgt die mittlere Kandidatenanzahl rund 8.976 bei einer Akzeptanzrate von ca. 0,01,% (vgl. Abschnitt 6.1). Bei einer geometrisch verteilten Anzahl benötigter Versuche liegt die Wahrscheinlichkeit, innerhalb von 50.000 Iterationen mindestens eine gültige Aufgabe zu finden, bei über 99,%. Wird innerhalb dieser Grenze keine gültige Aufgabe gefunden, wirft die Methode eine Laufzeitausnahme.

Zwei Hilfsfunktionen unterstützen die Validierung. Die Funktion `all_models()` bestimmt die Modellmenge einer Prämissenmenge durch vollständige Enumeration aller 2^n Belegungen und wird für die Prüfung der Erfüllbarkeit, der Modellanzahl und der Redundanzfreiheit eingesetzt. Die Funktion `deductive_literal_closure()` simuliert die Unit Propagation (vgl. Abschnitt 2.1.5), indem sie die Prämissen zunächst in die konjunktive Normalform überführt und anschließend iterativ Einheitsklauseln propagiert. Sie wird verwendet, um zu prüfen, ob eine Prämissenmenge durch direktes Schließen lösbar ist (bei *Direct Inference*) oder ob keine Variable initial ableitbar ist (bei *Case Split*).

Die Prämissen einer erzeugten Aufgabe werden in zwei Darstellungsformen gespeichert. Eine lesbare Form mit Unicode-Symbolen (z. B. $A \rightarrow \neg B \vee C$) dient der Anzeige im Frontend. Sie wird durch die Funktion `print_logical_pretty()` erzeugt, die den SymPy-Baum rekursiv traversiert und die Operatoren durch ihre logischen Symbole ($\neg$, $\wedge$, $\vee$, $\rightarrow$, $\oplus$, $\leftrightarrow$) ersetzt, wobei bei zusammengesetzten Teilausdrücken Klammern eingefügt werden. Eine zweite Darstellung in SymPy-Notation (über die Funktion `srepr()`) dient der Rekonstruktion der Formel für den Solver. Beim Laden einer Aufgabe aus der Datenbank wird diese Darstellung über `sympify()` zurück in ein SymPy-Objekt überführt. Die doppelte Speicherung ist notwendig, weil die Unicode-Darstellung nicht verlustfrei in eine SymPy-Formel zurückgewandelt werden kann, die SymPy-Darstellung aber für den Nutzer nicht lesbar ist.

5.2.2 Solver

Der in Abschnitt 4.3.3 konzeptionell beschriebene Deletion-Algorithmus ist in zwei Klassen umgesetzt: `BooleanTable` für die Repräsentation und Manipulation der booleschen Tabellen und `BucketElimination` für den eigentlichen Lösungsprozess.

Ein `BooleanTable`-Objekt besteht aus einer geordneten Liste von Variablen und einem

mehrdimensionalen NumPy-Array vom Typ `bool`. Jede Dimension des Arrays entspricht einer Variable, jede Achse hat die Größe 2 (für die Werte falsch und wahr, indiziert als 0 und 1). Ein Eintrag des Arrays gibt an, ob die entsprechende Belegungskombination zulässig ist. Zusätzlich speichert jede Tabelle eine Menge von Quellindizes, die festhalten, aus welchen ursprünglichen Prämissen die Tabelle abgeleitet wurde.

Die in Abschnitt 2.3.2 beschriebenen Grundoperationen auf booleschen Tabellen sind als Methoden der Klasse `BooleanTable` umgesetzt:

Die Methode `extend_to()` erweitert eine Tabelle auf eine größere Variablenmenge (vgl. Abschnitt 2.3.2). Konzeptionell bedeutet dies, dass die in der Tabelle enthaltene Information auf zusätzliche Variablen ausgedehnt wird, ohne neue Einschränkungen einzuführen. Technisch werden dazu die vorhandenen Array-Achsen per `np.moveaxis()` in die Zielreihenfolge permutiert und für jede fehlende Variable eine neue Dimension der Größe 1 eingefügt (`np.expand_dims()`). NumPys Broadcasting-Mechanismus repliziert die bestehenden Einträge automatisch entlang der neuen Dimensionen, sodass der Tabellenwert für jede Belegung der ergänzten Variable unverändert bleibt.

Die statische Methode `combine()` implementiert die Kombination zweier Tabellen. Sie bildet die Vereinigungsmenge der Variablen beider Tabellen, erweitert beide Tabellen auf diesen gemeinsamen Raum und verknüpft die Einträge per `np.logical_and()`. Die Quellindizes beider Tabellen werden in einer Menge zusammengeführt.

Die Methode `marginalize()` entfernt eine Variable durch Projektion. Dazu wird `np.any()` entlang der entsprechenden Achse angewendet, was der logischen ODER-Verknüpfung über die beiden möglichen Werte der Variable entspricht.

Die Methode `condition()` schränkt eine Tabelle auf einen festen Variablenwert ein. Sie extrahiert per `np.take()` den Schnitt entlang der entsprechenden Achse und entfernt die Dimension der konditionierten Variable.

Die Klasse `BucketElimination` setzt den eigentlichen Deletion-Algorithmus um. Die Initialisierung überführt jede Prämisse der Aufgabe in eine boolesche Tabelle, indem alle 2^k Belegungen der in der Prämisse vorkommenden k Variablen aufgezählt und durch Einsetzen in die SymPy-Formel ausgewertet werden.

Der Forward Pass durchläuft die Variablen in alphabetischer Reihenfolge (vgl. Abschnitt 4.3.4). Für jede Variable wird ein Bucket gebildet, der alle Tabellen enthält, in denen die Variable vorkommt. Die Tabellen im Bucket werden per `combine()` verknüpft und die aktuelle Variable per `marginalize()` entfernt. Die resultierende Tabelle wird für die weitere Verarbeitung übernommen. Die originalen Bucket-Tabellen werden im Attribut `history_buckets` gespeichert, da sie für den Backward Pass und die Feedbackkomponente benötigt werden. Tabellen, die nach der Marginalisierung nur noch den Wert wahr enthalten, werden nicht weitergeführt, da sie keine Information mehr beitragen.

Der Backward Pass durchläuft die Variablen in umgekehrter Eliminationsreihenfolge. Für jede Variable werden die gespeicherten Bucket-Tabellen kombiniert und auf die bereits bestimmten Variablenwerte konditioniert. Anschließend wird geprüft, ob die Variable unter den verbleibenden Einschränkungen den Wert wahr, den Wert falsch oder beide Werte annehmen kann.

Ist unter den verbleibenden Einschränkungen nur der Wert 1 zulässig, wird die Variable als *wahr* festgehalten; ist nur 0 zulässig, als *falsch*. Sind beide Werte möglich, wird

die Variable als *nicht eindeutig bestimmbar* markiert. Diese dreistufige Unterscheidung entspricht dem in Abschnitt 4.4.1 eingeführten Antwortschema mit den Ausprägungen *wahr*, *falsch* und *kein konkreter Schluss möglich*.

5.2.3 Feedbackkomponente

Die Feedbackkomponente ist in der Klasse `FeedbackEngine` umgesetzt. Sie erhält bei der Instanziierung ein `BucketElimination`-Objekt und ruft dessen Methode `solve()` auf, falls der Solver noch nicht ausgeführt wurde.

Die Methode `generate_feedback()` bestimmt zunächst den Fehlertyp durch Vergleich der Nutzereingabe mit der korrekten Lösung und erzeugt dann die in Abschnitt 4.5.2 beschriebene fehlertypspezifische Rückmeldung. Für die Fehlertypen *Falsche Belegung* und *Verpasste Schlussfolgerung* beschränkt sich die Rückmeldung auf die Nennung des korrekten Werts, da die Berechnung in beiden Fällen nur einen Vergleich erfordert.

Für den Fehlertyp *Unnötige Festlegung* ist die Berechnung aufwendiger, da ein Gegenbeispiel konstruiert werden muss (vgl. Abschnitt 4.5.2). Die Methode `_find_consistent_scenario()` durchläuft dazu die im Solver gespeicherten Buckets in umgekehrter Eliminationsreihenfolge. Sie fixiert die betroffene Variable auf den entgegengesetzten Wert der Nutzereingabe und bestimmt anschließend für jede weitere Variable schrittweise einen konsistenten Wert. In jedem Schritt werden die Bucket-Tabellen der aktuellen Variable kombiniert, auf die bereits festgelegten Werte konditioniert und geprüft, welche Belegungen noch möglich sind. Das Ergebnis ist eine vollständige Belegung aller Variablen, die alle Prämissen erfüllt und zugleich der Annahme des Nutzers widerspricht. Diese Belegung wird dem Nutzer als konkreter Beweis dafür präsentiert, dass seine Festlegung nicht zwingend aus den Prämissen folgt.

5.3 Umsetzung der Vermittlungsschicht

Die Vermittlungsschicht ist als Django-Projekt mit mehreren Django-Apps umgesetzt. Sie verwaltet die Datenbankmodelle, stellt die API-Endpunkte bereit und koordiniert die Aufrufe der Kernlogik.

5.3.1 Anwendungsstruktur

Das Django-Projekt ist in drei aktive Apps unterteilt, die jeweils einen abgegrenzten Verantwortungsbereich abdecken:

- `generate_tasks` verwaltet die Aufgaben in der Datenbank, stellt den Endpunkt zur Aufgabenanforderung bereit und enthält den Vorgenerierungsdienst (vgl. Abschnitt 5.3.4).
- `solve_tasks` nimmt die Nutzereingaben entgegen, instanziiert den Solver der Kernlogik, führt die Bewertung durch, verwaltet den Lösungscache und erzeugt bei Bedarf das variablenspezifische Feedback. Die Feedbackerzeugung ist in dieser

App angesiedelt, da sie Zugriff auf die Solver-Ergebnisse benötigt. Da die Feedbackkomponente in der Kernlogik als eigenständige Klasse umgesetzt ist (vgl. Abschnitt 5.2.3), könnte sie bei einer späteren Erweiterung ohne Weiteres in eine eigene App ausgelagert werden.

- **users** verwaltet die Benutzerkonten, die Authentifizierung, den Fortschritt pro Aufgabentyp sowie die Avatar-Einstellungen.

Die Zuordnung der Apps zu den konzeptionellen Komponenten aus Kapitel 4 ist bewusst so gewählt, dass jede App einer fachlichen Verantwortlichkeit entspricht. Die Aufgaben-generierung, die Lösungsprüfung und die Benutzerverwaltung sind dadurch voneinander unabhängig und können separat getestet werden.

5.3.2 Datenmodell

Das Datenmodell bildet die in Abschnitt 4.7.1 beschriebenen vier Kategorien persistenter Daten auf fünf Datenbankmodelle ab. Abbildung 3 zeigt die Beziehungen zwischen den Modellen.

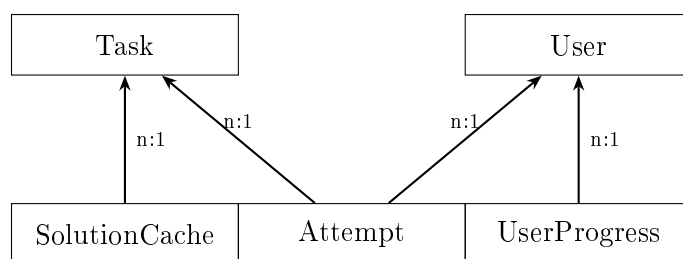


Abbildung 3: Beziehungen zwischen den Datenbankmodellen

Das Modell **Task** speichert eine vorgenerierte Aufgabe. Jeder Eintrag enthält den Aufgabentyp (*Direct Inference* oder *Case Split*), das Level, die Liste der Aussagenvariablen sowie die Prämissen in zwei Darstellungsformen: als Liste lesbarer Unicode-Zeichenketten für die Anzeige im Frontend und als Liste von SymPy-Repräsentationen für die Rekonstruktion der Formeln im Solver (vgl. Abschnitt 5.2.1).

Das Modell **User** erweitert das Standard-Benutzermodell von Django um anwendungsspezifische Felder. Dazu gehören vier Statistikfelder — die Gesamtzahl gelöster Aufgaben, die Anzahl korrekt gelöster Aufgaben, die aktuelle Erfolgsserie und der persönliche Highscore — sowie Felder für die Avatar-Konfiguration. Die Avatar-Felder bilden die Parameter des Dienstes DiceBear Avataars ab (vgl. Abschnitt 5.4.5), darunter Hautfarbe, Haarfarbe, Frisur, Augenform und Kleidungsstil. Die Speicherung erfolgt als einzelne Felder statt als JSON-Feld, da die Avatar-Parameter unabhängig voneinander geändert werden und beim Abruf der Nutzerdaten keine Deserialisierung erfordern sollen.

Das Modell **UserProgress** speichert den Fortschritt eines Nutzers für einen bestimmten Aufgabentyp im Lernpfad. Pro Nutzer und Aufgabentyp existiert genau ein Eintrag, der

das aktuelle Level, die Anzahl der aufeinanderfolgend korrekten Antworten im aktuellen Level und den Abschlussstatus enthält. Die Methode `record_answer()` setzt die in Abschnitt 4.6.2 beschriebene Aufstiegslogik um: Bei einer korrekten Antwort wird die Erfolgsserie erhöht; erreicht sie den für das aktuelle Level konfigurierten Schwellenwert, wird das Level hochgesetzt und die Serie zurückgesetzt. Bei einer fehlerhaften Antwort wird die Serie auf null zurückgesetzt, das erreichte Level bleibt erhalten. Die Schwellenwerte pro Level sind in einer separaten Konfigurationsstruktur hinterlegt, die von der Schwierigkeitskonfiguration der Kernlogik (vgl. Abschnitt 4.1) unabhängig ist, da sie eine didaktische und keine aufgabenbezogene Entscheidung abbilden.

Das Modell **Attempt** speichert einen einzelnen Lösungsversuch. Es verknüpft einen Nutzer mit einer Aufgabe und enthält die abgegebene Lösung als JSON-Feld sowie den Korrektheitsstatus. Die Zuordnung von Nutzer und Aufgabe dient dem Vorgenerierungsdienst zur Bestimmung, welche Aufgaben ein Nutzer bereits bearbeitet hat.

Das Modell **SolutionCache** speichert das Ergebnis eines Solver-Durchlaufs für eine bestimmte Aufgabe. Ein Eintrag enthält die Aufgaben-ID, den Namen des Solver-Verfahrens und das Ergebnis als JSON-Feld. Beim Prüfen einer Nutzereingabe wird zunächst geprüft, ob ein Cache-Eintrag für die betreffende Aufgabe existiert. Falls ja, wird das gespeicherte Ergebnis verwendet; andernfalls wird der Solver ausgeführt und das Ergebnis anschließend im Cache abgelegt.

5.3.3 API-Endpunkte

Die Vermittlungsschicht stellt ihre Funktionalität über REST-Endpunkte bereit, die von der Präsentationsschicht aufgerufen werden. Tabelle 3 gibt einen Überblick über die Endpunkte, gruppiert nach Verantwortungsbereich.

Tabelle 3: REST-Endpunkte der Vermittlungsschicht

Endpunkt	Methode	Beschreibung
<i>Aufgabenverwaltung</i>		
/api/tasks/generate/	POST	Liefert eine vorgenerierte, vom Nutzer noch nicht bearbeitete Aufgabe für den angegebenen Aufgabentyp und das Level
/api/tasks/pool_status/	GET	Gibt den aktuellen Füllstand des Aufgabenvorrats pro Kombination zurück
<i>Lösungsprüfung und Feedback</i>		
/api/solve/	POST	Nimmt die Antworten des Nutzers entgegen, prüft sie gegen die Solver-Lösung, aktualisiert den Fortschritt und gibt das Ergebnis zurück
/api/feedback/	POST	Erzeugt für eine fehlerhaft beantwortete Variable den Feedbacktext
/api/solution/{id}/	GET	Gibt die vollständige Solver-Lösung einer Aufgabe zurück
<i>Benutzerverwaltung</i>		
/api/auth/register/	POST	Registriert einen neuen Nutzer und erstellt die zugehörigen UserProgress-Einträge
/api/auth/login/	POST	Authentifiziert den Nutzer und erstellt eine Sitzung
/api/auth/logout/	POST	Beendet die aktuelle Sitzung
/api/auth/me/	GET	Gibt die Daten des eingeloggten Nutzers zurück (Profil, Fortschritt, Statistiken, Avatar)
/api/auth/password-change/	POST	Ändert das Passwort des eingeloggten Nutzers
/api/auth/reset-progress/	POST	Setzt den gesamten Fortschritt und alle Statistiken zurück
/api/auth/avatar/	POST	Aktualisiert die Avatar-Einstellungen
/api/auth/avatar/random/	POST	Erzeugt eine zufällige Avatar-Konfiguration

Zusätzlich zu den in Tabelle 3 aufgeführten Endpunkten stellt das Django REST Framework über die registrierten ViewSets automatisch Standard-Endpunkte für das Auflisten, Abrufen und Löschen von Aufgaben sowie das Auflisten von Nutzern bereit. Diese Endpunkte werden von der Präsentationsschicht im regulären Betrieb nicht verwendet und dienen ausschließlich administrativen Zwecken und der Fehlersuche während der Entwicklung.

Der Endpunkt `/api/tasks/generate/` liefert eine Aufgabe für einen bestimmten Aufgabentyp und ein bestimmtes Level. Ist der Nutzer eingeloggt, wählt der Endpunkt aus dem Vorrat eine zufällige Aufgabe aus, die dieser Nutzer noch nicht bearbeitet hat. Dazu werden alle Aufgaben der gewünschten Kombination ermittelt und diejenigen ausgeschlossen, für die bereits ein Attempt des anfragenden Nutzers existiert. Ist der Nutzer nicht authentifiziert, wird eine beliebige Aufgabe aus dem Vorrat zurückgegeben. Sollte für einen eingeloggten Nutzer keine unbearbeitete Aufgabe mehr verfügbar sein, wird eine Aufgabe unmittelbar über den Generator der Kernlogik erzeugt und gleichzeitig die asynchrone Nachgenerierung angestoßen (vgl. Abschnitt 5.3.4).

Der Endpunkt `/api/solve/` setzt den in Abschnitt 4.7.2 beschriebenen Datenfluss um. Er lädt die Aufgabe aus der Datenbank, konvertiert die gespeicherten SymPy-Repräsentationen zurück in Formelobjekte, instanziiert den Solver, bestimmt die korrekte Lösung (oder liest sie aus dem Cache), vergleicht jede Variable mit der Nutzereingabe, aktualisiert die globalen Statistiken des Nutzers und den aufgabentypspezifischen Fortschritt und speichert den Versuch als Attempt. Die Antwort enthält das Ergebnis pro Variable, die aktualisierte Fortschrittsinformation und die Nutzerstatistiken, sodass die Präsentationsschicht alle benötigten Informationen in einer einzigen Anfrage erhält.

Der Endpunkt `/api/feedback/` ist von `/api/solve/` getrennt, da Feedback nicht automatisch für alle Variablen erzeugt wird, sondern nur auf Anfrage für einzelne fehlerhaft beantwortete Variablen. Die Präsentationsschicht ruft diesen Endpunkt erst auf, wenn der Nutzer das Feedback für eine bestimmte Variable anfordert. Bei jedem Aufruf wird der Solver neu instanziiert, da die Feedbackkomponente Zugriff auf die gespeicherten Bucket-Zwischenergebnisse benötigt.

Die Authentifizierung basiert auf Djangos Session-Mechanismus. Bei der Registrierung wird der Nutzer automatisch eingeloggt, und es werden für beide Aufgabentypen leere `UserProgress`-Einträge angelegt. Endpunkte, die nutzerspezifische Daten verarbeiten — Lösungsprüfung, Feedback, Fortschritt und Avatar — erfordern eine authentifizierte Sitzung. Die Aufgabenanforderung funktioniert auch ohne Authentifizierung, liefert dann aber eine zufällige Aufgabe ohne Berücksichtigung bereits bearbeiteter Aufgaben.

5.3.4 Vorgenerierungsdienst

Die in den Abschnitten 4.2.5 und 4.7.3 konzeptionell beschriebene Vorgenerierung ist als Django-Management-Befehl `prefill_tasks` umgesetzt. Er befüllt den Aufgabenvorrat auf einen konfigurierbaren Zielwert von 200 Aufgaben pro Kombination aus Aufgabentyp und Level. Der Wert ist als Standardeinstellung zu verstehen und kann bei Bedarf über einen Befehlsparameter angepasst werden. Dieser Zielwert dient primär der Sicherstellung einer ausreichenden Aufgabenvielfalt bei Inbetriebnahme.

Die Generierung erfolgt standardmäßig parallel über mehrere Prozesse, da insbesondere die Erzeugung von *Case-Split*-Aufgaben rechenintensiv ist. Für jede Kombination aus Aufgabentyp und Level wird ein eigener Prozess gestartet, der den Aufgabengenerator der Kernlogik aufruft und die erzeugten Aufgaben einzeln in der Datenbank speichert. Die Anzahl der parallelen Prozesse entspricht standardmäßig der Anzahl der verfügbaren CPU-Kerne und kann über einen Befehlsparameter eingeschränkt werden. Ein Fortschrittsbalken pro Kombination zeigt den aktuellen Generierungsstand an.

Da die parallelen Prozesse auf separaten Python-Instanzen laufen, muss jeder Prozess eine eigene Django-Umgebung initialisieren, bevor er auf die Datenbank zugreifen kann. Die Worker-Funktion ist deshalb in ein separates Modul ausgelagert, das die Django-Initialisierung innerhalb der Funktion durchführt, nicht auf Modulebene. Fortschrittsmeldungen werden über eine prozessübergreifende Warteschlange an den Hauptprozess übermittelt.

Neben der initialen Befüllung unterstützt der Dienst auch die Nachgenerierung im laufenden Betrieb. Wenn ein eingeloggter Nutzer alle verfügbaren Aufgaben eines Levels bearbeitet hat und der Endpunkt `/api/tasks/generate/` keine unbearbeitete Aufgabe aus dem Vorrat liefern kann, wird eine Aufgabe unmittelbar generiert und gleichzeitig ein Stapel von 20 weiteren Aufgaben in einem Hintergrund-Thread erzeugt. Der Hintergrund-Thread ist als Daemon-Thread konfiguriert, sodass er beim Beenden des Servers automatisch abgebrochen wird, und verwendet eine Sperre, um konkurrierende Nachgenerierungen für dieselbe Kombination zu verhindern.

5.4 Umsetzung der Präsentationsschicht

Die Präsentationsschicht wurde als Single-Page-Application mit React und TypeScript umgesetzt. Vor der Implementierung wurde ein vollständiges Oberflächenkonzept in Form von Figma-Mockups erstellt, das die Seitenstruktur, die Navigationsflüsse und das visuelle Erscheinungsbild der Anwendung festlegte. Die Mockups dienten als verbindliche Vorlage für die Umsetzung und ermöglichten es, gestalterische Entscheidungen von der technischen Implementierung zu trennen. Der daraus entstandene Komponentencode konnte anschließend direkt in das Projekt übernommen werden.

5.4.1 Seitenstruktur und Routing

Die Anwendung verwendet React Router für die clientseitige Navigation. Die Seitenstruktur gliedert sich in öffentliche Seiten, die ohne Anmeldung zugänglich sind, und geschützte Seiten, die eine authentifizierte Sitzung erfordern.

Zu den öffentlichen Seiten gehören die Startseite mit der Übersicht der vier Hauptbereiche (Lernpfad, Freies Üben, Grundlagen, Referenzen), die Login- und Registrierungsseite sowie die Datenschutzerklärung.

Die geschützten Seiten umfassen den Lernpfad, das Freie Üben sowie die Account-Seite. Der Zugriffsschutz ist über eine `ProtectedRoute`-Komponente realisiert, die den Authen-

tifizierungsstatus aus einem zentral verwalteten **AuthContext** ausliest. Nicht eingeloggte Nutzer werden automatisch zur Login-Seite weitergeleitet.

Der **AuthContext** verwaltet den Benutzerzustand über die gesamte Anwendung hinweg. Er prüft beim Laden der Anwendung über den Endpunkt `/api/auth/me/`, ob eine gültige Sitzung besteht, und stellt allen Komponenten den aktuellen Nutzer mit Fortschritt, Statistiken und Avatar-Daten zur Verfügung. Nach jeder Lösungsprüfung wird der Kontext aktualisiert, sodass Fortschrittsänderungen sofort in der gesamten Oberfläche sichtbar werden.

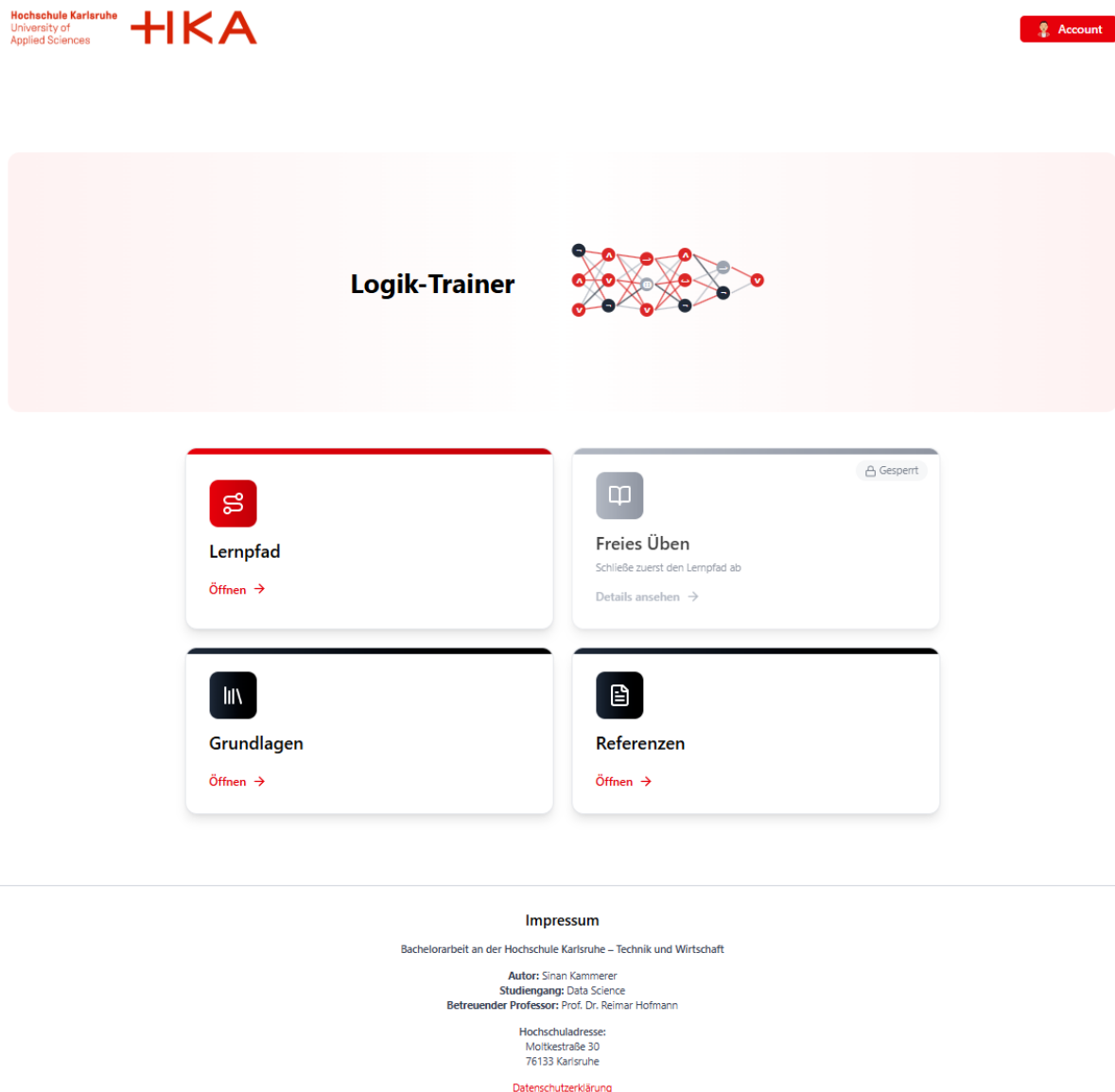


Abbildung 4: Startseite der Anwendung. Die vier Hauptbereiche sind als Auswahlkarten dargestellt. Der Bereich „Freies Üben“ ist gesperrt, bis der Lernpfad abgeschlossen wurde.

5.4.2 Aufgabenbearbeitung

Die Aufgabenbearbeitung ist für beide Aufgabentypen in einer identischen Oberflächenstruktur umgesetzt. Eine Aufgabenseite besteht aus drei Bereichen: der Anzeige der Prämissen, dem Eingabebereich mit einer Gruppe von Radio-Buttons pro Variable und dem Ergebnisbereich nach der Prüfung der Nutzereingabe.

Die Prämissen werden als nummerierte Liste in der lesbaren Unicode-Darstellung angezeigt. Für jede Variable der Aufgabe wird eine Zeile mit drei Radio-Buttons dargestellt, die den drei Antwortoptionen *wahr*, *falsch* und *kein konkreter Schluss möglich* entsprechen (vgl. Abschnitt 4.4). Die Abgabe ist erst möglich, wenn für alle Variablen eine Auswahl getroffen wurde.

Nach der Abgabe werden die Radio-Buttons deaktiviert und jede Variable mit einem Häkchen oder einem Kreuz als korrekt oder fehlerhaft markiert. Für fehlerhaft beantwortete Variablen erscheint die korrekte Antwort sowie ein Info-Symbol, über das der Nutzer das Feedback der Feedbackkomponente abrufen kann. Das Feedback wird erst bei Klick auf das Symbol geladen, nicht automatisch für alle Variablen (vgl. Abschnitt 5.3.3).

Gegeben:

(P1) $\neg A \leftrightarrow \neg B$

(P2) $\neg B$

Welche Schlüsse können gezogen werden?

Variable A:  ☒ Wahr ☐ Falsch ☐ Kein konkreter Schluss möglich 

 Die Regeln erzwingen, dass A Falsch ist (und nicht Wahr).

Korrekte Antwort: Falsch

Variable B:  ☐ Wahr ☒ Falsch ☐ Kein konkreter Schluss möglich

 Nicht ganz richtig. Klicke auf das Info-Symbol für detailliertes Feedback.

Nächste Aufgabe

Abbildung 5: Aufgabenansicht nach der Prüfung. Korrekte Variable ist mit einem Häkchen markiert, fehlerhafte mit einem Kreuz. Für die Variable A wurde das Feedback eingeblendet.

Die Aufgabenkomponente wird sowohl im Lernpfad als auch im Freien Üben wiederverwendet. Die beiden Modi unterscheiden sich in der Art, wie das Level bestimmt wird: Im Lernpfad ergibt sich das Level aus dem aktuellen Fortschritt des Nutzers, im Freien Üben wird es vom Nutzer selbst gewählt oder per Zufallsauswahl bestimmt. Die Unterscheidung erfolgt über URL-Parameter (`mode=free, level=...`), die die Aufgabenkomponente beim Laden auswertet.

5.4.3 Lernpfad-Darstellung

Der in Abschnitt 4.6 beschriebene Lernpfad wird als visuelle Checkpoint-Darstellung umgesetzt. Für jeden Aufgabentyp werden die Level als kreisförmige Checkpoints angezeigt, die über Verbindungslinien miteinander verbunden sind. Ein Checkpoint kann drei Zustände annehmen: aktiv (hüpfende Animation, anklickbar), abgeschlossen (grünes Häkchen) oder gesperrt (Schloss-Symbol, nicht anklickbar). Am Ende jeder Checkpoint-Reihe steht ein Zielsymbol, das den Abschluss des Aufgabentyps markiert. Unterhalb der Checkpoints zeigt eine Fortschrittsanzeige das aktuelle Level, die laufende Erfolgsserie und den Schwellenwert für den nächsten Levelaufstieg an.

Die Freischaltlogik zwischen den Aufgabentypen (vgl. Abschnitt 4.6.1) wird direkt in der Darstellung widerspiegelt: Die Checkpoint-Reihe für den Aufgabentyp Case Split ist ausgegraut und als gesperrt gekennzeichnet, solange nicht alle Level des Typs Direct Inference abgeschlossen sind.

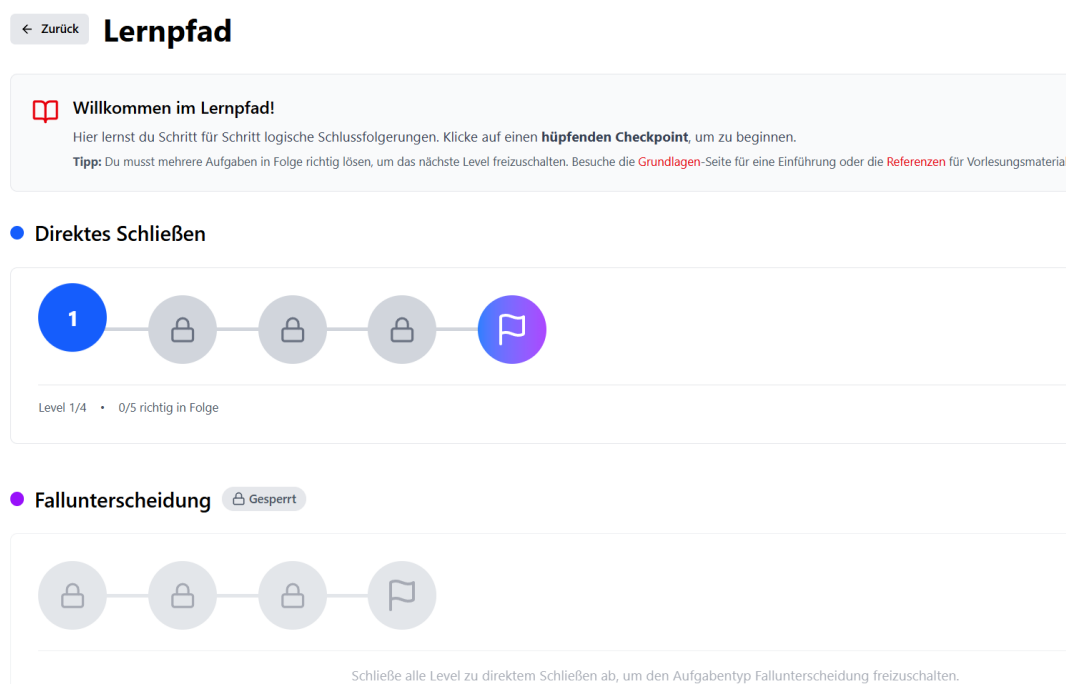


Abbildung 6: Lernpfad-Ansicht direkt nach der Registrierung eines neuen Nutzers.

Bei Abschluss eines Aufgabentyps oder des gesamten Lernpfads wird ein modales Dialogfenster angezeigt, das den Erfolg würdigt und nach Abschluss des gesamten Lernpfads auf die Freischaltung des Freien Übens hinweist.

5.4.4 Freies Üben

Der Modus Freies Üben wird freigeschaltet, sobald alle Level beider Aufgabentypen im Lernpfad abgeschlossen sind (vgl. Abschnitt 4.6.1). Die Seite zeigt zwei Auswahlkarten — eine pro Aufgabentyp — mit einer Levelauswahl und einer Option für ein zufällig gewähltes Level. Nach der Auswahl wird der Nutzer zur Aufgabenbearbeitung weitergeleitet, wobei der gewählte Aufgabentyp und das Level als URL-Parameter übergeben werden. Die Aufgabenkomponente selbst ist dieselbe wie im Lernpfad (vgl. Abschnitt 5.4.2); der Unterschied liegt darin, dass im Freien Üben kein Fortschritt in Form von Levelaufstiegen verfolgt wird und der Nutzer das Level frei wechseln kann.

Freies Üben

Wähle einen Aufgabentyp und ein Level aus, um gezielt zu üben.

The image shows two side-by-side cards for selecting a task type and level. The left card is titled 'Direktes Schließen' and features the logical formula $A \rightarrow B$. Below the title, it says 'Ziehe direkte Schlussfolgerungen aus einzelnen Fakten und vereinfache die Formel.' It has a 'Level auswählen:' section with a 'Zufällig' button (highlighted with a red border) and buttons for 'Level 1', 'Level 2', 'Level 3', and 'Level 4'. A red 'Starten' button is at the bottom. The right card is titled 'Fallunterscheidung' and features a decision tree diagram with 'True' and 'False' branches. Below the title, it says 'Löse komplexe Probleme, indem du verschiedene Fälle systematisch untersuchst.' It has a 'Level auswählen:' section with a 'Zufällig' button (highlighted with a red border) and buttons for 'Level 1', 'Level 2', and 'Level 3'. A red 'Starten' button is at the bottom.

Abbildung 7: Freies Üben-Ansicht.

Für Nutzer, die den Lernpfad noch nicht abgeschlossen haben, zeigt die Seite statt der Auswahlkarten eine Sperrungsansicht an, die den aktuellen Fortschritt zusammenfasst und einen Link zum Lernpfad bereitstellt.

5.4.5 Begleitende Seiten

Neben den Übungsfunktionen stellt die Präsentationsschicht drei begleitende Seiten bereit.

Die Grundlagenseite fasst die für die Aufgabenbearbeitung relevanten Konzepte der Aussagenlogik zusammen. Sie enthält die Wahrheitstabelle aller in der Plattform verwendeten logischen Operatoren, eine Übersicht der wichtigsten Umformungsregeln (De Morgan, Kontraposition, Implikationsauflösung und weitere) sowie je ein durchgerechnetes Beispiel für beide Aufgabentypen. Die Beispiele sind mit Abbildungen veranschaulicht und zeigen den Lösungsweg schrittweise. Die Seite ist ohne Anmeldung zugänglich und unabhängig vom Lernfortschritt nutzbar, sodass Studierende jederzeit die fachlichen Grundlagen nachschlagen können (vgl. Abschnitt 4.6.3).

Die Referenzenseite verweist auf die externen zugrunde liegenden Lehrmaterialien, auf denen die Aufgabentypen der Plattform basieren.

Die Account-Seite zeigt dem eingeloggteten Nutzer sein Profil, den Lernfortschritt pro Aufgabentyp mit Fortschrittsbalken, die Statistiken (gelöste Aufgaben, korrekt gelöste Aufgaben, aktuelle Erfolgsserie, Highscore) sowie Funktionen zur Passwortverwaltung und zum Zurücksetzen des Fortschritts. Ergänzend bietet die Anwendung einen Avatar-Editor, über den der Nutzer eine personalisierte Profildarstellung konfigurieren kann (vgl. Abschnitt 4.6.3). Zur Generierung der Avatare wird der Open-Source-Dienst DiceBear verwendet; eingesetzt wird dabei der Stil *Avataaars* (DiceBear 2026).

5.5 Betriebsmodi und Einrichtung

Die Anwendung kann in zwei Modi betrieben werden, die sich in der Art unterscheiden, wie Präsentationsschicht und Vermittlungsschicht zusammenarbeiten.

Im Entwicklungsmodus laufen zwei separate Server: ein Vite-Entwicklungsserver für die Präsentationsschicht und der Django-Entwicklungsserver für die Vermittlungsschicht. Der Vite-Server leitet API-Anfragen über eine Proxy-Konfiguration an den Django-Server weiter, sodass beide Server aus Sicht des Browsers unter demselben Ursprung erreichbar sind. Dieser Modus bietet automatisches Neuladen bei Codeänderungen im Frontend und eignet sich für die Entwicklung und Fehlersuche. Im Produktionsmodus wird die Präsentationsschicht vorab in statische Dateien übersetzt. Die Auslieferung erfolgt über WhiteNoise und Waitress (vgl. Tabelle 2), sodass die gesamte Anwendung über einen einzigen Server und Port erreichbar ist. Die gesamte Anwendung ist dadurch über einen einzigen Server und Port erreichbar. Für beide Modi stehen plattformspezifische Startskripte bereit (PowerShell für Windows, Bash für macOS und Linux), die die notwendigen Schritte — Aktivierung der virtuellen Umgebung, ggf. Frontend-Build, Start der Server — automatisieren. Ein separates Setup-Skript installiert die Python-Version über den Paketmanager uv, erstellt die virtuelle Umgebung, installiert die Abhängigkeiten und führt die Datenbankmigrationen durch.

5.6 Tests

Die Plattform wird auf drei Ebenen getestet: durch Unit-Tests der Kernlogik, durch API-Tests der Vermittlungsschicht und durch einen manuellen Integrationstest, der den gesamten Verarbeitungsablauf abdeckt.

Die Unit-Tests der Kernlogik prüfen die algorithmische Korrektheit des Solvers. Da der Bucket-Elimination-Solver ein eigenständig implementiertes Verfahren ist, wird sein Ergebnis gegen eine unabhängige Referenz geprüft: die Funktion `satisfiable()` der Bibliothek SymPy, die alle erfüllenden Belegungen einer Formel bestimmen kann. Jeder Testfall definiert eine Prämissenmenge, löst sie mit dem Solver und vergleicht das Ergebnis in drei Schritten. Zunächst wird geprüft, ob Solver und SymPy in der Frage der Erfüllbarkeit übereinstimmen. Anschließend wird die vom Solver bestimmte konkrete Belegung in die Originalformel eingesetzt und über SymPy verifiziert. Zuletzt wird für jede Variable, die der Solver als nicht eindeutig bestimmbar markiert, geprüft, ob tatsächlich sowohl der Wert wahr als auch der Wert falsch mit den Prämissen vereinbar sind. Die Testfälle decken verschiedene Szenarien ab, darunter einfache Implikationsketten, widersprüchliche Prämissenmengen, Tautologien und komplexe Fallunterscheidungsaufgaben mit fünf Variablen.

Die API-Tests prüfen die Endpunkte der Vermittlungsschicht über Djangos Testframework. Für jede der drei Apps (Aufgabenverwaltung, Lösungsprüfung, Benutzerverwaltung) existiert eine Testklasse, die den jeweiligen Endpunkt mit gültigen und ungültigen Eingaben aufruft und die erwartete Antwort prüft. Die Tests der Aufgabenverwaltung prüfen die Generierung für beide Aufgabentypen, die Validierung ungültiger Parameter und die Nutzung des Aufgabenvorrats. Die Tests der Lösungsprüfung prüfen die Erkennung korrekter und fehlerhafter Antworten, die Aktualisierung der Nutzerstatistiken, die Rückgabe pro Variable und die Feedbackerzeugung. Die Tests der Benutzerverwaltung decken Registrierung, Login, Logout, Passwortänderung, Fortschrittsreset und Avatar-Aktualisierung ab. Alle API-Tests verwenden eine temporäre Testdatenbank, die nach jedem Testlauf verworfen wird.

Der manuelle Integrationstest verbindet alle Komponenten der Kernlogik in einem durchgehenden Ablauf: Er erzeugt über den Generator eine zufällige Aufgabe, löst sie über den Solver und simuliert anschließend eine interaktive Nutzereingabe über die Kommandozeile. Für jede eingegebene Antwort erzeugt die Feedbackkomponente eine Rückmeldung, die direkt ausgegeben wird. Dieser Test dient nicht der automatisierten Regressionsprüfung, sondern der manuellen Überprüfung, dass Generator, Solver und Feedbackkomponente im Zusammenspiel plausible Ergebnisse liefern und die erzeugten Rückmeldungen inhaltlich nachvollziehbar sind.

6 Evaluation

Das vorliegende Kapitel evaluiert die in den Kapiteln 4 und 5 beschriebenen Kernkomponenten der Plattform anhand dreier Perspektiven. Abschnitt 6.1 untersucht die Aufgabengenerierung hinsichtlich Akzeptanzraten und Laufzeiten. Abschnitt 6.2 prüft die Korrektheit des Solvers durch eine systematische Verifikation gegen eine unabhängige Referenzimplementierung. Abschnitt 6.3 gleicht die umgesetzte Funktionalität mit der in Abschnitt 1.3 formulierten Zielsetzung ab.

Die Messungen wurden auf einem handelsüblichen Entwicklungsrechner mit einem AMD Ryzen 5 4600H mit 3,00 GHz und 16 GB RAM durchgeführt. Das System basiert auf einer Windows 11 64-Bit-Architektur. Die konkreten Hardwarespezifikationen sind für die relative Einordnung der Ergebnisse relevant, jedoch nicht für deren qualitative Aussage. Die Messungen dienen der Charakterisierung des Generierungsverhaltens und der Begründung der Vorgenerierungsstrategie, nicht dem Nachweis absoluter Leistungskennzahlen.

6.1 Aufgabengenerierung

Für die Evaluation der in Abschnitt 4.2 beschriebenen Aufgabengenerierung wurde pro Kombination aus Aufgabentyp und Level Aufgaben erzeugt, wobei für jede einzelne Aufgabe drei Kenngrößen erfasst wurden: die Anzahl der erzeugten und verworfenen Kandidaten, die daraus resultierende Akzeptanz, die Dauer des gesamten Generierungsvorgangs sowie die Dauer der anschließenden Lösungsbestimmung durch den Solver. Die Stichprobengrößen betragen 100 Aufgaben pro Level für den Aufgabentyp Direct Inference und 50, 30 beziehungsweise 20 Aufgaben für die Level 1 bis 3 des Aufgabentyps Case Split.

Die reduzierten Stichprobengrößen für Case Split sind durch die deutlich höhere Generierungsdauer pro Aufgabe begründet, die eine Erzeugung gleich großer Stichproben in vertretbarer Zeit nicht ermöglicht. Angesichts der hohen Streuung (vgl. Tabelle 4) liefern die Stichproben belastbare Schätzungen der Mittelwerte und Größenordnungen, die relativen Unterschiede zwischen den Levels sind jedoch mit Vorsicht zu interpretieren. Tabelle 4 fasst die Ergebnisse zusammen. Die einzelnen Aspekte werden in den folgenden Unterabschnitten erläutert.

Tabelle 4: Ergebnisse der Aufgabengenerierung nach Aufgabentyp und Level

Aufgabentyp	Level	n	$\varnothing$ Kand.	σ Kand.	Akz.-Rate (%)	$\varnothing$ Gen.-Dauer (s)	$\varnothing$ Solve-Dauer (ms)
Direct Inference	1	100	5,2	4,0	19,42	0,003	0,16
	2	100	35,8	33,9	2,80	0,038	0,20
	3	100	146,0	135,2	0,69	0,190	0,29
	4	100	53,1	50,4	1,88	0,180	0,31
Case Split	1	50	640,7	502,9	0,16	0,77	0,33
	2	30	2 777,6	2 586,3	0,04	9,59	0,46
	3	20	8 975,6	8 043,7	0,01	58,93	0,63

Akzeptanzraten

Die Akzeptanzrate gibt an, welcher Anteil der zufällig erzeugten Kandidaten die Validierungskriterien erfüllt. Sie errechnet sich als Kehrwert der mittleren Kandidatenanzahl und ist ein direktes Maß für die Selektivität der Kriterien.

Für den Aufgabentyp Direct Inference zeigt sich ein differenziertes Bild. In Level 1 liegt die Akzeptanzrate bei rund 19 %, das heißt etwa jeder fünfte Kandidat wird akzeptiert. Dies ist plausibel, da die Aufgaben lediglich zwei Variablen und zwei Prämissen mit maximaler Tiefe 1 umfassen, sodass ein vergleichsweise großer Anteil der zufälligen Prämissenmengen die Kriterien erfüllt. Mit steigendem Level sinkt die Akzeptanzrate zunächst deutlich: In Level 2 werden durchschnittlich 36 Kandidaten benötigt (2,8 %), in Level 3 bereits 146 Kandidaten (0,69 %). Auffällig ist, dass Level 4 mit durchschnittlich 53 Kandidaten eine höhere Akzeptanzrate aufweist als Level 3, obwohl die maximale Verschachtelungstiefe von 2 auf 3 ansteigt (vgl. Tabelle 1). Eine mögliche Erklärung liegt darin, dass die tieferen Formeln in Level 4 strukturell reichhaltigere Ausdrücke erzeugen, die häufiger zu ableitbaren Variablenbelegungen führen, während in Level 3 die Kombination aus vier Variablen und maximaler Tiefe 2 (vgl. Tabelle 1) besonders oft Prämissenmengen erzeugt, in denen sich nicht genügend Variablen durch Unit Propagation bestimmen lassen.

Für den Aufgabentyp Case Split liegen die Akzeptanzraten erwartungsgemäß deutlich niedriger. In Level 1 beträgt die mittlere Kandidatenanzahl bereits 641 (0,16 %), in Level 2 steigt sie auf rund 2 778 (0,04 %) und in Level 3 auf rund 8 976 (0,01 %). Die niedrigen Akzeptanzraten spiegeln die strengeren Validierungskriterien wider (vgl. Abschnitt 4.2.3): Insbesondere die Forderung, dass keine Variable initial durch Unit Propagation ableitbar sein darf und gleichzeitig mindestens zwei Variablen als sinnvolle Fallunterscheidungsvariablen dienen müssen, schließt den überwiegenden Teil der zufällig erzeugten Prämissenmengen aus. Hinzu kommt, dass die Prüfung selbst aufwendiger ist, da sie für jede Variable eine Fallunterscheidung simuliert, was die Validierungsdauer pro Kandidat erhöht.

Generierungsdauer und Lösedauer

Die Generierungsdauer umfasst die gesamte Zeit vom Beginn der Kandidatenerzeugung bis zur Akzeptanz einer gültigen Aufgabe, einschließlich aller verworfenen Kandidaten und deren Validierungsprüfungen. Die Lösedauer bezeichnet die anschließende Ausführung des Deletion-Algorithmus auf der akzeptierten Aufgabe.

Für den Aufgabentyp Direct Inference liegt die mittlere Generierungsdauer zwischen 3 ms (Level 1) und 190 ms (Level 3). Selbst im ungünstigsten Fall bewegt sich die Generierung im Bereich unter einer Sekunde und wäre damit prinzipiell auch in Echtzeit realisierbar. Für den Aufgabentyp Case Split ergibt sich ein grundlegend anderes Bild. Bereits in Level 1 beträgt die mittlere Generierungsdauer rund 0,8 Sekunden. In Level 2 steigt sie auf durchschnittlich 9,6 Sekunden, in Level 3 auf durchschnittlich 58,9 Sekunden — mit Einzelwerten, die deutlich darüber liegen können, da die Standardabweichung in Level 3 mit rund 8 000 Kandidaten eine erhebliche Streuung anzeigt. Die Gesamtdauer

für die Erzeugung aller 20 Aufgaben in Case Split Level 3 betrug knapp 20 Minuten. Diese Werte belegen empirisch die Notwendigkeit der in Abschnitt 4.2.5 beschriebenen Vorgenerierungsstrategie: Eine Echtzeitgenerierung bei jeder Nutzeranfrage ist für Case-Split-Aufgaben der höheren Level nicht praktikabel.

Die Lösedauer des Solvers ist demgegenüber vernachlässigbar. Sie liegt für alle Kombinationen im Bereich von 0,16 bis 0,63 Millisekunden und steigt moderat mit der Anzahl der Variablen an. Der in Abschnitt 4.7.3 beschriebene Ergebnis-Cache ist damit primär eine konzeptionelle Maßnahme zur Vermeidung wiederholter Berechnungen; ein spürbarer Laufzeitvorteil für die Nutzererfahrung ergibt sich aus ihm bei den vorliegenden Variablenmengen nicht.

Neben den Mittelwerten ist die Streuung der Kandidatenanzahl aufschlussreich. Wie Tabelle 4 zeigt, liegt die Standardabweichung bei allen Kombinationen in derselben Größenordnung wie der Mittelwert. Dieses Verhältnis ist eine erwartbare Eigenschaft des Generate-and-Filter-Ansatzes:

Da jeder zufällig erzeugte Kandidat unabhängig von den vorherigen geprüft wird und die Akzeptanzwahrscheinlichkeit p pro Versuch konstant bleibt, folgt die Anzahl der benötigten Versuche bis zum ersten Erfolg einer geometrischen Verteilung mit Erwartungswert $\mu = 1/p$ und Standardabweichung $\sigma = \sqrt{(1-p)/p^2}$ (Li 2019). Für kleine p gilt $\sigma \approx \mu$, was die in Tabelle 4 beobachtete Streuung in derselben Größenordnung wie der Mittelwert erklärt.

Für die Nutzererfahrung bedeutet diese hohe Variabilität, dass die Generierungsdauer bei einzelnen Anfragen erheblich vom Durchschnitt abweichen kann. So ist es bei Case Split Level 3 möglich, dass eine Aufgabe bereits nach wenigen hundert Kandidaten gefunden wird, während eine andere über 25 000 Kandidaten erfordert. Diese Unvorhersehbarkeit bekräftigt die Notwendigkeit der Vorgenerierungsstrategie (vgl. Abschnitt 4.2.5) zusätzlich zu den absoluten Generierungsdauern: Selbst wenn die mittlere Dauer akzeptabel wäre, würden die Ausreißer im oberen Bereich zu spürbaren Wartezeiten führen.

6.2 Korrektheit des Solvers

Die Korrektheit des Bucket-Elimination-Solvers wurde durch eine systematische Verifikation gegen die Funktion `satisfiable()` der Bibliothek SymPy (SymPy Development Team 2025) als unabhängige Referenz geprüft. Das Verifikationsverfahren ist in Abschnitt 5.6 konzeptionell beschrieben; in diesem Abschnitt werden die Ergebnisse dargestellt.

Jede der im vorherigen Abschnitt generierten Aufgaben wurde dem in Abschnitt 5.6 beschriebenen dreistufigen Verifikationsverfahren unterzogen: Prüfung der Erfüllbarkeitsentscheidung (A), Verifikation der konkreten Belegungen (B) und Prüfung der Freiheitsgrade (C). Tabelle 5 zeigt die Ergebnisse.

Tabelle 5: Ergebnisse der Solver-Verifikation gegen SymPy

Aufgabentyp	Level	Getestet	Bestanden	Fehlgeschlagen	Fehlerrate (%)
Direct Inference	1	100	100	0	0,0
	2	100	100	0	0,0
	3	100	100	0	0,0
	4	100	100	0	0,0
Case Split	1	50	50	0	0,0
	2	30	30	0	0,0
	3	20	20	0	0,0
Gesamt		500	500	0	0,0

Über alle 500 getesteten Aufgaben hinweg haben sämtliche drei Prüfungen bestanden. Die Fehlerrate beträgt 0,%. Dieses Ergebnis stützt die korrekte Implementierung des Deletion-Algorithmus für die in der Plattform verwendeten Aufgabentypen und Variablenmengen. Die dreistufige Prüfstruktur — Erfüllbarkeitsentscheidung, konkrete Belegungen und Freiheitsgrade — stellt sicher, dass nicht nur die Gesamtentscheidung, sondern auch die differenzierte Marginalinformation pro Variable geprüft wird, die für das Antwortschema und die Feedbackgenerierung benötigt wird. Einschränkend ist festzuhalten, dass die Verifikation keinen formalen Korrektheitsbeweis darstellt, sondern auf dem Abgleich mit einer Referenzimplementierung über eine endliche Testmenge beruht. Die 500 Testfälle stammen ausschließlich aus dem Aufgabengenerator und decken damit genau den Bereich ab, den die Plattform im Betrieb erzeugt. Konstellationen außerhalb der vorgesehenen Schwierigkeitsparameter — etwa Prämissenmengen mit mehr als fünf Variablen oder manuell konstruierte Grenzfälle — wurden nicht systematisch getestet. Die Vertrauenswürdigkeit der Referenz beruht darauf, dass SymPy eine breit eingesetzte und aktiv gewartete Bibliothek für symbolisches Rechnen ist (SymPy Development Team 2025), deren Erfüllbarkeitsprüfung ihrerseits auf etablierten Algorithmen basiert.

6.3 Funktionale Überprüfung

In diesem Abschnitt wird die umgesetzte Funktionalität der Plattform systematisch gegen die in Abschnitt 1.3 formulierte Zielsetzung abgeglichen. Tabelle 6 fasst den Umsetzungsstand der einzelnen Ziele zusammen.

Tabelle 6: Abgleich der umgesetzten Funktionalität mit der Zielsetzung aus 1.3

Ziel	Status	Anmerkung
Automatische Erzeugung von Aufgaben zum logischen Schließen für verschiedene Aufgabentypen	✓	Umgesetzt für zwei Aufgabentypen (Direct Inference, Case Split) über einen Generate-and-Filter-Ansatz mit Vorgenerierungsstrategie (Abschnitt 4.2, 5.2.1, 5.3.4).
Abbildung unterschiedlicher Schwierigkeitsgrade für einen schrittweisen Kompetenzaufbau	✓	Regelbasierte Konfiguration mit vier Leveln für Direct Inference und drei für Case Split (Abschnitt 4.1).
Algorithmische Bestimmung der korrekten Lösung pro Variable (wahr, falsch oder nicht eindeutig festgelegt)	✓	Deletion-Algorithmus auf Basis boolescher Tabellen, verifiziert gegen SymPy mit 0 % Fehlerrate über 500 Aufgaben (Abschnitt 4.3).
Handhabbare und eindeutig auswertbare Eingabestruktur	✓	Dreistufiges Antwortschema (<i>wahr, falsch, kein konkreter Schluss möglich</i>), umgesetzt als Radio-Button-Gruppe pro Variable (Abschnitt 4.4, 5.4.2).
Automatische Auswertung der Nutzerantworten	✓	Variablenweiser Vergleich mit Solver-Ergebnis; nur vollständig korrekte Versuche zählen als Erfolg (Abschnitt 4.4.3, 5.3.3).
Unmittelbare, ergebnisbezogene Rückmeldung bei fehlerhaften Antworten	●	Prototypisch umgesetzt. Bei unnötiger Festlegung wird ein Gegenbeispiel konstruiert; bei den übrigen Fehlertypen beschränkt sich die Rückmeldung auf die Nennung des korrekten Werts (Abschnitt 4.5, 5.2.3).
Didaktische Strukturierung des Übungsprozesses mit nachvollziehbarem Kompetenzaufbau	✓	Geführter Lernpfad mit Checkpoint-Darstellung, serienbasiertem Levelaufstieg und Freischaltlogik zwischen Aufgabentypen; ergänzt um einen freien Übungsmodus nach Abschluss (Abschnitt 4.6, 5.4.3 f.).

Fortsetzung auf der nächsten Seite

Ziel	Status	Anmerkung
Sichtbarmachung des individuellen Lernstands	✓	Persistente Fortschrittsspeicherung, Statistikübersicht, Erfolgsserie und Highscore über das UserProgress-Modell (Abschnitt 4.6.3, 5.3.2, 5.4.5).
Unterstützung durch spielerische Elemente in begrenztem Umfang	●	Umgesetzt als Levelsystem, Erfolgsserie und Avatar-Editor. Die motivationale Wirkung wurde nicht empirisch geprüft (Abschnitt 4.6.3, 5.4.3, 5.4.5).
Fachliche Einrahmung durch begleitende Inhalte	✓	Grundlagenseite mit Wahrheitstabellen, Umformungsregeln und Beispielen sowie Referenzseite zu den zugrunde liegenden Lehrmaterialien (Abschnitt 4.6.3, 5.4.5).

Die Plattform deckt den in der Zielsetzung beschriebenen Funktionsumfang weitgehend ab. Zwei der neun Teilziele sind als prototypisch einzustufen: Die Feedbackkomponente liefert bei einem der drei Fehlertypen (unnötige Festlegung) ein konstruiertes Gegenbeispiel, beschränkt sich bei den übrigen Fehlertypen jedoch auf die Nennung des korrekten Werts (vgl. Abschnitt 4.5). Die motivationsfördernden Elemente sind technisch implementiert, ihre tatsächliche Wirkung auf das Lernverhalten wurde jedoch nicht untersucht. Die zentrale Einschränkung der funktionalen Überprüfung liegt darin, dass sie ausschließlich die technische Umsetzung bewertet, nicht die didaktische Wirksamkeit. Ob die Plattform ihren übergeordneten Zweck — die wirksame Unterstützung des Übungsprozesses — tatsächlich erfüllt, lässt sich auf Grundlage der vorliegenden Evaluation nicht beurteilen, da keine Erprobung mit Studierenden stattgefunden hat (vgl. Abschnitt 1.4). Diese Lücke stellt zugleich den naheliegendsten Ansatzpunkt für eine weiterführende Untersuchung dar (vgl. Kapitel 7.4).

7 Diskussion und Fazit

Die vorliegende Arbeit hatte zum Ziel, eine digitale Übungsplattform für das logische Schließen in der Aussagenlogik zu konzipieren und prototypisch umzusetzen. Die Evaluation in Kapitel 6 hat gezeigt, dass die technischen Kernkomponenten funktionsfähig sind und die in der Zielsetzung formulierten Anforderungen weitgehend erfüllt werden. Im Folgenden werden die getroffenen Entwurfsentscheidungen kritisch reflektiert (Abschnitt 7.1), die Grenzen der Arbeit benannt (Abschnitt 7.2), die Ergebnisse zusammenfassend eingeordnet (Abschnitt 7.3) und Ansatzpunkte für eine Weiterentwicklung skizziert (Abschnitt 7.4).

7.1 Reflexion der Entwurfsentscheidungen

Rückblickend lassen sich mehrere Entwurfsentscheidungen identifizieren, deren Tragfähigkeit sich erst im Verlauf der Umsetzung und Evaluation vollständig beurteilen ließ.

Generate-and-Filter-Ansatz

Der Generate-and-Filter-Ansatz hat den Vorteil, eine hohe Aufgabenvielfalt zu erzeugen. In der Praxis zeigt sich jedoch ein Skalierungsproblem: Mit zunehmender Anzahl und Strenge der Validierungskriterien sinkt die Akzeptanzrate deutlich. Die Evaluation hat diesen Zusammenhang quantifiziert: Während in Direct Inference Level 1 noch jeder fünfte Kandidat akzeptiert wird (19,4%), sinkt die Akzeptanzrate bei Case Split Level 3 auf 0,01 %, was einer mittleren Generierungsdauer von knapp einer Minute pro Aufgabe entspricht (vgl. Tabelle 4). Die Vorgenerierungsstrategie kompensiert dieses Problem im Betrieb wirkungsvoll, sodass die Nutzer keine Wartezeiten erfahren. Für den Rahmen eines Prototyps hat sich die Kombination aus zufälliger Erzeugung und nachgelagerter Filterung bewährt. Bei einer Erweiterung der Plattform um größere Variablen- und Prämissenmengen oder zusätzliche Aufgabentypen könnte es sinnvoll sein, ergänzend einen konstruktiven Ansatz zu untersuchen, der Aufgaben gezielt aus gewünschten Eigenschaften heraus aufbaut. Ob ein solcher Ansatz die in Abschnitt 4.2.4 vermutete Einschränkung der Aufgabenvielfalt tatsächlich mit sich bringt, ließe sich erst durch einen konkreten Vergleich klären.

Deletion-Algorithmus

Die Wahl des Deletion-Algorithmus als Lösungsverfahren war primär durch die Verfügbarkeit strukturierter Zwischenergebnisse motiviert, die für die Feedbackkomponente genutzt werden sollten. Im aktuellen Prototyp greift einer der drei Feedbacktypen – die Konstruktion von Gegenbeispielen bei unnötiger Festlegung – unmittelbar auf diese Zwischenergebnisse zurück, während die beiden übrigen Feedbacktypen auf einem einfachen Vergleich mit dem Solver-Ergebnis basieren. Das Potenzial der gespeicherten Bucket-Strukturen wird bislang nur für die Konstruktion von Gegenbeispielen ausgeschöpft. Sie bieten jedoch eine natürliche Grundlage für weiterführende Feedbackfunktionen wie etwa

schrittweise Erklärungen oder die Identifikation fehlerverursachender Prämissen, die ohne grundlegende Änderung am Lösungsverfahren realisierbar wären (vgl. Abschnitt 7.4). Die Verifikation gegen SymPy über 500 Aufgaben mit einer Fehlerrate von 0 % (vgl. Tabelle 5) stützt die Korrektheit der Implementierung für die vorgesehenen Variablenmengen, stellt jedoch keinen formalen Beweis dar.

Antwortschema

Das dreistufige Antwortschema hat sich als praktikabel erwiesen: Die Radio-Button-Eingabe erfordert keine Einarbeitungszeit und liefert eindeutig auswertbare Ergebnisse. Der Preis dieses Kompromisses liegt darin, dass der Lösungsweg selbst nicht Gegenstand der Bewertung ist. Fehler im Denkprozess, die zufällig zum richtigen Ergebnis führen, bleiben ebenso unerkannt wie systematische Fehlvorstellungen, die sich nur in den Zwischenschritten manifestieren. Für eine Übungsplattform, die auf wiederholtes Üben und Selbstkontrolle abzielt, ist diese Einschränkung vertretbar. Für ein System mit tiefergehendem diagnostischem Anspruch wäre sie hingegen problematisch.

Schwierigkeitsmodell

Das regelbasierte Schwierigkeitsmodell bietet den Vorteil der Transparenz und Reproduzierbarkeit und hat sich im Rahmen der prototypischen Umsetzung als praktikable Grundlage für die Levelstruktur erwiesen. Zugleich stützt es sich ausschließlich auf formale Parameter. Ob die daraus resultierende Abstufung der tatsächlich wahrgenommenen Schwierigkeit durch Studierende entspricht, ist eine offene Frage. Es ist durchaus denkbar, dass bestimmte Operatorkombinationen oder Prämissenstrukturen für Studierende schwieriger sind als andere, ohne dass sich dies in den gewählten Parametern widerspiegelt.

Feedbackkomponente

Die Feedbackkomponente verdient eine differenzierte Bewertung. Der Feedbacktyp der unnötigen Festlegung, bei dem ein konkretes Gegenbeispiel konstruiert wird, geht über eine Richtig-Falsch-Rückmeldung hinaus und bietet einen inhaltlichen Ansatzpunkt für die Reflexion des eigenen Fehlers. Für die beiden übrigen Fehlertypen beschränkt sich die Rückmeldung hingegen auf die Nennung des korrekten Werts. Gerade bei der falschen Annahme von Wahrheitswerten fehlt ein Hinweis darauf, auf welchem Weg der Variablenwert aus den Prämissen folgt. Diese Asymmetrie im Informationsgehalt der Feedbacktypen stellt den naheliegendsten Ansatzpunkt für eine Weiterentwicklung des Systems dar (vgl. Abschnitt 7.4).

7.2 Einschränkungen

Die Ergebnisse der vorliegenden Arbeit unterliegen mehreren Einschränkungen, die bei der Einordnung des Beitrags berücksichtigt werden müssen.

Die gravierendste Einschränkung besteht darin, dass die Plattform nicht mit Studierenden erprobt wurde. Es liegen keine Daten zur Lernwirksamkeit, zur Usability oder zur motivationalen Wirkung der implementierten Gamification-Elemente vor. Die Evaluation in Kapitel 6 beschränkt sich auf die technische Korrektheit und Leistungsfähigkeit der Kernkomponenten. Ob die Plattform ihren übergeordneten didaktischen Zweck – die wirksame Unterstützung des Übungsprozesses – in der Praxis tatsächlich erfüllt, lässt sich auf dieser Grundlage nicht beurteilen. Insbesondere die Annahme, dass ein serienbasierter Levelaufstieg das studentische Engagement fördert, stützt sich auf die Meta-Analyse von (Sailer und Homner 2020), nicht auf eine eigene empirische Überprüfung. Ob der gewählte Mechanismus in der Praxis motivierend wirkt oder einzelne Studierende durch das Zurücksetzen der Erfolgsserie eher frustriert, lässt sich ohne Nutzerstudie nicht beurteilen.

Eng damit verbunden ist die fehlende empirische Validierung des Schwierigkeitsmodells. Die Levelstruktur stützt sich auf formale Parameter (Variablenanzahl, Formeltiefe, Operatormenge), deren Korrespondenz mit der tatsächlich wahrgenommenen Schwierigkeit nicht untersucht wurde. Eine empirische Validierung – etwa durch die Analyse von Fehlerquoten und Bearbeitungszeiten pro Level über eine größere Studierendengruppe – könnte zeigen, ob die gewählte Abstufung den intendierten Kompetenzaufbau tatsächlich abbildet oder ob einzelne Level in ihrer Schwierigkeit angepasst werden sollten.

Die Solver-Verifikation stützt die korrekte Implementierung des Deletion-Algorithmus, stellt jedoch keinen formalen Korrektheitsbeweis dar. Die 500 Testfälle stammen ausschließlich aus dem Aufgabengenerator und decken damit den vorgesehenen Einsatzbereich ab. Konstellationen außerhalb der Schwierigkeitsparameter – etwa manuell konstruierte Grenzfälle oder Prämissenmengen mit mehr als fünf Variablen – wurden nicht systematisch getestet.

Schließlich ist die Arbeit auf zwei Aufgabentypen innerhalb der Aussagenlogik beschränkt, und die Plattform wurde ausschließlich als lokal ausführbarer Prototyp betrieben. Fragen des Mehrbenutzerbetriebs, der Skalierbarkeit und der institutionellen Integration sind daher offen und nicht Gegenstand der Evaluation.

7.3 Fazit

Die in Abschnitt 1.2 formulierte Problemstellung beschrieb den Bedarf nach einer digitalen Übungsumgebung, die ein systematisches Bearbeiten logischer Schlüsselaufgaben ermöglicht und dabei über statische Einzelaufgaben hinausgeht. Die vorliegende Arbeit hat gezeigt, dass es möglich ist, automatische Aufgabengenerierung, algorithmische Lösungsbestimmung, standardisierte Bewertung und eine strukturierte Lernumgebung zu einem geschlossenen Übungssystem zusammenzuführen, das diesen Anforderungen entspricht.

Die fachliche Korrektheit der Plattform ist algorithmisch abgesichert und nicht von probabilistischen Verfahren abhängig. Damit unterscheidet sich der gewählte Ansatz grundlegend von den in Abschnitt 3.2 diskutierten LLM-basierten Systemen, deren fachliche Zuverlässigkeit in formal-logischen Domänen nach aktuellem Forschungsstand nicht garantiert werden kann (Reddig, Arora und MacLellan 2025; Ji u. a. 2023). Zugleich bildet

die algorithmische Lösung eine belastbare Grundlage, auf der weiterführende Komponenten – etwa LLM-gestützte Feedbackgenerierung – aufsetzen könnten, ohne die fachliche Korrektheit zu gefährden. Die Evaluation hat bestätigt, dass der Deletion-Algorithmus für die vorgesehenen Variablenmengen korrekte Ergebnisse liefert und dass die Aufgabengenerierung trotz niedriger Akzeptanzraten über die Vorgenerierungsstrategie im Betrieb praktikabel ist (vgl. Abschnitt 6.1).

Der Beitrag der Arbeit liegt nicht in einzelnen Teilkomponenten allein, sondern in deren integrierter Zusammenführung zu einer funktionsfähigen Gesamtanwendung. Die Plattform stellt damit einen Prototyp dar, der zeigt, wie algorithmisch gesicherte Aufgabengenerierung und -bewertung mit einer didaktisch strukturierten Lernumgebung verbunden werden können. Ob dieses technische Fundament auch die intendierte didaktische Wirkung entfaltet, ist die zentrale offene Frage, deren Beantwortung eine empirische Erprobung mit Studierenden erfordert.

7.4 Ausblick

Aus dem Prototypcharakter der Anwendung und den in Abschnitt 7.2 benannten Einschränkungen ergeben sich mehrere Ansatzpunkte für eine Weiterentwicklung, die im Folgenden skizziert werden.

Empirische Evaluation und Validierung des Schwierigkeitsmodells.

Der naheliegendste nächste Schritt besteht in einer empirischen Erprobung der Plattform mit Studierenden. Eine solche Untersuchung sollte sowohl die Lernwirksamkeit als auch die Usability und die wahrgenommene Motivation adressieren. Dabei wäre insbesondere ein Vergleich mit einer Kontrollgruppe ohne Plattformnutzung aufschlussreich, wie er etwa bei der Evaluation von Logic-ITA durchgeführt wurde (Yacef 2005). Gleichzeitig könnten die im Betrieb anfallenden Daten – Fehlerquoten, Bearbeitungszeiten und Abbruchraten pro Level – genutzt werden, um das regelbasierte Schwierigkeitsmodell empirisch zu validieren und gegebenenfalls nachzujustieren.

Weiterentwicklung des Feedbacksystems.

Die in Abschnitt 7.1 beschriebene Asymmetrie im Informationsgehalt der Feedbacktypen stellt den naheliegendsten inhaltlichen Ansatzpunkt dar. Die im Lösungsprozess gespeicherten Bucket-Zwischenergebnisse des Solvers (vgl. Abschnitt 4.3) könnten stärker für die Feedbackgenerierung genutzt werden. So ließe sich beispielsweise identifizieren, welche Prämissen für die Ableitung einer bestimmten Variable entscheidend sind, oder schrittweise nachvollziehen, auf welchem Weg ein Variablenwert aus den Prämissen folgt. Darüber hinaus bietet der in Abschnitt 3.2 diskutierte hybride Ansatz eine Perspektive für die Weiterentwicklung: Die algorithmisch gesicherte Lösung des Solvers könnte als fachliche Grundlage dienen, auf der ein großes Sprachmodell natürlichsprachliche Erklärungen erzeugt. Die in der Literatur beschriebenen Risiken von Halluzinationen und

fachlich fehlerhaften Rückmeldungen (Ji u. a. 2023) ließen sich in einer solchen Architektur gezielt abmildern, da die inhaltliche Korrektheit nicht vom Sprachmodell abhängt, sondern durch den Solver gewährleistet bleibt. Die Umsetzung eines solchen hybriden Ansatzes würde jedoch eine sorgfältige Evaluation der erzeugten Rückmeldungen erfordern, um sicherzustellen, dass die natürlichsprachlichen Erklärungen die algorithmisch bestimmte Lösung korrekt und verständlich wiedergeben.

Betrieb und institutionelle Einbindung.

Für einen produktiven Einsatz im Lehrbetrieb wäre ein Deployment auf einem Server erforderlich, einschließlich einer Anbindung an die Authentifizierungsdienste des Hochschulrechenzentrums, sodass sich Studierende mit ihren bestehenden Zugangsdaten anmelden können. Die in Abschnitt 5.1 beschriebene Verwendung von SQLite müsste durch ein Datenbanksystem mit Unterstützung für parallele Schreibzugriffe ersetzt werden.

Erweiterung der Aufgabentypen.

Die Plattform unterstützt mit dem direkten Schließen und der Fallunterscheidung zwei Aufgabentypen, die sich an den zugrunde liegenden Lehrmaterialien orientieren. Eine Erweiterung um zusätzliche Aufgabentypen innerhalb der Aussagenlogik – etwa das Prüfen logischer Äquivalenzen oder das Vereinfachen von Formeln mithilfe von Umformungsregeln – würde den Übungsumfang vergrößern, ohne die bestehende Architektur grundlegend verändern zu müssen. Die Schichtentrennung zwischen Kernlogik und Vermittlungsschicht (vgl. Abschnitt 4.7.1) ist darauf ausgelegt, neue Aufgabentypen als zusätzliche Module in der Kernlogik zu ergänzen. Über die Aussagenlogik hinaus wäre langfristig auch eine Anpassung für andere Logikbereiche denkbar, etwa die Prädikatenlogik oder die Modallogik. Eine solche Erweiterung würde jedoch erhebliche Anpassungen am Solver, an der Aufgabengenerierung und am Eingabekonzept erfordern und geht deutlich über den Rahmen der vorliegenden Arbeit hinaus.

Literatur

- The Joint Task Force on Computing Curricula (Aug. 2023). *Computer Science Curricula 2023: Version Gamma*. Association for Computing Machinery, IEEE Computer Society, und Association for the Advancement of Artificial Intelligence. URL: <https://csed.acm.org/wp-content/uploads/2023/09/Version-Gamma.pdf>.
- Herman, Geoffrey L. u. a. (2012). „Describing the What and Why of Students’ Difficulties in Boolean Logic“. In: *ACM Transactions on Computing Education* 12.1, Article 3. DOI: 10.1145/2133797.2133800. URL: <https://lisakaczmarczyk.com/wp-content/uploads/2022/07/2012-T0CE-Boolean-misconceptions.pdf>.
- Donoghue, Gregory M. und John A. C. Hattie (2021). „A Meta-Analysis of Ten Learning Techniques“. In: *Frontiers in Education* 6, S. 581216. DOI: 10.3389/feduc.2021.581216. URL: <https://www.frontiersin.org/journals/education/articles/10.3389/feduc.2021.581216/full>.
- Shute, Valerie J. (März 2007). *Focus on Formative Feedback*. RR-07-11. Educational Testing Service. URL: https://myweb.fsu.edu/vshute/pdf/shute%202007_f.pdf.
- De Bruijn-Smolers, M. und F. R. Prinsen (2024). „Effective Student Engagement with Blended Learning: A Systematic Review“. In: *Heliyon* 10.23, e39439. DOI: 10.1016/j.heliyon.2024.e39439. URL: <https://www.sciencedirect.com/science/article/pii/S2405844024154709>.
- Sailer, Michael und Lisa Homner (2020). „The Gamification of Learning: A Meta-Analysis“. In: *Educational Psychology Review* 32, S. 77–112. DOI: 10.1007/s10648-019-09498-w. URL: <https://link.springer.com/article/10.1007/s10648-019-09498-w>.
- Dicheva, Darina u. a. (2015). „Gamification in Education: A Systematic Mapping Study“. In: *Educational Technology & Society* 18.3, S. 75–88. URL: <https://www.wssu.edu/profiles/dichevc/gamification-in-education-systematic-mapping-study.pdf>.
- Hofmann, R. (2026). *ILIAS-Kurs zur Veranstaltung Mathematik 1*. Interner Kursraum der Hochschule Karlsruhe. URL: https://ilias.h-ka.de/goto.php?target=crs_1052080&client_id=Hska (besucht am 30.03.2026).
- Huth, Michael und Mark Ryan (2004). *Logic in Computer Science: Modelling and Reasoning about Systems*. 2. Aufl. Cambridge University Press. ISBN: 9780521543101. URL: <https://www.cambridge.org/de/universitypress/subjects/computer-science/programming-languages-and-applied-logic/logic-computer-science-modelling-and-reasoning-about-systems-2nd-edition?format=PB&isbn=9780521543101>.
- Nieuwenhuis, Robert, Albert Oliveras und Cesare Tinelli (2006). „Solving SAT and SAT Modulo Theories: From an Abstract Davis–Putnam–Logemann–Loveland Procedure to DPLL(T)“. In: *Journal of the ACM* 53.6, S. 937–977. DOI: 10.1145/1217856.1217859.
- Díaz-Macías, Efraín und Serafín Moral (2023). „A Deletion Algorithm for the Marginal Problem in Propositional Logic Based on Boolean Arrays“. In: *Mathematics* 11.12, S. 2748. DOI: 10.3390/math11122748. URL: <https://www.mdpi.com/2227-7390/11/12/2748>.

- Yacef, Kalina (2005). „The Logic-ITA in the Classroom: A Medium Scale Experiment“. In: *International Journal of Artificial Intelligence in Education* 15.1, S. 41–62. URL: <https://telearn.hal.science/hal-00257107/document>.
- Geck, Gaetano u. a. (2018). „Introduction to Iltis: An Interactive, Web-Based System for Teaching Logic“. In: *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*. ACM, S. 141–146. DOI: 10.1145/3197091.3197095. URL: <https://dl.acm.org/doi/10.1145/3197091.3197095>.
- Nkambou, Roger u. a. (2023). „Learning Logical Reasoning Using an Intelligent Tutoring System: A Hybrid Approach to Student Modeling“. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Bd. 37. 13, S. 15930–15937. DOI: 10.1609/aaai.v37i13.26891. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/26891>.
- Inamdar, Amogh u. a. (2025). *LogicLearner: A Tool for the Guided Practice of Propositional Logic Proofs*. DOI: 10.48550/arXiv.2503.19280. arXiv: 2503.19280. URL: <https://arxiv.org/abs/2503.19280> (besucht am 22.03.2026).
- Reddig, Jennifer M., Arav Arora und Christopher J. MacLellan (2025). „Generating In-Context, Personalized Feedback for Intelligent Tutors with Large Language Models“. In: *International Journal of Artificial Intelligence in Education* 35, S. 3459–3500. DOI: 10.1007/s40593-025-00505-6. URL: <https://link.springer.com/article/10.1007/s40593-025-00505-6>.
- Ji, Ziwei u. a. (2023). „Survey of Hallucination in Natural Language Generation“. In: *ACM Computing Surveys* 55.12, S. 1–38. DOI: 10.1145/3571730. URL: <https://dl.acm.org/doi/10.1145/3571730>.
- Mirzadeh, Iman u. a. (2025). „GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models“. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=AjXkRZIVjB>.
- Gu, Jun u. a. (1997). „Algorithms for the Satisfiability (SAT) Problem: A Survey“. In: *Satisfiability Problem: Theory and Applications*. Bd. 35. DIMACS Series in Discrete Mathematics and Theoretical Computer Science. American Mathematical Society, S. 19–151. DOI: 10.1090/dimacs/035/02. URL: https://www.researchgate.net/profile/Benjamin-Wah/publication/290129291_Algorithms_for_Satisfiability_SAT_problem_a_survey/links/56a300fd08aeef24c5860a51/Algorithms-for-Satisfiability-SAT-problem-a-survey.pdf.
- SymPy Development Team (2025). *SymPy 1.14.0 Documentation*. URL: <https://docs.sympy.org/latest/> (besucht am 31.03.2026).
- Harris, Charles R. u. a. (Sep. 2020). „Array programming with NumPy“. In: *Nature* 585.7825, S. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: <https://doi.org/10.1038/s41586-020-2649-2> (besucht am 31.03.2026).
- Django Software Foundation (2025). *Django Documentation*. URL: <https://docs.djangoproject.com/en/6.0/> (besucht am 31.03.2026).
- Meta Platforms, Inc. (2025). *React v19.2 Documentation*. URL: <https://react.dev/> (besucht am 31.03.2026).
- Microsoft (2025). *TypeScript 6.0 Documentation*. URL: <https://www.typescriptlang.org/> (besucht am 31.03.2026).

- Tailwind Labs Inc. (2026). *Tailwind CSS v4.2 Documentation*. URL: <https://tailwindcss.com/> (besucht am 31.03.2026).
- Radix UI (2026). *Radix Primitives Documentation*. URL: <https://www.radix-ui.com/primitives/docs/overview/introduction> (besucht am 31.03.2026).
- DiceBear (2026). *DiceBear | Library to Generate Avatars & SVG API*. Zugegriffen am 31.03.2026. URL: <https://www.dicebear.com/> (besucht am 31.03.2026).
- Li, Shi (2019). *Probability Theory*. Lecture 5, CSE 632: Analysis of Algorithms II: Randomized Algorithms, University at Buffalo. Mitschrift von Kairui Wang. URL: <https://cse.buffalo.edu/~shil/courses/CSE632-Fall2019/Notes/5-Probability.pdf> (besucht am 31.03.2026).